



应用开发指南

文档版本 08

发布日期 2025-05-20

版权所有 © 海思技术有限公司2025。保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HISILICON、海思和其他海思商标均为海思技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受海思公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，海思公司对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

海思技术有限公司

地址：上海市青浦区虹桥港路2号101室 邮编：201721

网址：<https://www.hisilicon.com/cn/>

客户服务邮箱：support@hisilicon.com



前言

概述

本文用于指导开发人员基于现有模型、使用SVP ACL（Smart Vision Processing Advanced Computing Language）提供的C语言API库开发图像分析工具应用，用于实现目标识别、图像分类等功能。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
Hi3519D	V500
Hi3516D	V500
Hi3516C	V608
Hi3516C	V610

说明

- Hi3519DV500/Hi3516DV500以musl库工具链aarch64-v01c01-linux-musl-gcc描述为例，使用glibc库工具链时，请使用aarch64-v01c01-linux-gnu-gcc替换aarch64-v01c01-linux-musl-gcc即可。
- Hi3516CV610仅支持musl库的工具链arm-v01c02-linux-musleabi-gcc。
- 本文档在描述stub库使用时，统一使用acclib/libxx描述。Hi3519DV500/Hi3516DV500为64位芯片，stub库在acclib/lib64目录；Hi3516CV610为32位芯片，stub库在acclib/lib32目录。
- 本文如无特殊说明，Hi3519DV500与Hi3516DV500，Hi3516CV608与Hi3516CV610内容完全一致。

读者对象

本文档适用于基于SVP ACL接口进行应用开发的人员，通过本文档您可以达成：

- 了解SVP ACL的功能架构、基本概念以及接口的典型调用流程。



- 使用SVP ACL接口进行应用开发的基本流程和实现方法。
- 能够基于本文档中的样例，扩展进行其它应用的开发。

掌握以下经验和技能可以更好地理解本文档：

- 具备C++/C语言程序开发能力。
- 对机器学习、*图像分析方法*有一定的了解。

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
08	2025-05-20	第8次正式版本发布 “3.7.2 Proc信息说明”小节涉及修改
07	2025-03-28	第7次正式版本发布 “3.7.2 Proc信息说明”小节涉及修改 9.12小节中 svp_acl_md5_set_aipp_padding_params涉及修改。 删除MindStudio的相关描述



文档版本	发布日期	修改说明
06	2024-09-15	第6次正式版本发布 8.7小节，新增多次采集的场景注意事项 9.12.12、9.12.15.1小节涉及修改
05	2024-06-20	第5次正式版本发布 3.7.2、9.1、9.5.1小节涉及修改 9.9.1小节，新增说明 新增“9.12.31.1 svp_acl_query_status”
04	2024-04-28	第4次正式版本发布 8.3.5、9.12.12小节涉及修改
03	2024-03-31	第3次正式版本发布 3.7.2小节，修改ops_sel的描述 4.4.5小节，修改模型执行的输入/输出数据准备流程图 9.1小节，添加注意 10.1小节，添加说明
02	2024-03-15	第2次正式版本发布 添加Hi3516CV610的相关内容
01	2024-01-30	第1次正式版本发布 7.3.1.3、7.3.3.1、11.1小节中涉及修改。 新增8.3.5小节。 8.4和9.1小节涉及更新。 9.8小节中，svp_acl_mdl_set_input_aipp涉及修改。 9.12小节中“svp_acl_mdl_get_input_dims”、“svp_acl_mdl_get_input_data_type”、svp_acl_mdl_set_aipp_pixel_var_reci、svp_acl_aipp_info涉及修改。新增svp_acl_buffer_reuse_type、svp_acl_inout_reuse_type、svp_acl_mdl_get_reuse_buffer_type、svp_acl_mdl_get_input_reuse_type、svp_acl_mdl_get_output_reuse_type。svp_acl_mdl_get_input_default_stride、svp_acl_mdl_get_dynamic_hw新增须知。



文档版本	发布日期	修改说明
00B04	2023-09-15	第4次临时版本发布 3.7.2、4.2、4.4.1、4.4.3、4.4.5、8.5、8.6、11.4和11.5小节涉及修改 “9.7.9 svp_acl_rt_free_sec”、“9.12.2 svp_acl_data_type”、“9.12.12 svp_acl_prof_aicore_metrics”、“9.12.15.8 svp_acl_md1_get_input_dims”、“9.12.28.3 svp_acl_md1_set_aipp_csc_params”涉及修改
00B03	2023-06-30	第3次临时版本发布 3.7.2、4.4.1、4.4.3、4.4.5、8.5和8.6小节涉及修改 “9.12.2 svp_acl_data_type”、“9.12.12 svp_acl_prof_aicore_metrics”、“9.12.15.8 svp_acl_md1_get_input_dims”、“9.12.28.3 svp_acl_md1_set_aipp_csc_params”涉及修改
00B02	2023-05-10	第2次临时版本发布 前言添加说明 3.3.1、9.6.1、9.6.2小节补充说明 3.7.2小节，涉及更新 4.4.3、4.4.5，8.5、8.6、10.2.2小节涉及更新。
00B01	2023-03-10	第1次临时版本发布



目录

前言.....	i
1 使用约束.....	1
2 新手指引.....	2
2.1 文档结构总览.....	2
2.2 表达约定.....	2
2.2.1 接口命名规则.....	2
2.2.2 接口类别.....	3
3 简介.....	4
3.1 什么是 SVP ACL.....	4
3.2 基本概念.....	5
3.3 进程、线程、Device、Context、Stream 之间的关系.....	7
3.3.1 Device、Context、Stream 之间的关系.....	7
3.3.2 线程、Context、Stream 之间的关系.....	7
3.3.3 一个进程内多个线程间的 Context 迁移.....	8
3.3.4 默认 Context 和默认 Stream 的使用场景.....	9
3.3.5 多线程、多 stream 的性能说明.....	10
3.4 SVP ACL 内存申请使用说明.....	10
3.5 如何获取 Sample.....	11
3.6 如何查看日志.....	12
3.7 如何查看 Proc 信息.....	12
3.7.1 概述.....	12
3.7.2 Proc 信息说明.....	12
4 接口调用流程介绍.....	19
4.1 主要接口调用流程.....	19
4.2 运行管理资源申请.....	20
4.3 模型加载.....	22
4.4 模型执行.....	22
4.4.1 基本的模型执行流程.....	23
4.4.2 设置动态 Batch 和 Total_t.....	24
4.4.3 设置动态分辨率.....	25
4.4.4 设置动态 AIPP.....	26
4.4.5 准备模型执行的输入/输出数据.....	27



4.5 同步等待.....	32
4.5.1 多 Device 场景.....	33
4.5.2 多 Stream 场景.....	34
4.5.3 Callback 场景.....	35
4.6 运行管理资源释放.....	36
5 开发流程.....	38
6 准备环境.....	40
6.1 准备开发环境.....	40
6.2 准备运行环境.....	40
7 开发首个应用.....	42
7.1 开发场景分析.....	42
7.1.1 开发场景.....	42
7.1.2 场景分析.....	42
7.2 创建代码目录.....	42
7.3 开发应用.....	43
7.3.1 资源初始化.....	43
7.3.1.1 SVP ACL 初始化.....	43
7.3.1.2 运行管理资源申请（单进程+单线程+单 Stream）.....	43
7.3.1.3 模型推理资源申请.....	45
7.3.2 读入图片数据.....	46
7.3.3 模型推理.....	46
7.3.3.1 示例代码.....	47
7.3.4 运行管理资源释放与 SVP ACL 去初始化.....	48
7.4 编译及运行应用.....	48
8 开发典型功能点的介绍.....	51
8.1 Stream 管理.....	51
8.1.1 原理介绍.....	51
8.1.2 单线程单 Stream.....	51
8.1.3 单线程多 Stream.....	51
8.1.4 多线程多 Stream.....	52
8.2 同步等待.....	53
8.2.1 原理介绍.....	53
8.2.2 关于 Event 的同步等待.....	53
8.2.3 关于 Stream 内任务的同步等待.....	54
8.2.4 关于 Stream 间任务的同步等待.....	54
8.2.5 关于 Device 的同步等待.....	54
8.3 模型推理.....	55
8.3.1 单 Batch+固定 shape+单模型同步推理.....	55
8.3.2 异步推理+callback 回调处理.....	55
8.3.3 多模型推理.....	59
8.3.4 动态 APP.....	60



8.3.5 WorkBuf 内存复用.....	62
8.4 带 cache 属性的内存管理.....	64
8.5 推理使用模式识别 CPU 算子.....	65
8.6 任务优先级.....	67
8.7 Profiling 性能数据采集.....	68
9 SVP ACL API 参考.....	72
9.1 接口列表.....	72
9.2 系统配置.....	80
9.2.1 svp_acl_init.....	80
9.2.2 svp_acl_finalize.....	81
9.3 Device 管理.....	81
9.3.1 svp_acl_rt_set_device.....	82
9.3.2 svp_acl_rt_reset_device.....	82
9.3.3 svp_acl_rt_get_device.....	83
9.3.4 svp_acl_rt_get_run_mode.....	83
9.3.5 svp_acl_rt_get_device_count.....	84
9.4 Context 管理.....	84
9.4.1 svp_acl_rt_create_context.....	84
9.4.2 svp_acl_rt_destroy_context.....	85
9.4.3 svp_acl_rt_set_current_context.....	85
9.4.4 svp_acl_rt_get_current_context.....	86
9.5 Stream 管理.....	86
9.5.1 svp_acl_rt_create_stream.....	86
9.5.2 svp_acl_rt_destroy_stream.....	87
9.6 同步等待.....	87
9.6.1 svp_acl_rt_synchronize_device.....	87
9.6.2 svp_acl_rt_synchronize_stream.....	88
9.6.3 svp_acl_rt_subscribe_report.....	88
9.6.4 svp_acl_rt_launch_callback.....	89
9.6.5 svp_acl_rt_process_report.....	89
9.6.6 svp_acl_rt_unsubscribe_report.....	90
9.6.7 svp_acl_rt_set_op_wait_timeout.....	90
9.6.8 svp_acl_rt_create_event.....	91
9.6.9 svp_acl_rt_create_event_with_flag.....	91
9.6.10 svp_acl_rt_destroy_event.....	92
9.6.11 svp_acl_rt_record_event.....	92
9.6.12 svp_acl_rt_reset_event.....	93
9.6.13 svp_acl_rt_query_event.....	94
9.6.14 svp_acl_rt_query_event_wait_status.....	94
9.6.15 svp_acl_rt_synchronize_event.....	94
9.6.16 svp_acl_rt_event_elapsed_time.....	95
9.6.17 svp_acl_rt_stream_wait_event.....	95



9.7 内存管理.....	96
9.7.1 svp_acl_rt_malloc.....	96
9.7.2 svp_acl_rt_malloc_cached.....	97
9.7.3 svp_acl_rt_mem_flush.....	98
9.7.4 svp_acl_rt_mem_invalidate.....	98
9.7.5 svp_acl_rt_free.....	99
9.7.6 svp_acl_rt_malloc_host.....	99
9.7.7 svp_acl_rt_free_host.....	100
9.7.8 svp_acl_rt_malloc_sec.....	100
9.7.9 svp_acl_rt_free_sec.....	101
9.8 模型加载与执行.....	102
9.8.1 svp_acl_mdl_load_from_mem.....	102
9.8.2 svp_acl_mdl_execute.....	103
9.8.3 svp_acl_mdl_execute_async.....	104
9.8.4 svp_acl_mdl_unload.....	105
9.8.5 svp_acl_mdl_set_dynamic_batch_size.....	105
9.8.6 svp_acl_mdl_set_total_t.....	106
9.8.7 svp_acl_mdl_init_dump.....	107
9.8.8 svp_acl_mdl_set_dump.....	107
9.8.9 svp_acl_mdl_finalize_dump.....	109
9.8.10 svp_acl_mdl_set_config_opt.....	109
9.8.11 svp_acl_mdl_get_config_opt.....	110
9.8.12 svp_acl_mdl_load_with_config.....	110
9.8.13 svp_acl_mdl_set_model_config.....	111
9.8.14 svp_acl_mdl_set_input_aipp.....	112
9.8.15 svp_acl_mdl_set_aipp_by_input_index.....	114
9.8.16 svp_acl_mdl_set_output_aipp.....	115
9.8.17 svp_acl_mdl_get_first_aipp_info.....	116
9.8.18 svp_acl_mdl_get_last_aipp_info.....	117
9.8.19 svp_acl_mdl_get_input_aipp_type.....	117
9.8.20 svp_acl_mdl_get_output_aipp_type.....	118
9.8.21 svp_acl_mdl_get_dynamic_batch.....	118
9.8.22 svp_acl_mdl_set_dynamic_hw_size.....	119
9.9 Profiling 配置.....	120
9.9.1 功能及约束说明.....	120
9.9.2 svp_acl_prof_init.....	122
9.9.3 svp_acl_prof_start.....	123
9.9.4 svp_acl_prof_stop.....	123
9.9.5 svp_acl_prof_finalize.....	124
9.9.6 svp_acl_prof_model_subscribe.....	124
9.9.7 svp_acl_prof_model_unsubscribe.....	124
9.9.8 svp_acl_prof_get_op_desc_size.....	125



9.9.9	svp_acl_prof_get_op_num.....	125
9.9.10	svp_acl_prof_get_op_type_len.....	126
9.9.11	svp_acl_prof_get_op_type.....	126
9.9.12	svp_acl_prof_get_op_name_len.....	127
9.9.13	svp_acl_prof_get_op_name.....	128
9.9.14	svp_acl_prof_get_op_start.....	128
9.9.15	svp_acl_prof_get_op_end.....	129
9.9.16	svp_acl_prof_get_op_duration.....	130
9.9.17	svp_acl_prof_get_model_id.....	130
9.10	获取数据大小.....	131
9.10.1	svp_acl_data_type_size.....	131
9.11	SoC 扩展接口.....	131
9.11.1	svp_acl_ext_process_aicpu_task.....	131
9.11.2	svp_acl_ext_get_mdl_aicpu_task_num.....	132
9.11.3	svp_acl_ext_get_mdl_net_type.....	132
9.12	数据类型及其操作接口.....	132
9.12.1	svp_acl_error.....	132
9.12.2	svp_acl_data_type.....	139
9.12.3	svp_acl_float16.....	139
9.12.4	svp_acl_format.....	139
9.12.5	svp_acl_rt_context.....	140
9.12.6	svp_acl_rt_stream.....	140
9.12.7	svp_acl_rt_event.....	140
9.12.8	svp_acl_rt_event_status.....	140
9.12.9	svp_acl_rt_event_wait_status.....	140
9.12.10	svp_acl_rt_run_mode.....	140
9.12.11	svp_acl_mdl_io_dims.....	140
9.12.12	svp_acl_buffer_reuse_type.....	141
9.12.13	svp_acl_inout_reuse_type.....	141
9.12.14	svp_acl_prof_aicore_metrics.....	141
9.12.15	svp_acl_data_buffer.....	141
9.12.15.1	svp_acl_create_data_buffer.....	141
9.12.15.2	svp_acl_destroy_data_buffer.....	142
9.12.15.3	svp_acl_get_data_buffer_addr.....	143
9.12.15.4	svp_acl_get_data_buffer_size.....	143
9.12.15.5	svp_acl_get_data_buffer_stride.....	143
9.12.15.6	svp_acl_update_data_buffer.....	143
9.12.16	svp_acl_mdl_dataset.....	144
9.12.16.1	svp_acl_mdl_create_dataset.....	144
9.12.16.2	svp_acl_mdl_destroy_dataset.....	145
9.12.16.3	svp_acl_mdl_add_dataset_buffer.....	145
9.12.16.4	svp_acl_mdl_get_dataset_num_buffers.....	145



9.12.16.5	svp_acl_mdl_get_dataset_buffer.....	146
9.12.17	svp_acl_mdl_desc.....	146
9.12.17.1	svp_acl_mdl_create_desc.....	146
9.12.17.2	svp_acl_mdl_destroy_desc.....	146
9.12.17.3	svp_acl_mdl_get_desc.....	147
9.12.17.4	svp_acl_mdl_get_reuse_buffer_type.....	147
9.12.17.5	svp_acl_mdl_get_num_inputs.....	147
9.12.17.6	svp_acl_mdl_get_num_outputs.....	148
9.12.17.7	svp_acl_mdl_get_input_size_by_index.....	148
9.12.17.8	svp_acl_mdl_get_output_size_by_index.....	149
9.12.17.9	svp_acl_mdl_get_input_dims.....	149
9.12.17.10	svp_acl_mdl_get_output_dims.....	150
9.12.17.11	svp_acl_mdl_get_input_name_by_index.....	150
9.12.17.12	svp_acl_mdl_get_output_name_by_index.....	151
9.12.17.13	svp_acl_mdl_get_input_format.....	151
9.12.17.14	svp_acl_mdl_get_output_format.....	152
9.12.17.15	svp_acl_mdl_get_input_data_type.....	152
9.12.17.16	svp_acl_mdl_get_output_data_type.....	153
9.12.17.17	svp_acl_mdl_get_input_index_by_name.....	153
9.12.17.18	svp_acl_mdl_get_output_index_by_name.....	154
9.12.17.19	svp_acl_mdl_get_input_default_stride.....	154
9.12.17.20	svp_acl_mdl_get_output_default_stride.....	155
9.12.17.21	svp_acl_mdl_get_dynamic_hw.....	155
9.12.17.22	svp_acl_mdl_get_input_class_id_by_index.....	156
9.12.17.23	svp_acl_mdl_get_output_class_id_by_index.....	156
9.12.17.24	svp_acl_mdl_get_input_reuse_type.....	157
9.12.17.25	svp_acl_mdl_get_output_reuse_type.....	157
9.12.18	svp_acl_prof_config.....	158
9.12.18.1	svp_acl_prof_create_config.....	158
9.12.18.2	svp_acl_prof_destroy_config.....	159
9.12.19	svp_acl_prof_subscribe_config.....	159
9.12.19.1	svp_acl_prof_create_subscribe_config.....	160
9.12.19.2	svp_acl_prof_destroy_subscribe_config.....	160
9.12.20	svp_acl_mdl_config_attr.....	161
9.12.21	svp_acl_mdl_config_handle.....	161
9.12.21.1	svp_acl_mdl_create_config_handle.....	161
9.12.21.2	svp_acl_mdl_destroy_config_handle.....	162
9.12.22	svp_acl_aipp_format.....	162
9.12.23	svp_acl_aipp_input_format.....	163
9.12.24	svp_acl_mdl_io_dims.....	163
9.12.25	svp_acl_aipp_dims.....	163
9.12.26	svp_acl_aipp_padding_switch.....	163



9.12.27	svp_acl_aipp_resize_switch.....	164
9.12.28	svp_acl_mdl_aipp_type.....	164
9.12.29	svp_acl_aipp_info.....	164
9.12.30	svp_acl_mdl_aipp.....	165
9.12.30.1	svp_acl_mdl_create_aipp.....	165
9.12.30.2	svp_acl_mdl_set_aipp_input_format.....	165
9.12.30.3	svp_acl_mdl_set_aipp_csc_params.....	166
9.12.30.4	svp_acl_mdl_set_aipp_rbuw_swap_switch.....	168
9.12.30.5	svp_acl_mdl_set_aipp_ax_swap_switch.....	168
9.12.30.6	svp_acl_mdl_set_aipp_src_image_size.....	169
9.12.30.7	svp_acl_mdl_set_aipp_scf_params.....	169
9.12.30.8	svp_acl_mdl_set_aipp_crop_params.....	171
9.12.30.9	svp_acl_mdl_set_aipp_padding_params.....	172
9.12.30.10	svp_acl_mdl_set_aipp_dtc_pixel_mean.....	173
9.12.30.11	svp_acl_mdl_set_aipp_dtc_pixel_min.....	174
9.12.30.12	svp_acl_mdl_set_aipp_pixel_var_reci.....	175
9.12.30.13	svp_acl_mdl_destroy_aipp.....	176
9.12.31	svp_acl_status.....	176
9.12.31.1	svp_acl_query_status.....	176
10	SVP ACL 样例使用指导.....	178
10.1	总体说明.....	178
10.2	基于 Caffe ResNet-50 网络实现图片分类（同步推理）.....	178
10.2.1	样例介绍.....	178
10.2.1.1	获取样例.....	178
10.2.1.2	功能描述.....	178
10.2.1.3	原理介绍.....	179
10.2.1.4	目录结构.....	180
10.2.2	编译及运行应用.....	181
10.3	基于 Caffe ResNet-50 网络实现图片分类（异步推理）.....	183
10.3.1	样例介绍.....	183
10.3.1.1	获取样例.....	183
10.3.1.2	功能描述.....	183
10.3.1.3	原理介绍.....	184
10.3.1.4	目录结构.....	185
10.3.2	编译及运行应用.....	186
11	FAQ.....	190
11.1	SVP_NPU 离线模型保护.....	190
11.2	第一次推理耗时长的问题.....	190
11.3	系统出现 OOM 异常处理.....	190
11.4	任务优先级调度使用约束.....	190
11.5	任务优先级抢占使用约束.....	191



插图目录

图 3-1 逻辑架构图.....	5
图 4-1 接口调用流程图.....	19
图 4-2 运行管理资源申请流程.....	21
图 4-3 模型加载流程.....	22
图 4-4 基本的模型推理流程.....	23
图 4-5 设置动态分辨率/动态 AIPP 流程.....	26
图 4-6 模型输入/输出数据.....	27
图 4-7 模型执行的输入/输出数据准备流程.....	28
图 4-8 <code>svp_acl_md5_dataset</code> 类型与 <code>svp_acl_data_buffer</code> 类型的关系.....	29
图 4-9 模型输入/输出数据排布（通道为 2，batch 为 2 示意图）.....	30
图 4-10 YUV420SP/YUV420SP 数据排布（通道为 2，frame 为 2 示意图）.....	31
图 4-11 检测网阈值输入数据排布.....	31
图 4-12 检测框结果数据排布（通道为 2，chn 为 2 示意图）.....	32
图 4-13 同步等待流程多 Device 场景.....	33
图 4-14 同步等待流程多 Stream 场景.....	34
图 4-15 同步等待流程 Callback 场景模型推理.....	35
图 4-16 运行管理资源释放流程.....	36
图 5-1 开发流程（不包含数据预处理）.....	38
图 10-1 Sample 示例.....	179
图 10-2 Sample 示例.....	184



表格目录

表 3-1 概念介绍.....	5
表 3-2 Sample 列表.....	11
表 7-1 头文件与库文件的对应关系.....	49
表 9-1 输出图片宽、高计算公式.....	113
表 9-2 返回码列表.....	133
表 9-3 图像输入维度定义规则.....	149
表 9-4 图像输入数据类型定义规则.....	152
表 10-1 Sample 中的关键功能点.....	179
表 10-2 Sample 中的关键功能点.....	184



1 使用约束

- 不支持使用fork函数创建多个进程，且在进程中调用SVP ACL接口的场景，否则进程运行时会报错或者卡死。
- 对于创建类接口（例如：`svp_acl_rt_create_context`、`svp_acl_rt_create_stream`、`svp_acl_create_data_buffer`等），用户调用该类接口创建对应的资源后，资源使用完成后，建议及时调用对应的销毁类接口（例如：`svp_acl_rt_destroy_context`、`svp_acl_rt_destroy_stream`、`svp_acl_destroy_data_buffer`等），否则，程序可能会异常。
- 对于销毁类接口（例如：`svp_acl_rt_destroy_context`、`svp_acl_rt_destroy_stream`、`svp_acl_rt_free`、`svp_acl_destroy_data_buffer`等），用户调用该类接口后，不能继续使用已释放或销毁的资源，建议用户调用销毁类接口后，将相关资源设置为无效值（例如，置为NULL）。
- 一个Device上默认最多只能支持64个用户进程。



2 新手指引

2.1 文档结构总览

本文档分为以下几章：

- **前言**：介绍文档概述和读者对象。
- **使用约束**：介绍SVP ACL的总体使用约束。
- **新手指引**：介绍文档结构、接口命名规则。
- **简介**：介绍SVP（Smart Vision Platform）ACL（Advanced Computing Language）的基础知识，包括功能、基本概念、各概念之间的关系、如何查看日志等。
- **接口调用流程介绍**：介绍各场景下的SVP ACL接口调用流程。
- **开发流程**：介绍使用SVP ACL提供的接口开发应用的基本步骤。
- **准备环境**：介绍准备开发环境和板端环境时需要参考的文档。
- **开发首个应用**：以开发图片分类应用（不包含抠图、缩放、解码等数据预处理的开发）为例，按照开发流程，结合示例代码，介绍各步骤的基本原理。
- **开发典型功能点的介绍**：SVP ACL各功能点的详细介绍。
- **SVP ACL API参考**：介绍API的功能、原型、参数等。
- **SVP ACL样例使用指导**：介绍如何使用SVP ACL提供的样例。

2.2 表达约定

2.2.1 接口命名规则

接口命名同时满足如下规则：

1. 规则1：`svp_acl+接口类别缩写+操作动词+对象`
2. 规则2：操作动词和对象均采用小写



2.2.2 接口类别

接口类别	缩写	描述
runtime	rt	表示运行管理类的接口。
AIPP	aipp	表示aipp(AI Preprocessing/Postprocessing)类的接口
model	mdl	表示模型推理类的接口

注：

1. 缩写原则上不超过4个字母
2. 在接口命名中，如果类别与操作对象重叠时，操作动词后的对象将省略。

如：svp_acl_mdload_from_mem，表示model类接口，这个接口表示含义是load model from mem，因此在接口命名中Load后面 mdl将被省略。



3 简介

本文用于指导开发人员基于现有模型、使用SVP ACL (Smart Vision Processing Advanced Computing Language) 提供的C语言API库开发图像分析工具应用，用于实现目标识别、图像分类等功能。

3.1 什么是 SVP ACL

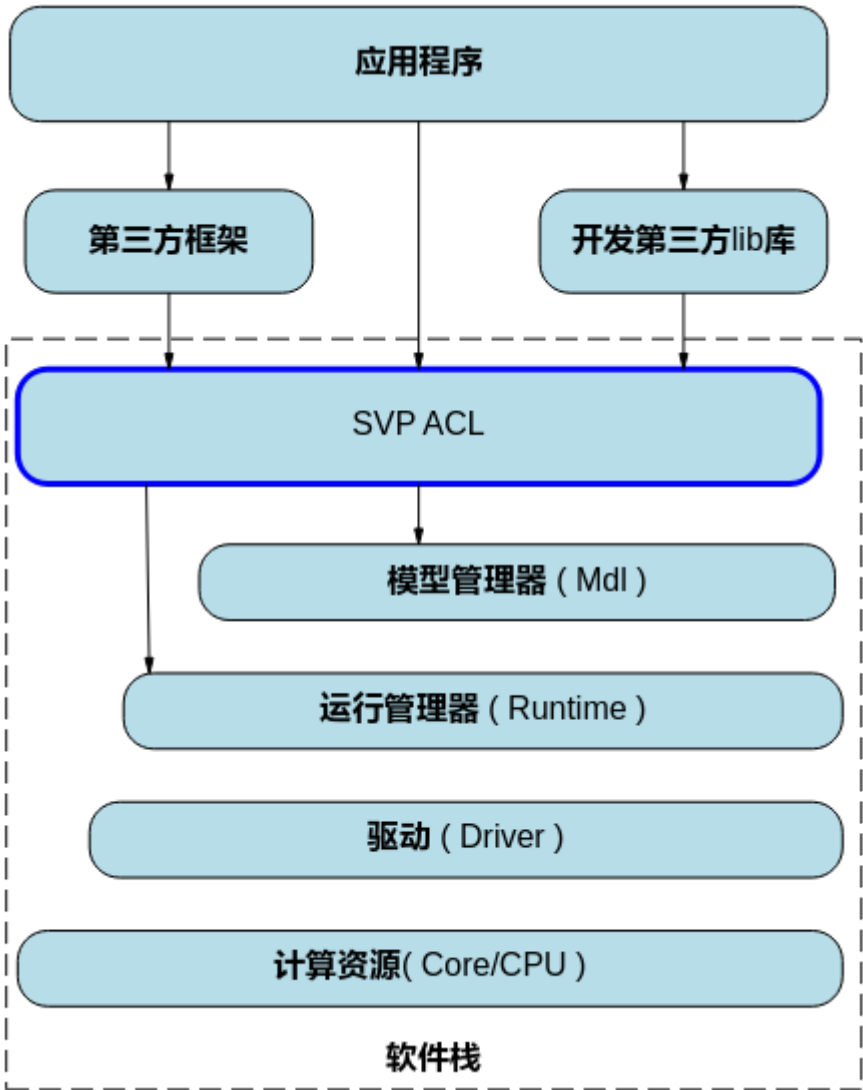
SVP ACL (Smart Vision Processing Advanced Computing Language) 提供Device管理、Context管理、Stream管理、内存管理、模型加载与执行等C语言API库供用户开发图像分析工具应用，用于实现目标识别、图像分类等功能。用户可以通过第三方框架调用SVP ACL接口，以便使用SoC的计算能力；用户还可以使用SVP ACL封装实现第三方lib库，以便提供SoC的运行管理、资源管理能力。

在运行应用时，SVP ACL调用mdl管理器提供的接口实现模型的加载与执行、调用运行管理器的接口实现Device管理/Context管理/Stream管理/内存管理等。

计算资源层是SoC的硬件算力基础，主要完成图像分析工具的矩阵相关计算、完成控制算子/标量/向量等通用计算和执行控制功能、完成图像和视频数据的预处理，为图像分析工具计算提供了执行上的保障。



图 3-1 逻辑架构图



3.2 基本概念

表 3-1 概念介绍

概念	描述
同步/异步	本文中提及的同步、异步是站在调用者和执行者的角度，在当前场景下，若在板端环境调用接口后不等待Device执行完成再返回，则表示板端环境的调度是异步的；若在板端环境调用接口后需等待Device执行完成再返回，则表示板端环境的调度是同步的。
进程/线程	本文中提及的进程、线程，若无特别注明，则表示板端环境上的进程、线程。
Device	Device表示板端环境上的图像分析引擎。



概念	描述
Context	Context作为一个容器，管理了所有对象（包括Stream、设备内存等）的生命周期。不同Context的Stream之间、不同的Context之间是完全隔离的，无法建立同步等待关系。 Context分为两种： ● 默认Context：调用svp_acl_rt_set_device接口指定用于运算的Device时，系统会自动隐式创建一个默认Context，一个Device对应一个默认Context，默认Context不能通过svp_acl_rt_destroy_context接口来释放。 ● 显式创建的Context：推荐，在进程或线程中调用svp_acl_rt_create_context接口显式创建一个Context。
Stream	Stream用于维护一些异步操作的执行顺序，确保按照应用程序中的代码调用顺序在Device上执行。 Stream分两种： ● 默认Stream：调用svp_acl_rt_set_device接口指定用于运算的Device时，系统会自动隐式创建一个默认Stream，一个Device对应一个默认Stream，默认Stream不能通过svp_acl_rt_destroy_stream接口来释放。 ● 显式创建的Stream：推荐，在进程或线程中调用svp_acl_rt_create_stream接口显式创建一个Stream。
Event (Hi3519DV500/ Hi3516DV500/ Hi3516CV610不支持)	支持调用SVP ACL接口同步Stream之间的任务。 例如：若stream2的任务依赖stream1的任务，想保证stream1中的任务先完成，这时可创建一个Event，并将Event插入到stream1中，并执行stream2的任务前，先同步等待Event完成。
AIPP	AIPP用于在<i>模式识别</i>Core上完成图像预处理，包括色域转换（转换图像格式）、图像归一化（减均值/乘系数）和抠图（指定抠图起始点，抠出<i>图像分析工具</i>需要大小的图片）等。 AIPP区分为静态AIPP和动态AIPP。只能选择静态AIPP或动态AIPP方式来处理图片，不能同时配置静态AIPP和动态AIPP两种方式。 ● 静态AIPP：模型转换时设置AIPP模式为静态，同时设置AIPP参数，模型生成后，AIPP参数值被保存在离线模型（*.om）中，每次模型推理过程采用固定的AIPP预处理参数（无法修改）。 ● 动态AIPP：模型转换时仅设置AIPP模式为动态，每次模型推理前，根据需求，在执行模型前设置动态AIPP参数值，然后在模型执行时可使用不同的AIPP参数。可调用svp_acl_mdl_aipp数据类型下的操作接口设置动态AIPP参数值。 ● 如果使用AIPP功能，多Batch情况下共用同一份AIPP参数。

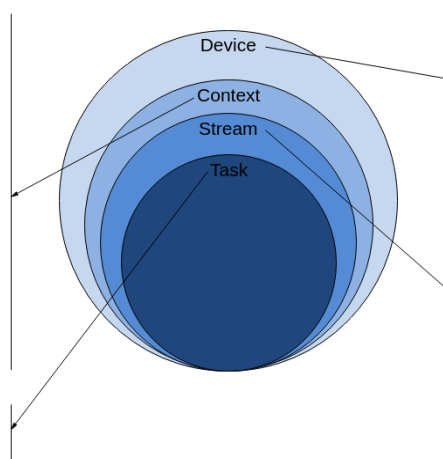
概念	描述
动态Batch/动态分辨率	在某些场景下，模型每次输入的Batch数或分辨率是不固定的，如检测出目标后再执行目标识别网络，由于目标个数不固定导致目标识别网络输入BatchSize不固定。 - 动态Batch：用户执行推理时，其Batch数是动态可变的。 - 动态分辨率：用户执行推理时，每张图片的分辨率H*W是动态可变的；如果和动态Batch一起使用，则一次多Batch推理的多张图片需要使用相同的分辨率。
动态维度（ND格式）	为了支持Transformer等网络在输入格式的维度不确定的场景，需要支持ND格式下任意维度的动态设置。 ND表示支持任意格式，当前N≠4。
通道	在RGB色彩模式下，图像通道就是指单独的红色R、绿色G、蓝色B部分。也就是说，一幅完整的图像，是由红色绿色蓝色三个通道组成的，它们共同作用产生了完整的图像。

3.3 进程、线程、Device、Context、Stream 之间的关系

各基本概念的介绍请参见[基本概念](#)。

3.3.1 Device、Context、Stream 之间的关系

- Context在Device下，一个Context一定属于一个唯一的Device。
- Context分隐式创建和显式创建。
- 隐式创建的Context（即默认Context），生命周期始于调用sva_acl_rt_set_device接口，终止于调用sva_acl_rt_reset_device接口使引用计数为零时。隐式Context只会被创建一次，调用sva_acl_rt_set_device接口重复指定同一个Device，只增加隐式创建的Context的引用计数。
- 显式创建的Context，生命周期始于调用sva_acl_rt_create_context接口，终止于调用acl_svp_rt_destroy_context接口。
- Context与用户线程绑定，一个用户线程对应一个Context，可调用sva_acl_rt_set_current_context接口绑定。隐式创建的Context不需要调用sva_acl_rt_set_current_context接口指定。
- 进程内的Context是共享的，可以通过sva_acl_rt_set_current_context进行切换。
- Device上真正的执行体，从属于Stream。
- 用户编程不感知。



- 用户指定计算设备，生命周期源于首次调用sva_acl_rt_set_device接口。
- 每次调用sva_acl_rt_set_device接口，系统会进行引用计数加1；调用sva_acl_rt_reset_device接口，系统会进行引用计数减1。
- 当引用计数减为零时，在本进程中Device上的资源不可用。
- Device上的执行流，在同一个Stream中的任务执行严格保序。
- Stream分隐式创建和显式创建。
- 每个Context都会包含一个默认的Stream，这个属于隐式创建，隐式创建的stream生命周期同归属的Context。
- 用户可以显式创建stream，显式创建的stream生命周期始于调用sva_acl_rt_create_stream，终止于调用sva_acl_rt_destroy_stream接口。显式创建的stream归属的Context被销毁或生命周期结束后，会影响该stream的使用，虽然此时stream没有被销毁，但不可再用。

须知

除了正常的模型推理是Task之外，Stream同步、Callback任务、Event记录、Event等待都是Task，所以在执行这些操作时请保证Stream上的节点数足够。

3.3.2 线程、Context、Stream 之间的关系

- 一个用户线程一定会绑定一个Context，所有Device的资源使用或调度，都必须基于Context。

- 一个线程中当前会有一个唯一的Context在用，Context中已经关联了本线程要使用的Device。
- 可以通过[svp_acl_rt_set_current_context](#)进行Device的快速切换。示例代码如下，仅供参考，不可以直接拷贝编译运行：

```
...
svp_acl_rt_create_context(&ctx1, 0);
svp_acl_md_execute(md1, input1, output1);
svp_acl_rt_create_context(&ctx2, 1);

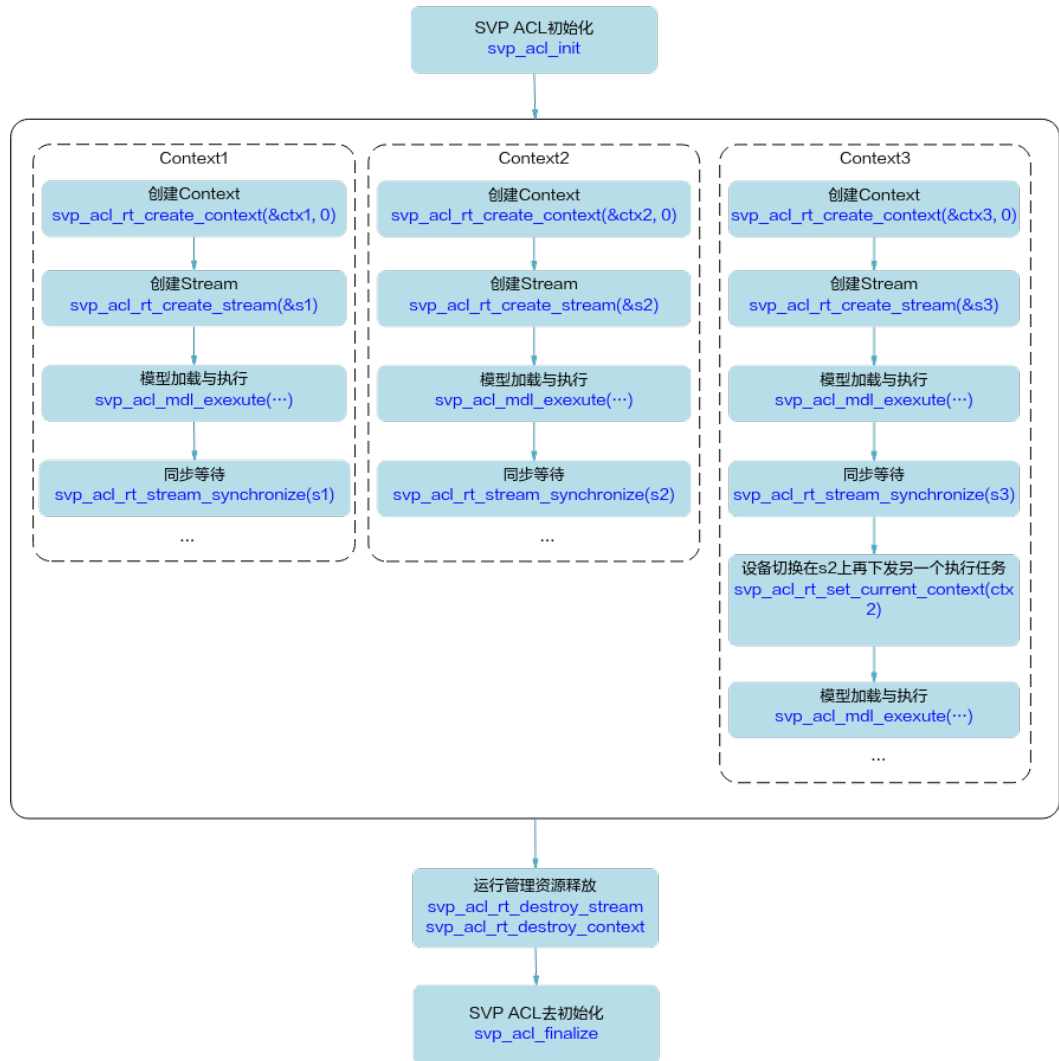
/*在当前线程中，创建ctx2后，当前线程对应的Context切换为ctx2，对应在Device 1进行后续
的计算任务，本例中将在Device 1上进行md2的执行调用 */
svp_acl_md_execute(md2, input2, output2);
svp_acl_rt_set_current_context(ctx1);

/*在当前线程中，通过Context切换，使后续模型计算任务在对应的Device 0上进行*/
svp_acl_md_execute(md3, input3, output3);
...
```

- 一个线程中可以创建多个Stream，不同的Stream上计算任务是可以并行执行；多线程场景下，也可以每个线程创建一个Stream，线程之间的Stream在Device上相互独立，每个Stream内部的任务是按照Stream下发的顺序执行。
- 多线程的调度依赖于运行应用的操作系统调度，Device侧多Stream调度，由Device上调度组件进行调度。

3.3.3 一个进程内多个线程间的 Context 迁移

- 一个进程中可以创建多个Context，但一个线程同一时刻只能使用一个Context。
- 线程中创建的多个Context，线程缺省使用最后一次创建的Context。
- 进程内创建的多个Context，可以通过[svp_acl_rt_set_current_context](#)设置当前需要使用的Context。



3.3.4 默认 Context 和默认 Stream 的使用场景

- Device上执行操作下发前，必须有Context和Stream，这个Context、Stream可以显式创建，也可以隐式创建。隐式创建的Context、Stream就是默认Context、默认Stream。
默认Stream作为接口入参时，直接传NULL。
- 默认Context不允许用户执行[svp_acl_rt_get_current_context](#)或[svp_acl_rt_set_current_context](#)操作，也不允许执行[svp_acl_rt_destroy_context](#)操作。
- 默认Context、默认Stream一般适用于简单应用，用户仅仅需要一个Device的计算场景下。多线程应用程序建议全部使用显式创建的Context和Stream。

示例代码如下，仅供参考，不可以直接拷贝编译运行：

```

...
svp_acl_init(...);
svp_acl_rt_set_device(0);

/*已经创建了一个default ctx，并且在当前线程可用*/
...
  
```



```
/*在推理过程如果没有记录的default stream, 会在default ctx中创建了一个default stream, 并且在当前线程可用*/
svp_acl_md1_execute_async(md1,input1,output1,NULL); //最后一个NULL表示在default stream上执行模型mdl1
svp_acl_md2_execute_async(md2,input2,output2,NULL); //最后一个NULL表示在default stream上执行模型mdl2
svp_acl_rt_synchronize_stream(NULL);

/*等待计算任务全部完成（mdl1、mdl2执行结束），用户根据需要获取计算任务的输出结果*/
...
svp_acl_rt_reset_device(0); //释放计算设备0，对应的default ctx及default stream生命周期也终止
```

3.3.5 多线程、多 stream 的性能说明

- 线程调度依赖运行的操作系统，Stream上下发了任务后，Stream的调度由Device的调度单元调度，但如果一个进程内的多Stream上的任务在Device存在资源争抢的时候，性能可能会比单Stream低。
- 当前芯片有不同的执行部件，如模式识别Core、模式识别CPU等，对应使用不同执行部件的任务，建议多Stream的创建按照算子执行引擎划分。
- 单线程多Stream与多线程多Stream（进程属于多线程，每个线程中一个Stream）性能上哪个更优，具体取决于应用本身的逻辑实现，一般来说前者性能略好，原因是相对后者，应用层少了线程调度开销。

3.4 SVP ACL 内存使用说明

用户内存管理有两种管理方式：

1. 独立内存管理，根据需要单独申请所需的内存，内存不做拆分或者二次分配。
2. 内存池管理内存，用户一次性申请一块较大内存，并在使用时从这块较大内存中二次分配所需内存。

在内存二次分配时，使用如下接口从内存池申请对应内存，由于接口对申请的内存地址、大小有约束，在内存池管理时，需要关注，否则容易出现内存越界。

接口	用途	输入内存/输出内存
svp_acl_rt_malloc	申请Device上的内存，同步接口。	• 使用svp_acl_rt_malloc接口申请的内存，需要通过svp_acl_rt_free接口释放内存。 • 频繁调用svp_acl_rt_malloc接口申请内存、调用svp_acl_rt_free接口释放内存，会损耗性能，建议用户提前做内存预先分配或二次管理，避免频繁申请/释放内存。



接口	用途	输入内存/输出内存
svp_acl_rt_malloc_host	申请Host或Device上的内存，Device上的内存按普通页申请。同步接口。	- 使用svp_acl_rt_malloc_host接口申请的内存，需要通过svp_acl_rt_free_host接口释放内存。 - 如果没有Host端，调用该接口会获取Device侧内存，也可调用svp_acl_rt_free释放。 - 频繁调用svp_acl_rt_malloc_host接口申请内存、调用svp_acl_rt_free_host接口释放内存，会损耗性能，建议用户提前做内存预先分配或二次管理，避免频繁申请/释放内存。
svp_acl_rt_malloc_sec	申请Device上的安全内存，同步接口。	- 使用svp_acl_rt_malloc_sec接口申请的内存，需要通过svp_acl_rt_free_sec接口释放内存。 - 频繁调用svp_acl_rt_malloc_sec接口申请内存、调用svp_acl_rt_free_sec接口释放内存，会损耗性能，建议用户提前做内存预先分配或二次管理，避免频繁申请/释放内存。 - 该接口获取到的安全内存虚拟地址为安全内存句柄，不支持在REE侧进行访问，不支持flush等动作。在REE侧强行尝试读写存在设备挂死风险。在REE侧使用安全内存时必须整块使用，不支持内部分割处理。

3.5 如何获取 Sample

当前SVP ACL提供的样例如[表3-2](#)所示。

表 3-2 Sample 列表

Sample	基本功能	样例介绍、编译及运行运行
resnet50_imagenet_classification	基于Caffe ResNet-50网络实现图片分类（同步推理）	基于Caffe ResNet-50网络实现图片分类（同步推理）
resnet50_async_imagenet_classification	基于Caffe ResNet-50网络实现图片分类（异步推理）	基于Caffe ResNet-50网络实现图片分类（异步推理）

切至发布包Sample目录，解压samples.tar.gz

```
tar -zxvf samples.tar.gz
```



3.6 如何查看日志

在控制台上可以使用cat命令查看信息，cat /dev/logmpp查看错误日志。

3.7 如何查看 Proc 信息

3.7.1 概述

调试信息采用了Linux下的proc文件系统，可实时反映当前系统的运行状态，所记录的信息可供问题定位及分析时使用。

【文件目录】

/proc/umap

【信息查看方法】

- 在控制台上可以使用cat命令查看信息，cat /proc/umap/svp_npu；也可以使用其他常用的文件操作命令，例如 cp /proc/umap/svp_npu ./，将文件拷贝到当前目录。
- 在应用程序中可以将上述文件当作普通只读文件进行读操作，例如fopen、fread等。

📖 说明

参数在描述时有以下2种情况需要注意：

- 取值为{0, 1}的参数，如未列出具体取值和含义的对应关系，则参数为1时表示肯定，为0时表示否定。
- 取值为{aaa, bbb, ccc}的参数，未列出具体取值和含义的对应关系，但可直接根据取值aaa、bbb或ccc判断参数含义。

3.7.2 Proc 信息说明

【调试信息】

```
# cat /proc/umap/svp_npu
[SVP_NPU] Version: Version: [xxxxVx.x.x.x B0xx Release], Build Time[mm dd yyyy, hh:mm:ss]

-----svp_npu module param-----
npu_save_power  max_task_node_num  max_stream_num  max_event_num  prof_buf_size
ops_sel
    0              32              64              32              2048              2.5T

-----dev[0] svp_npu resource info-----
max_model_num  max_sec_model_num  max_stream_num  max_event_num  max_report_num
    64           32           64           32           128
free_model_num  free_sec_model_num  free_stream_num  free_report_num  free_report_num
    63           32           63           32           128

-----svp_npu process info [pid:2546]-----
pid  model_num  stream_num  report_num  event_num  aicpu_num  prof_en  sync_timeout
2546    1         1           0           0           0         1      1048576

-----svp_npu process stream info-----
```



```
id sq_id head tail prior cb_tid cb_block cb_block_id push_cnt sqe_cnt rcv_cnt status
6 6 1 0 3 0 0 0 1 1 1 1

-----svp_npu process model info-----
id prior preempt_en up_step up_top stream_id last_task_time preempted_en name
32 3 0 0 0 0 17649 0 inst

id in_num out_num om_size work_buf_size bandwidth original_ops cube_ops
mem_reuse_mode
32 1 1 26114363 0 26302784 6964232192 10143137792 4

id input_00_shape output_00_shape
32 1x3x224x224 1x1000x1x1

-----svp_npu irq info-----
device_id irq_cnt_last_sec max_irq_cnt_per_sec total_irq_cnt
0 17 17 181

cur_irq_time max_irq_time irq_time_last_sec max_irq_time_per_sec
8 24 180 180
total_irq_time
2445

-----svp_npu runtime info-----
device_id hw_status last_sq_id last_task_node_id prof_flag mac_utilization
0 0 0 0 1 0

timeout_err_cnt hw_err_cnt aicpu_err_cnt
0 0 0

last_hw_task_time hw_utilization total_running_time
13 0% 897
max_hw_task_time min_hw_task_time max_node_cycle_cnt min_node_cycle_cnt
cur_node_cycle_cnt
37208 56 17666810 13593 13593
```

【调试信息分析】

记录当前SVP图像分析引擎模块参数信息，资源信息，被调用stream信息，中断信息和运行时状态信息。

【参数说明】

参数		描述
svp_npu module param SVP图像分析引擎 模块参数	npu_save_power	低功耗标志。通过模块参数svp_npu_save_power配置。 0：关闭低功耗； 1：打开低功耗。
	max_task_node_num	用于约束单个Stream最大任务节点个数，范围：[2, 512]，默认为32个，用户可通过模块参数svp_npu_max_task_node_num配置，同时下发最大的任务数为(svp_npu_max_task_node_num-1)。配置方式是在加载svp_npu的ko时，使用"svp_npu_max_task_node_num=512"参数并修改"512"的值，将其配置成需要设置的数值。



参数		描述
	max_stream_num	可用的Stream个数，范围：[1, 128]，默认为64个，用户可通过模块参数 <code>svp_npu_max_stream_num</code> 配置。 注：仿真不支持通过模块参数修改配置，默认stream最大值为128。 Hi3516CV610只有4条Stream，默认为4条。 注：仿真不支持通过模块参数修改配置，默认stream最大值为4。
	max_event_num	可用的硬件Event个数，范围：[0, 128]，默认为32个，用户可通过模块参数 <code>svp_npu_max_event_num</code> 配置。 Hi3516CV610不支持Event功能，相关显示为0。
	prof_buf_size	用于Profiling的内存大小，范围：[512, 1048576]，单位KB。启动Profiling功能时内部会申请此大小内存。用户可通过模块参数 <code>svp_npu_prof_buf_size</code> 配置。
	ops_sel	表示当前使用的算力，通过模块参数 <code>svp_npu_ops_sel</code> 配置， Hi3516DV500范围：[0, 8] 各档位值所代表的算力：0【0.5T】、1【0.8T】、2【1.0T】、3【1.2T】、4【1.6T】、5【1.8T】、6【2.0T】、7【2.2T】、8【2.5T】。 Hi3516CV610范围：[0, 3] 各档位值所代表的算力：0【0.2T】、1【0.5T】、2【0.8T】、3【1.0T】。 说明： - Hi3516DV500最高支持2.0T的算力，超过档位值6时，算力会调整为2.0T； - Hi3516CV610-10B最高支持0.5T的算力，超过档位1时，算力会调整为0.5T。 - Hi3516CV608最高支持0.2T的算力，超过档位0时，算力会调整为0.2T。
dev[0] svp_npu resource info 设备0 SVP/图像分析引擎资源信息	max_model_num	最多可加载的普通模型个数。
	free_model_num	剩余可加载的普通模型个数。
	max_sec_model_num	最多可加载的安全模型个数，Hi3516CV610不支持，相关显示为0。
	free_sec_model_num	剩余可加载的安全模型个数，Hi3516CV610不支持，相关显示为0。



参数		描述
	max_stream_num	最多可申请的Stream个数。
	free_stream_num	剩余可申请的Stream个数。
	max_event_num	最多可申请的Event个数，Hi3516CV610不支持Event功能，相关显示为0。
	free_event_num	剩余可申请的Event个数，Hi3516CV610不支持Event功能，相关显示为0。
	max_report_num	最多可申请的Report个数。
	free_report_num	剩余可申请的report个数。
svp_npu process info [pid:xxxx] SVP图像分析引擎运行进程占用资源	pid	进程ID。
	model_num	进程加载的模型个数。
	stream_num	进程申请的Stream个数。
	report_num	进程申请的report个数。
	event_num	进程申请的Event个数，Hi3516CV610不支持Event功能，相关显示为0。
	aicpu_num	进程运行当前的模式识别CPU任务个数。
	prof_en	进程是否使能Profiling。0代表没有使能，1代表使能。
	sync_timeout	进程默认阻塞等待的超时时间。 单位：ms。
svp_npu process stream info 进程中运行流的信息	id	流ID。
	sq_id	使用的硬件流的ID号。
	head	标记流的头指针。
	tail	标记流的尾指针。
	prior	当前任务的实时优先级。
	cb_tid	注册到流的Callback任务处理线程的ID。
	cb_block	是否有未完成的阻塞型Callback任务。
	cb_block_id	即将运行的阻塞型Callback任务的ID。
	last_task_time	当前Stream上最近一次任务的执行耗时。执行耗时统计的是任务下发到任务完成的时间，所以多个Stream执行场景，此耗时包含当前任务等待其它Stream上任务执行的时间。



参数		描述
	push_cnt	下发的任务占用流上任务节点的个数。
	sqe_cnt	流上实际未完成的任务个数。
	recv_cnt	流上收到的任务总个数。
	status	同步流的运行状态信息。 0: 空闲状态 1: 任务可下发或任务执行中 2: 模式识别CPU任务处理中 3: 任务执行完成 4: 任务执行失败
svp_npu process model info 进程中模型信息	id	模型的ID编号。
	prior	模型配置的优先级。
	preempt_en	模型配置的是否抢占其它低优先级任务的使能。 0: 不抢占; 1: 抢占。
	up_step	优先级单步提升的超时时间, 单位ms, 范围 [0,600000)。0代表不开启优先级单步提升功能。
	up_top	优先级最高提升的超时时间, 单位ms, 范围 [0,600000)。0代表不开启优先级最高提升功能。
	stream_id	模型最近一次执行所在流的ID。
	last_task_time	模型最近一次执行的耗时。执行耗时统计的是任务下发到任务完成的时间, 所以多个Stream执行场景, 此耗时包含当前任务等待其它Stream上任务执行的时间。
	preempted_en	模型是否支持被打断标识, 在模型生成时指定。具体配置可参考《ATC工具使用指南》。
	name	模型名称, 模型离线生成的时候指定。显示最多32个字符, 超出部分会被裁剪掉。
	in_num	模型原始输入个数, 不包括扩展输入。
	out_num	模型原始输出个数, 不包括扩展输出。
	om_size	模型大小, 单位Byte。
	work_buf_size	模型执行时需要的临时内存大小, 单位Byte。
	bandwidth	模型执行时理论数据带宽开销, 单位Byte。数据为0代表模型中没有带宽开销信息。
	original_ops	原始模型单帧计算次数, 用于衡量原始模型的复杂度和算力, 单位ops。数据为0代表模型中没有原始算力信息。



参数		描述
	cube_ops	原始模型参数对齐架构后的单帧CUBE计算次数，用于衡量模型对齐CUBE单元的算力。数据为0代表模型中没有Cube算力信息。
	mem_reuse_mode	输入内存/输出内存/WorkBuf内存复用模式。
	input_xx_shape	原始模型输入的Shape信息，xx代表输入编号，Shape格式为“NxCxHxW”。
	output_xx_shape	原始模型输出的Shape信息，xx代表输出编号，Shape格式为“NxCxHxW”。
svp_npu irq info SVP图像分析引擎中断信息	device_id	设备ID。
	irq_cnt_last_sec	最近1秒内中断执行次数。
	max_irq_cnt_per_sec	历史上的1秒内最大的中断执行次数。
	total_irq_cnt	产生中断的总次数。
	cur_irq_time	最近一次执行中断的执行耗时。 单位：us。
	max_irq_time	执行一次中断的最大耗时。 单位：us。
	irq_time_last_sec	最近一秒执行中断的执行耗时。 单位：us。
	max_irq_time_per_sec	历史上一秒内执行中断的最大耗时。 单位：us。
	total_irq_time	中断处理总时间。 单位：us。
svp_npu runtime info SVP图像分析引擎设备运行信息	device_id	设备ID。
	hw_status	硬件所处状态。 0：空闲。 1：工作。
	last_sq_id	最近一次下发任务到图像分析引擎的硬件流的ID号。
	last_task_node_id	最近一次在图像分析引擎上执行的任务节点id。
	prof_flag	是否有进程使能Profiling，并申请了Profiling内存。0代表没有使能，1代表使能。



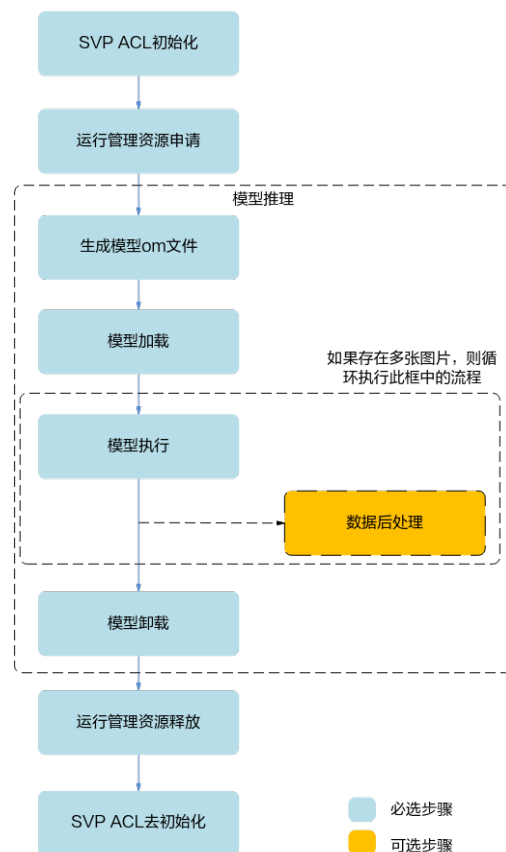
参数		描述
	mac_utilization	最近一次完成推理的任务的MAC利用率，Hi3519DV500/Hi3516DV500不支持。
	timeout_err_cnt	设备上发生超时错误的个数。
	hw_err_cnt	设备上发生逻辑错误的个数。
	aicpu_err_cnt	设备上发生模式识别CPU任务执行错误的个数
	last_hw_task_time	最近一次图像分析引擎执行任务的耗时。
	hw_utilization	最近1s内硬件执行时间所占的比例。
	total_running_time	设备运行总时间。 单位：s
	max_hw_task_time	图像分析引擎执行任务的最大耗时。（仅适用于Hi3516CV610）
	min_hw_task_time	图像分析引擎执行任务的最小耗时。（仅适用于Hi3516CV610）
	max_node_cycle_cnt	图像分析引擎执行任务的最大硬件cycle数。（仅适用于Hi3516CV610）
	min_node_cycle_cnt	图像分析引擎执行任务的最小硬件cycle数。（仅适用于Hi3516CV610）
	cur_node_cycle_cnt	最近一次图像分析引擎执行任务的硬件cycle数。（仅适用于Hi3516CV610）



4 接口调用流程介绍

4.1 主要接口调用流程

图 4-1 接口调用流程图



上图根据应用开发中的典型功能抽象出主要的接口调用流程，例如，如果需要通过模型推理的功能，则需要先加载模型，模型推理结束后，则需要卸载模型；如果模型推理后，需要从推理结果中查找最大置信度的类别标识对图片分类，则需要数据后处理。

1. SVP ACL初始化：调用[svp_acl_init](#)接口实现初始化SVP ACL。
2. 运行管理资源申请：依次申请运行管理资源：[Device](#)、[Context](#)、[Stream](#)。具体流程，请参见“[运行管理资源申请](#)”。
3. 模型推理。
 - a. 生成模型om文件：需使用ATC工具将第三方网络（例如，Caffe ResNet-50网络）转换为适配SoC的离线模型（*.om文件），请参见《ATC工具使用指南》。
 - b. 模型加载：模型推理前，需要先将对应的模型加载到系统中。具体流程，请参见“[模型加载](#)”。
 - c. 模型执行：使用模型实现图片分类、目标识别等功能，目前SVP ACL提供同步推理接口和异步推理接口，支持动态Batch、动态分辨率、动态AIPP等场景。具体流程，请参见“[模型执行](#)”。
 - d. （可选）数据后处理：处理模型推理的结果，此处根据用户的实际需求来处理推理结果，例如用户可以将获取到的推理结果写入文件、从推理结果中找到每张图片最大置信度的类别标识等。
 - e. 模型卸载：调用[svp_acl_md1_unload](#)接口卸载模型。
4. 运行管理资源释放：所有数据处理都结束后，需要依次释放运行管理资源：[Stream](#)、[Context](#)、[Device](#)。具体流程，请参见“[运行管理资源释放](#)”。
5. SVP ACL去初始化：调用[svp_acl_finalize](#)接口实现SVP ACL去初始化。

说明

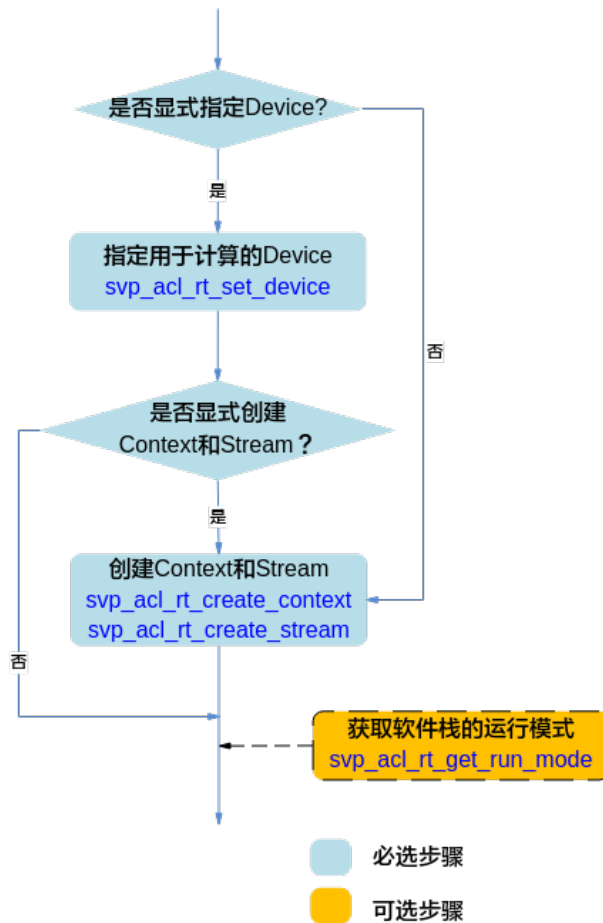
在应用开发过程中，各环节都涉及内存的申请与释放、数据传输（通过内存复制实现）、数据类型的创建与销毁，因此未在图中一一标识，关于内存申请与释放、内存复制的接口请参见“[内存管理](#)”，数据类型的创建与销毁的接口请参见“[数据类型及其操作接口](#)”。

4.2 运行管理资源申请

您需要依次申请运行管理资源，包括：[Device](#)、[Context](#)、[Stream](#)。其中创建Context、Stream的方式分为隐式创建和显式创建。

- 隐式创建Context和Stream：适合简单、无复杂交互逻辑的应用，但缺点在于，在多线程编程中，每个线程都使用默认Context或默认Stream，默认Stream中任务的执行顺序取决于操作系统线程调度的顺序。
- 显式创建Context和Stream：**推荐显式**，适合大型、复杂交互逻辑的应用，且便于提高程序的可读性、可维护性。

图 4-2 运行管理资源申请流程

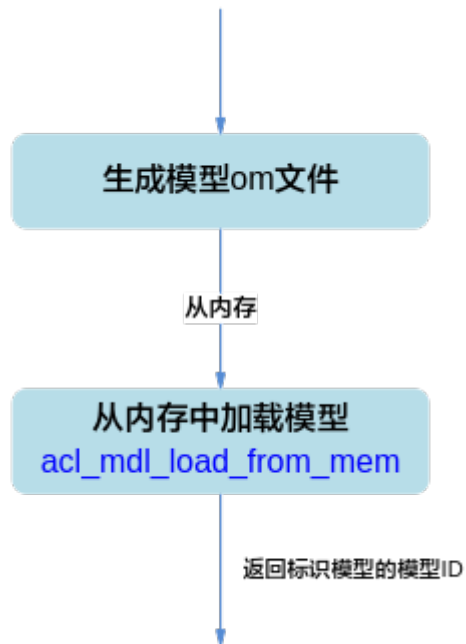


1. 申请运行管理资源时，需按顺序依次申请：**Device**、**Context**、**Stream**。
 - 调用**svp_acl_rt_set_device**接口显式指定用于运算的Device。
 - 调用**svp_acl_rt_create_context**接口显式创建Context，调用**svp_acl_rt_create_stream**接口显式创建Stream。
 - 不显式创建Context和Stream，系统会使用默认Context、默认Stream。默认Context是在调用**svp_acl_rt_set_device**接口时隐式创建的。默认Stream是在第一次执行推理任务时隐式创建的。异步推理时，没有显示指定Stream的模型在第一次推理时会基于当前Context创建一个默认Stream，并在销毁Context时释放该默认Stream；默认Stream作为接口入参，直接传NULL。同步推理时，每个模型会在第一次推理时隐式创建一个默认Stream，在卸载模型时释放该默认Stream。
 - 不显式指定用于运算的Device。调用**svp_acl_rt_create_context**接口显式创建Context，调用**svp_acl_rt_create_stream**接口显式创建Stream。系统在显式创建Context时，系统内部会调用**svp_acl_rt_set_device**接口指定运行的Device，Device ID通过**svp_acl_rt_create_context**接口传入。
2. （可选）调用**svp_acl_rt_get_run_mode**接口获取软件栈的运行模式，根据运行模式来判断后续的内存申请接口调用逻辑，如果查询结果为SVP_ACL_HOST，则

数据传输时涉及申请Host上的内存；如果查询结果为SVP_ACL_DEVICE，则数据传输时不涉及申请Host上的内存，仅需申请Device上的内存。

4.3 模型加载

图 4-3 模型加载流程

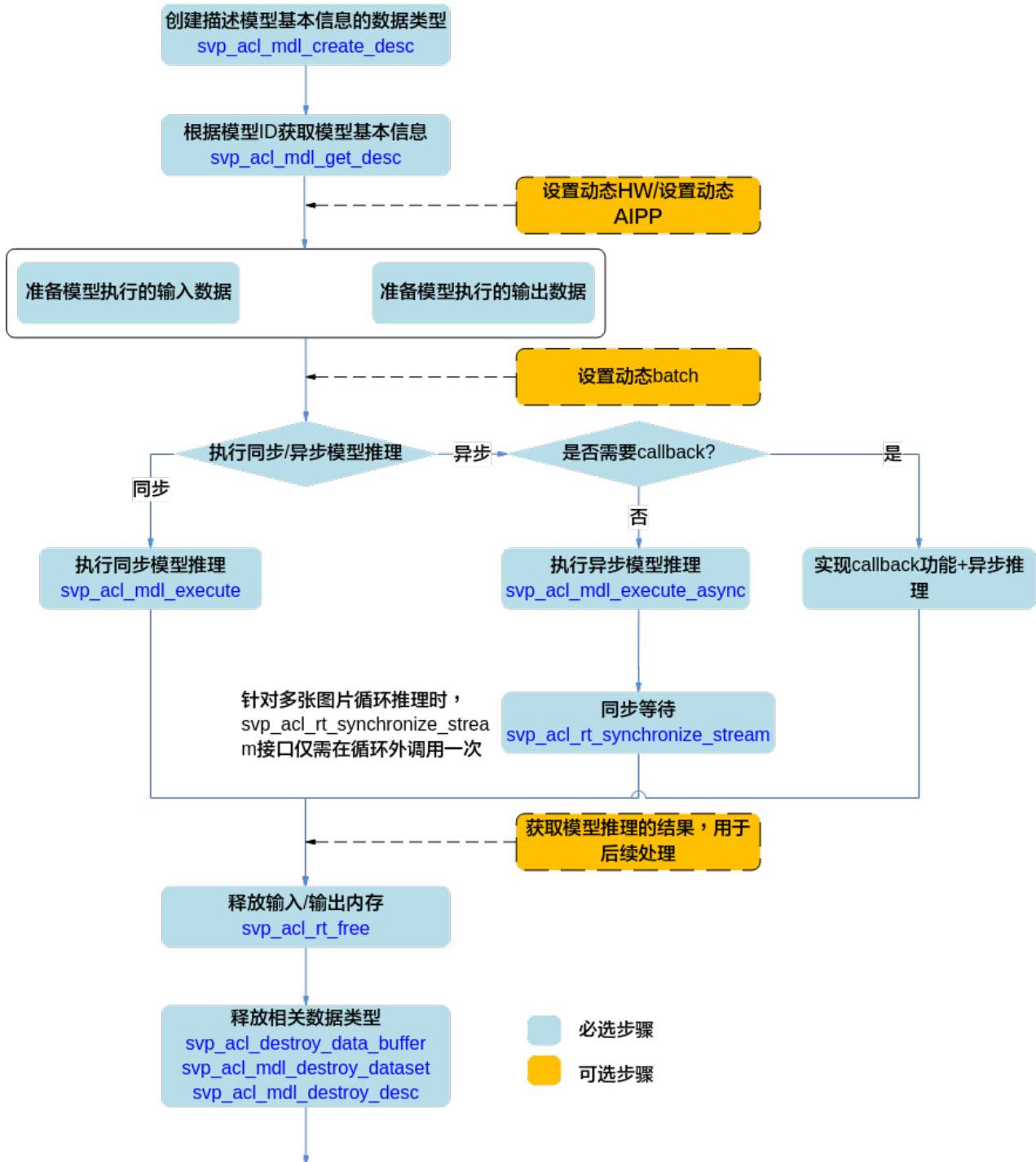


- 在模型加载前，需使用ATC工具将第三方网络（例如，Caffe ResNet-50网络）转换为适配SoC的离线模型（*.om文件），请参见《ATC工具使用指南》。
- 支持以下方式加载模型，模型加载成功后，返回标识模型的模型ID：
svp_acl_md1_load_from_mem：从内存加载离线模型数据，由系统内部管理内存。

4.4 模型执行

4.4.1 基本的模型执行流程

图 4-4 基本的模型推理流程



模型执行的关键流程说明如下：

1. 调用 `svp_acl_md1_create_desc` 接口创建描述模型基本信息的数据类型。
2. 调用 `svp_acl_md1_get_desc` 接口根据“模型加载”中返回的模型ID获取模型基本信息。



3. 准备模型执行的输入、输出数据，具体流程，请参见“[准备模型执行的输入/输出数据](#)”。
4. 设置动态Batch的具体流程，请参见“[设置动态Batch和Total_t](#)”。
5. 执行模型推理。
当前系统支持模型的同步推理和异步推理：
 - 同步推理时调用[sva_acl_mdl_execute](#)接口，同步推理时针对不同模型会创建不同的同步流来执行任务，创建的同步流会在卸载模型时释放。
 - 异步推理时调用[sva_acl_mdl_execute_async](#)接口
对于异步接口，还需调用[sva_acl_rt_synchronize_stream](#)接口阻塞Host运行，直到指定Stream中的所有任务都完成。
如果同时需要实现Callback功能，请参见“[Callback场景](#)”。
6. 获取模型推理的结果，用于后续处理。
对于同步推理，直接获取模型推理的输出数据即可。
对于异步推理，在实现Callback功能时，在回调函数内获取模型推理的结果，供后续使用。
7. 释放内存。
调用[sva_acl_rt_free](#)接口释放Device上的内存。
8. 释放相关数据类型的数据。
在模型推理结束后，需及时调用[sva_acl_destroy_data_buffer](#)接口和[sva_acl_mdl_destroy_dataset](#)接口释放描述模型输入的数据，且先调用[sva_acl_destroy_data_buffer](#)接口，再调用[sva_acl_mdl_destroy_dataset](#)接口。如果存在多个输入、输出，需调用多次[sva_acl_destroy_data_buffer](#)接口。

4.4.2 设置动态 Batch 和 Total_t

须知

- Recurrent网络只支持设置动态batch为1，支持设置Total_t（要处理的总帧数）；非Recurrent网络只支持设置动态batch，不支持设置Total_t。
- 图片输入时最大batch数为256。非图片输入时，最大batch数为5000。

1. 准备模型推理的动态Batch输入的数据，详细流程请参见“[准备模型执行的输入/输出数据](#)”。
 - a. 设置动态batch之前需要先调用[sva_acl_mdl_get_input_index_by_name](#)接口根据输入名称获取模型中标识动态Batch输入的index，动态Batch输入的名称固定为SVA_ACL_DYNAMIC_TENSOR_NAME，然后再调用[sva_acl_mdl_set_dynamic_batch_size](#)接口根据index设置动态batch。
申请动态Batch输入对应的内存前，需要先调用[sva_acl_mdl_get_num_inputs](#)获取输入个数，然后调用[sva_acl_mdl_get_input_size_by_index](#)接口根据index获取单batch所需内存大小，乘以用户要设置的动态batch值得到需要的内存大小(task_buf和work_buf不需要乘以动态batch值，详细请参见“[准备模型执行的输入/输出数据](#)”)。

说明

SVP_ACL_DYNAMIC_TENSOR_NAME是一个宏，宏的定义如下：

```
#define SVP_ACL_DYNAMIC_TENSOR_NAME "svp_acl_mbatch_shape_data"
```

2. 在成功加载模型之后，执行模型之前，设置动态Batch数或者设置Total_t。
 - a. 设置动态Batch数
调用[svp_acl_mdl_set_dynamic_batch_size](#)接口设置动态Batch数。
 - b. 设置Total_t
调用[svp_acl_mdl_set_total_t](#)接口设置Recurrent网络推理的帧数。

4.4.3 设置动态分辨率

- 准备模型推理的动态分辨率输入的数据，详细流程请参见“[准备模型执行的输入/输出数据](#)”。
 - 设置动态分辨率之前需要先调用[svp_acl_mdl_get_input_index_by_name](#)接口根据输入名称获取模型中标识动态分辨率输入的index，动态分辨率输入的名称固定为SVP_ACL_DYNAMIC_TENSOR_NAME。
 - 设置动态分辨率之前，调用[svp_acl_mdl_get_input_size_by_index](#)或者[svp_acl_mdl_get_output_size_by_index](#)接口根据index获取的是默认分辨率(最大分辨率)的size。
 - 设置动态分辨率之前，调用[svp_acl_mdl_get_input_default_stride](#)或者[svp_acl_mdl_get_output_default_stride](#)接口根据index获取的是默认分辨率(最大分辨率)的stride。
 - 设置动态分辨率之前，调用[svp_acl_mdl_get_input_dims](#)或者[svp_acl_mdl_get_output_dims](#)接口根据index获取的是默认分辨率(最大分辨率)的dims。
 - 设置动态分辨率之后，以上接口获取的size/stride/dims是当前设置的分辨率的size/stride/dims；workbuf/taskbuf获取的(最大分辨率)size/stride/dims。
 - 同时使用动态分辨率以及动态AIPP的时候，接口调用顺序需要先设置动态分辨率再设置动态AIPP参数。
 - 设置动态分辨率以及设置动态AIPP之后，以上接口获取的size/stride/dims是当前用户设置的aipp的值。
 - 设置动态分辨率以及设置动态AIPP之后，[svp_acl_mdl_get_first_aipp_info](#)或者[svp_acl_mdl_get_last_aipp_info](#)接口获取的HW信息，是当前动态分辨率对应的HW。

说明

SVP_ACL_DYNAMIC_TENSOR_NAME是一个宏，宏的定义如下：

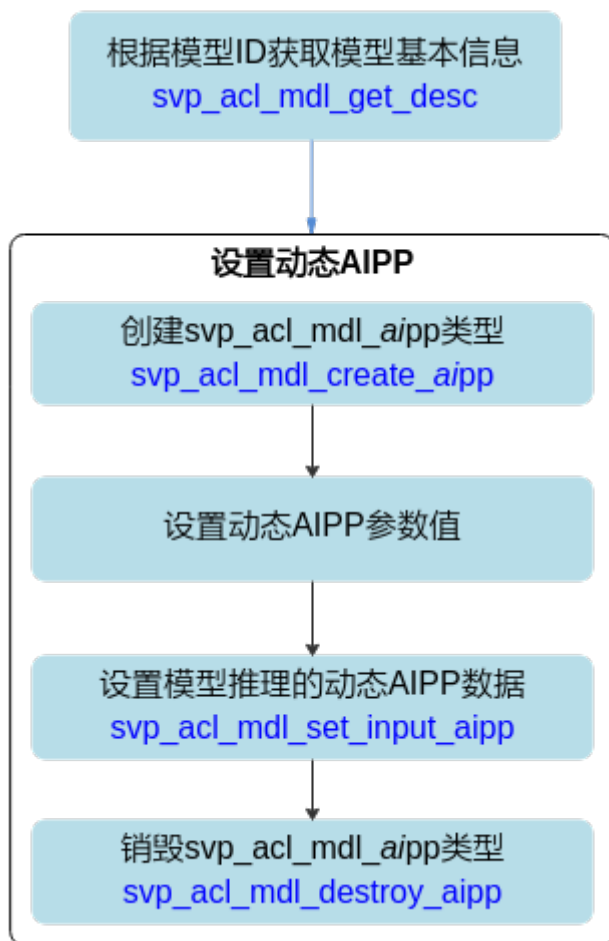
```
#define SVP_ACL_DYNAMIC_TENSOR_NAME "svp_acl_mbatch_shape_data"
```

当前动态分辨率的设置是跟随模型的，所以不支持多个线程同时对加载的一个模型进行同时设置不同的分辨率；当前支持的场景是设置一个分辨率，跑推理，当前推理完成；切换分辨率，继续推理。

- 在成功加载模型之后，执行模型之前，设置动态分辨率大小。
 - 调用[svp_acl_mdl_get_dynamic_hw](#)接口获取支持的动态分辨率个数和大小。
 - 调用[svp_acl_mdl_set_dynamic_hw_size](#)接口根据index设置动态分辨率大小。

4.4.4 设置动态 AIPP

图 4-5 设置动态分辨率/动态 AIPP 流程



在模型加载前，构建模型时，可配置输入动态AIPP、输出动态AIPP相关的信息，构建模型成功后，在生成的om模型中，会新增相应的输入/输出（下文简称输入动态AIPP/输出动态AIPP），在模型推理时通过该新增的输入/输出提供具体的AIPP配置。

例如，a输入的AIPP是动态的，在om模型中，会有与a对应的b输入来描述a的AIPP信息。在模型执行时，准备a输入的数据结构请参见[准备模型执行的输入/输出数据](#)，准备b输入的数据结构、设置b输入的数据请参见以下步骤（调用示例请参见[模型推理](#)）。

步骤1 准备输入动态AIPP的数据结构。

1. 申请输入动态AIPP对应的内存前，需要先调用 [svp_acl_md1_get_input_index_by_name](#) 接口根据输入名称（固定为 SVP_ACL_DYNAMIC_AIPP_NAME）获取模型中标识该输入的index。
2. 调用 [svp_acl_md1_get_input_size_by_index](#) 根据index获取输入内存大小。
3. 调用 [svp_acl_rt_malloc](#) 接口根据2.中的大小申请内存。
申请输入动态AIPP对应的内存后，无需用户设置该内存中的数据（否则可能会导致业务异常），用户调用[步骤2](#)中的接口后，系统会自动向该内存中填入数据。
4. 调用 [svp_acl_create_data_buffer](#) 接口创建 [svp_acl_data_buffer](#) 类型的数据，用于输入动态AIPP数据的内存地址、内存大小。

- 调用`sva_acl_mdl_create_dataset`接口创建`sva_acl_mdl_dataset`类型的数据，并调用`sva_acl_mdl_add_dataset_buffer`接口向`sva_acl_mdl_dataset`类型的数据中增加`sva_acl_data_buffer`类型的数据。

说明

SVP_ACL_DYNAMIC_AIPP_NAME是一个宏，宏的定义如下：

```
#define SVP_ACL_DYNAMIC_AIPP_NAME "sva_acl_dynamic_aipp_data"
```

步骤2 在成功加载模型之后，执行模型之前，设置输入动态AIPP参数值。

- 调用`sva_acl_mdl_get_input_index_by_name`接口根据输入名称（固定为SVP_ACL_DYNAMIC_AIPP_NAME）获取模型中标识该输入的index。
- 调用如下接口设置模型推理的动态AIPP配置。
 - 调用`sva_acl_mdl_create_aipp`接口创建`sva_acl_mdl_aipp`类型。
 - 根据实际需求，调用`sva_acl_mdl_aipp`数据类型下的操作接口设置动态AIPP参数值。输入动态AIPP场景下，`sva_acl_mdl_set_aipp_src_image_size`接口和`sva_acl_mdl_set_aipp_input_format`接口必须调用。
 - 调用`sva_acl_mdl_set_input_aipp`接口/`sva_acl_mdl_set_output_aipp`接口设置模型推理时的输入/输出动态AIPP数据。
 - 及时调用`sva_acl_mdl_destroy_aipp`接口销毁`sva_acl_mdl_aipp`类型。

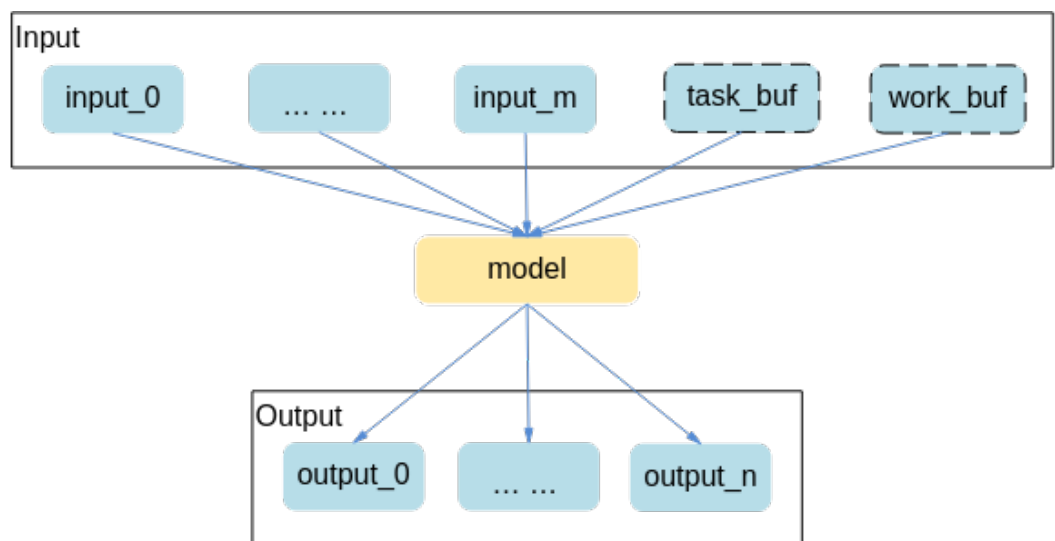
说明

- 同时使用动态分辨率/动态AIPP时，接口调用顺序需要先设置动态分辨率再设置动态AIPP参数。
- 模型输入或者输出都可以支持动态AIPP，接口调用方式类似；输入动态AIPP需要设置原始图像尺寸和图像格式，而输出动态AIPP不需要。
- 支持动态AIPP的模型需要在模型生成的时候指定支持的动态AIPP配置。

----结束

4.4.5 准备模型执行的输入/输出数据

图 4-6 模型输入/输出数据

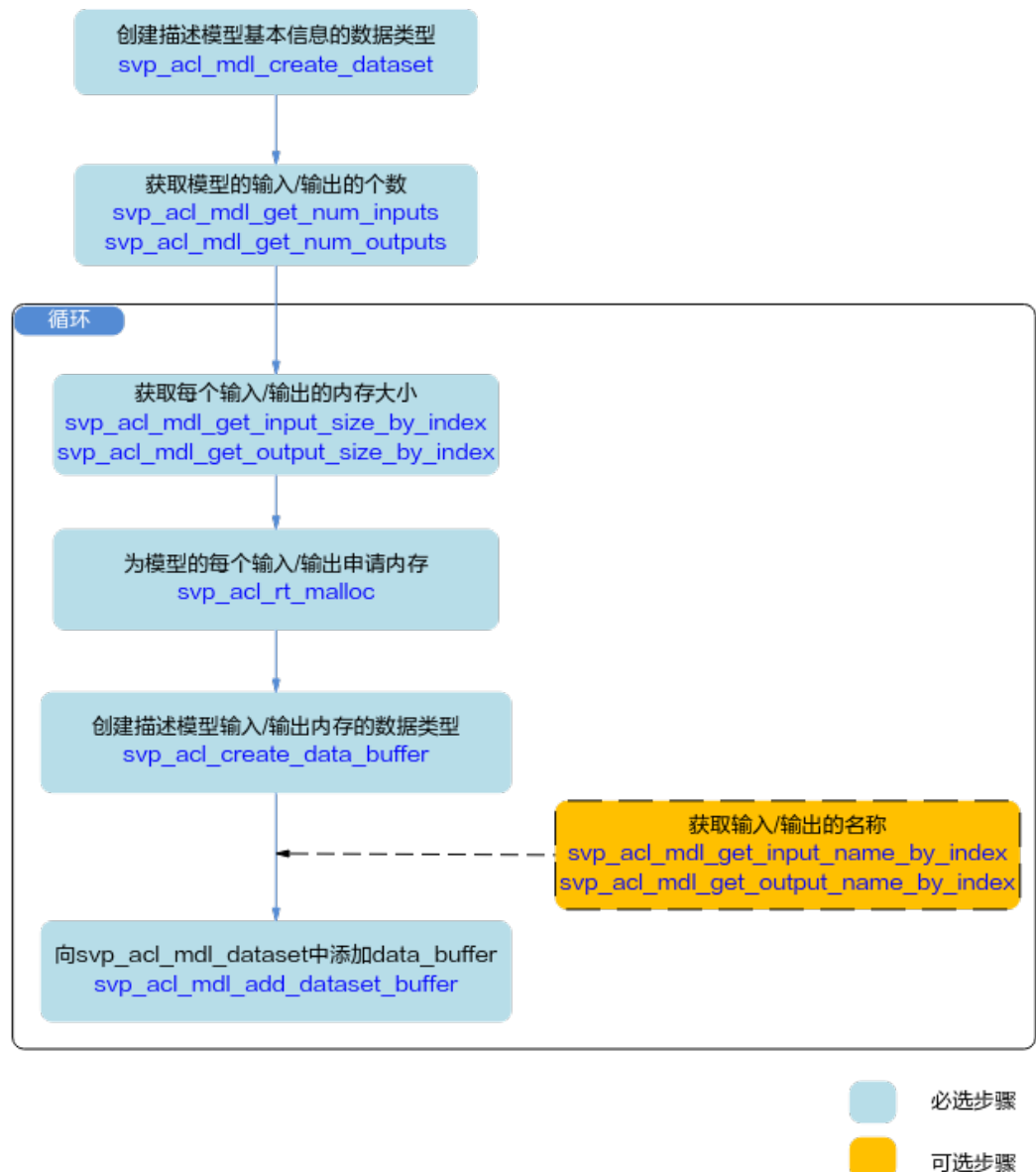


准备输入内存时需要额外输入两块内存，分别为task_buf，work_buf，对应的输入索引为M-2和M-1（M是通过`svp_acl_mdl_get_num_inputs`接口获取的输入个数）。在申请动态batch内存时，这两块内存不需要乘以batch值。

说明

- task_buf用于填充模型推理的任务信息，当前模型推理完之前不能被修改或被其他模型推理任务使用，task_buf的大小不为0。
- work_buf用于任务执行时存储中间结果，一个stream上如果会执行不同的模型，开辟一个最大的work_buf即可，这些模型在该stream上推理时可以共用，不同stream间不能共用，work_buf大小不为0。

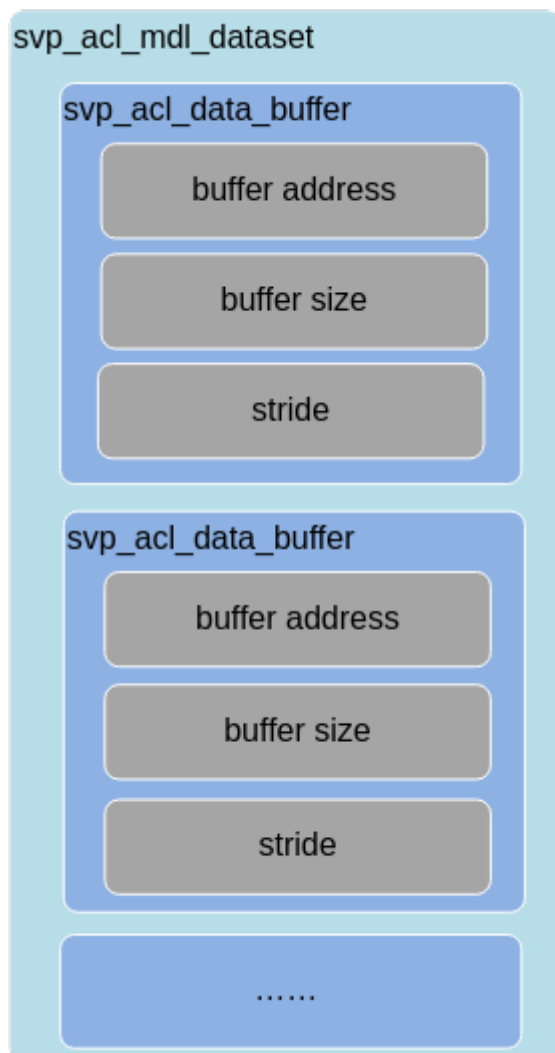
图 4-7 模型执行的输入/输出数据准备流程



使用`svp_acl_mdl_desc`类型的数据描述模型基本信息（例如输入/输出的个数、名称、数据类型、Format、维度信息等），使用`svp_acl_mdl_dataset`类型的数据描述模型

的输入/输出数据，模型可能存在多个输入、多个输出，每个输入/输出的内存地址、内存大小用`svp_acl_data_buffer`类型的数据来描述。

图 4-8 `svp_acl_mdl_dataset` 类型与 `svp_acl_data_buffer` 类型的关系

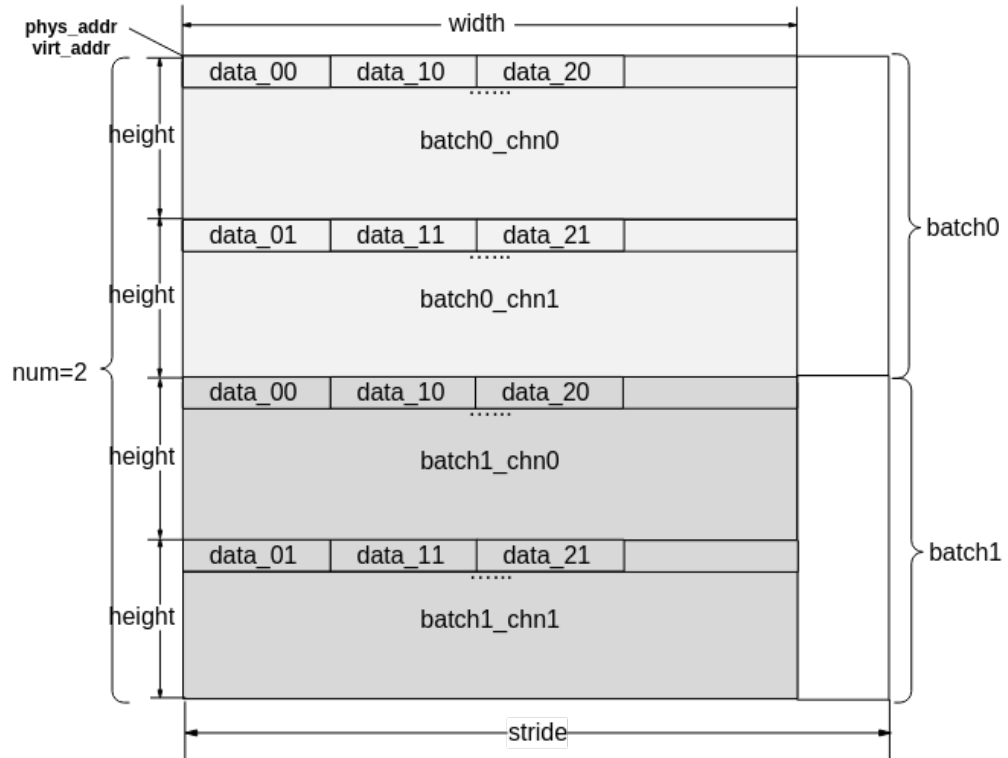


模型存在多个输入、输出时，用户在向`svp_acl_mdl_dataset`中添加`svp_acl_create_data_buffer`时，为避免顺序出错，可以先获取输入、输出的名称，根据输入、输出名称所对应的index的顺序添加。

说明

在创建`svp_acl_data_buffer`类型的数据时，内存大小可以通过调用`svp_acl_mdl_get_input_size_by_index`接口根据index获取。

图 4-9 模型输入/输出数据排布（通道为 2，batch 为 2 示意图）



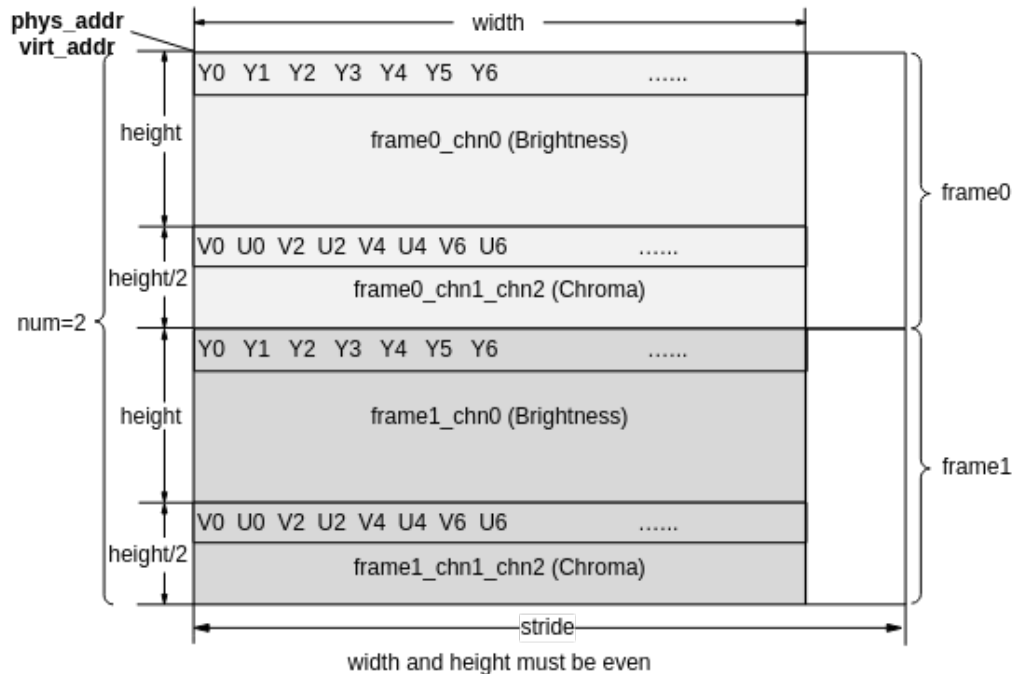
SVP_ACL输入输出数据stride都是按照最后一维对齐，除YVU420SP与YUV420SP输入格式外，数据排布都如图4-9所示。

- 如果是RGB_PACKAGE格式，data_xx数据类型为U24，通道数为1。
- 如果是XRGB_PACKAGE格式，data_xx数据类型为U32，通道数为1。
- 如果是XBGR_PACKAGE格式，data_xx数据类型为U32，通道数为1。
- 如果是RGBX_PACKAGE格式，data_xx数据类型为U32，通道数为1。
- 如果是BGRX_PACKAGE格式，data_xx数据类型为U32，通道数为1。
- 如果是BGR_PACKAGE格式，data_xx数据类型为U24，通道数为1。
- 如果是BGR_PLANAR格式，data_xx数据类型为U8，通道数为3。
- 如果是RGB_PLANAR格式，data_xx数据类型为U8，通道数为3。
- 如果是XRGB_PLANAR格式，data_xx数据类型为U8，通道数为3。
- 如果是XBGR_PLANAR格式，data_xx数据类型为U8，通道数为3。
- 如果是RGBX_PLANAR格式，data_xx数据类型为U8，通道数为3。
- 如果是BGRX_PLANAR格式，data_xx数据类型为U8，通道数为3。
- 如果是RAW_RGGB格式，data_xx数据类型为U8/U10/U12/U16，通道数为1。
- 如果是RAW_GRGB格式，data_xx数据类型为U8/U10/U12/U16，通道数为1。
- 如果是RAW_GBRG格式，data_xx数据类型为U8/U10/U12/U16，通道数为1。
- 如果是RAW_BGGR格式，data_xx数据类型为U8/U10/U12/U16，通道数为1。
- 如果是UV_PACKAGE格式，data_xx数据类型为U16，通道数为1。

说明

RAW格式支持RGGB/GRBG/GBRG/BGGR的排布，SOC中RAW格式排布宽高都是二维排布，所以涉及RAW格式的宽高配置的时候取像素宽高的2倍。仅支持紧密排布的RAW格式。

图 4-10 YVU420SP/YUV420SP 数据排布（通道为 2，frame 为 2 示意图）



SVP_ACL支持检测网阈值通过data层传入，阈值输入固定长度为4，分别是nms_threshold, score_threshold, min_height, min_width，同时将检测网络输出的检测框结果数据排布进行了统一，检测网阈值和检测框结果排布格式如[图4-11](#)和[图4-12](#)所示。

图 4-11 检测网阈值输入数据排布

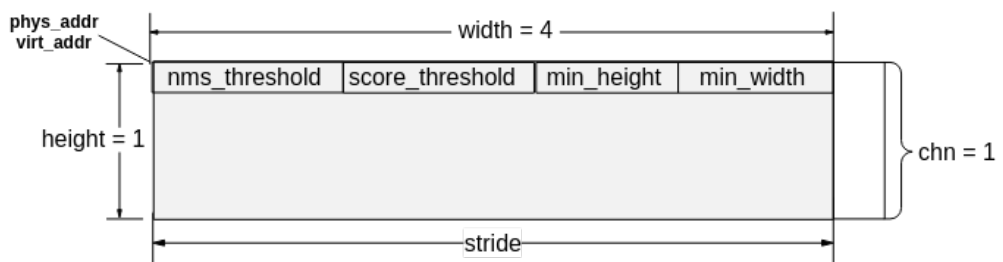
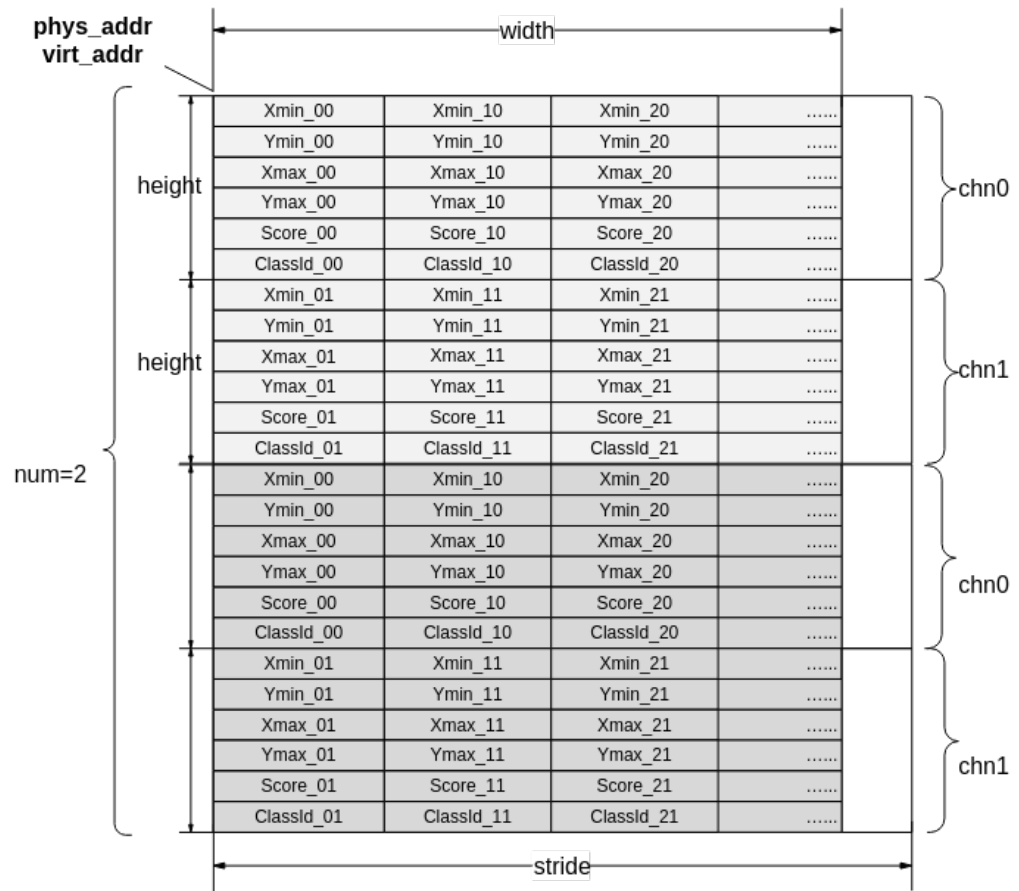


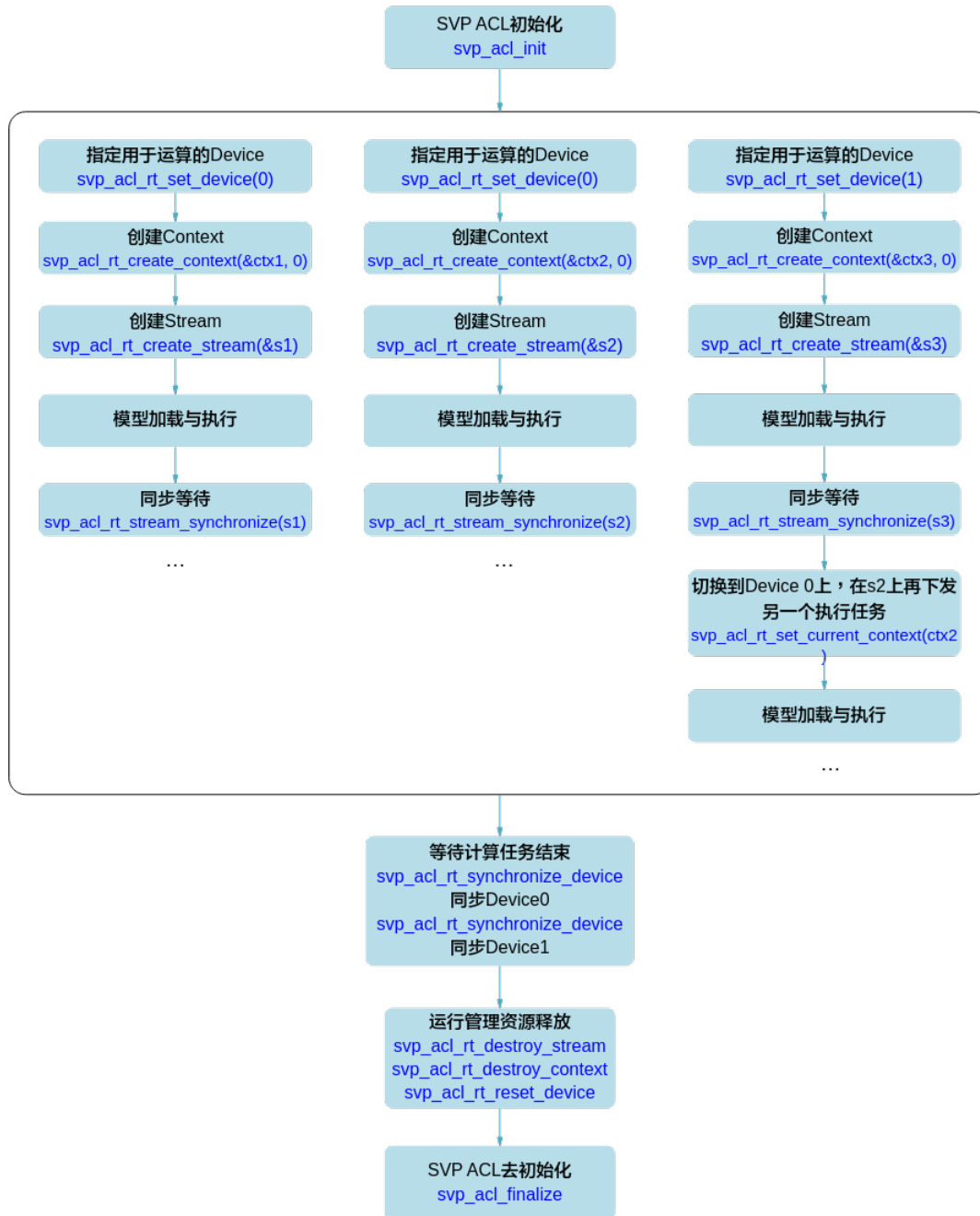
图 4-12 检测框结果数据排布（通道为 2，chn 为 2 示意图）



4.5 同步等待

4.5.1 多 Device 场景

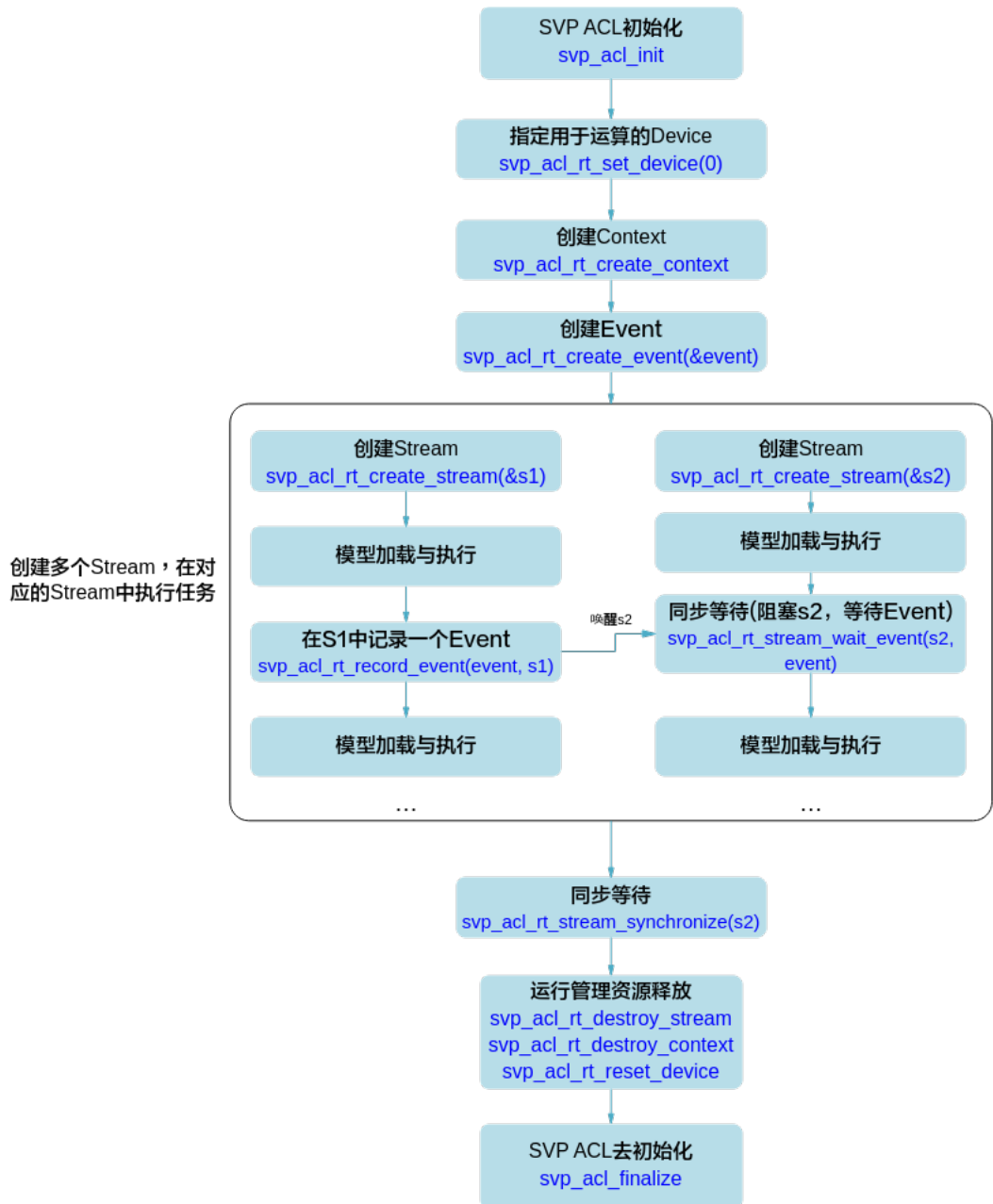
图 4-13 同步等待流程多 Device 场景



- 在多Device时，利用Context切换（调用**svp_acl_rt_set_current_context**接口）来切换Device，比使用**svp_acl_rt_set_device**接口效率高。
- 调用**svp_acl_rt_synchronize_device**接口等待Device上的计算任务结束。
- 模型加载的流程请参见“**模型加载**”，模型执行的流程请参见“**模型执行**”。

4.5.2 多 Stream 场景

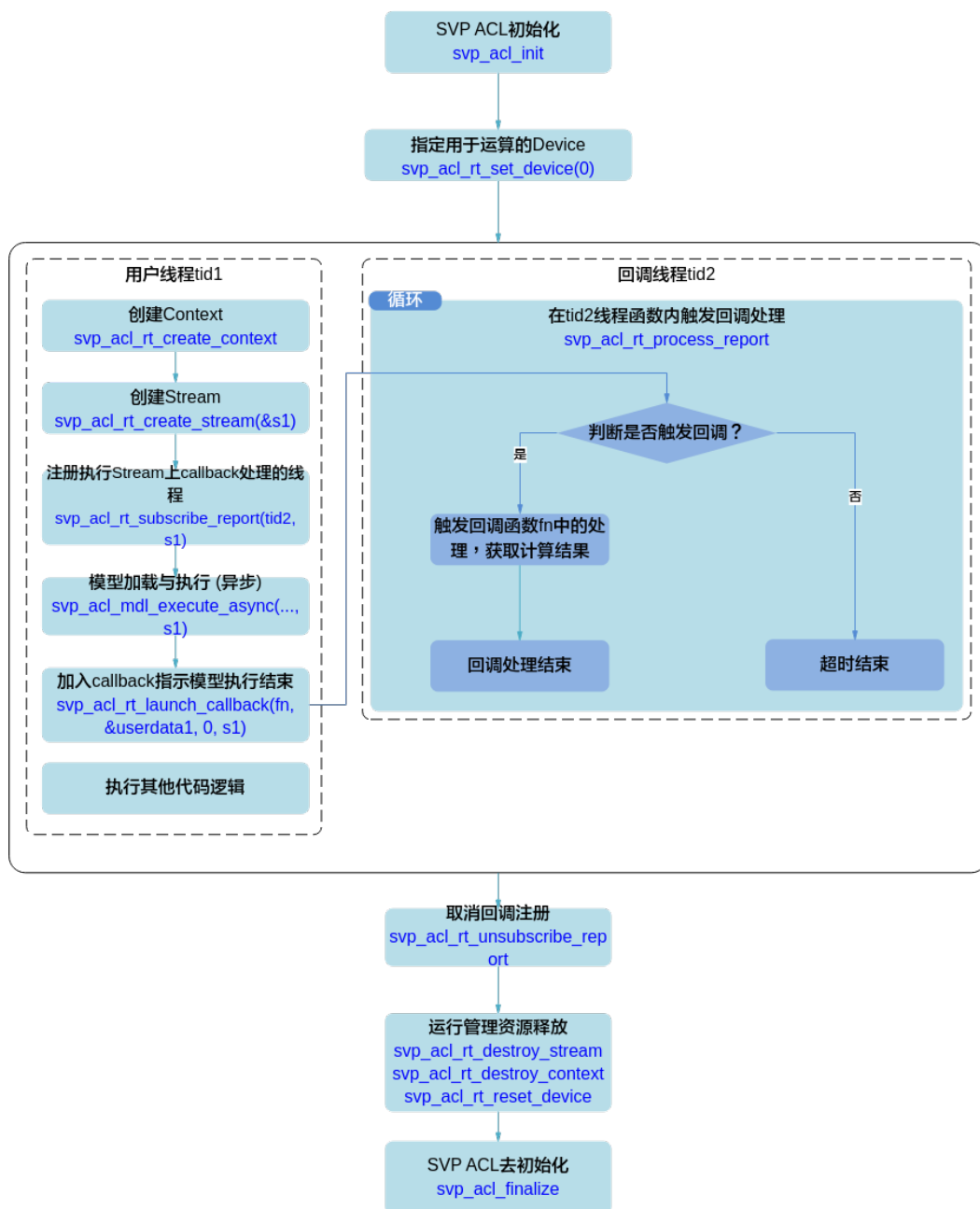
图 4-14 同步等待流程多 Stream 场景



模型加载的流程请参见“[模型加载](#)”，模型执行的流程请参见“[模型执行](#)”。

4.5.3 Callback 场景

图 4-15 同步等待流程 Callback 场景模型推理



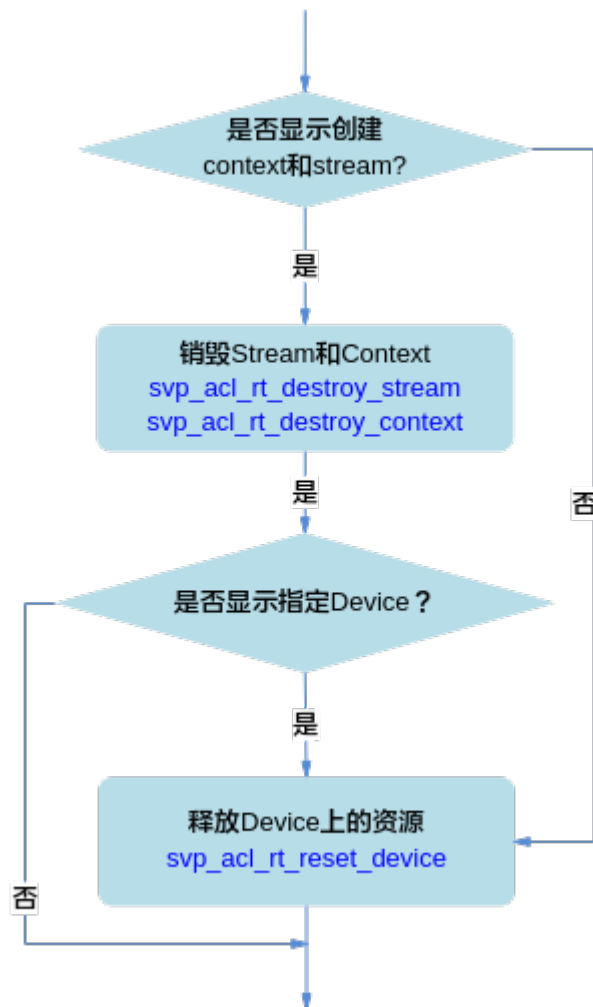
Callback流程中关键流程说明如下：

- 回调函数fn需由用户提前创建，用于获取并处理模型推理或算子执行的结果。
- 线程tid2需由用户提前创建，并自定义线程函数，在线程函数内调用 **svp_acl_rt_process_report** 接口，等待指定时间后，触发回调函数。
- 调用 **svp_acl_rt_subscribe_report** 接口：指定处理Stream上回调函数的线程，线程与tid2保持一致。

- 调用[svp_acl_rt_launch_callback](#)接口：在Stream的任务队列中增加一个需要在板端环境中执行的回调函数，回调函数与fn保持一致。
- 调用[svp_acl_rt_unsubscribe_report](#)接口：取消线程注册（Stream上的回调函数不再由指定线程处理）。
- 如果是异步推理Callback场景，为确保Stream中所有任务都完成、模型推理的结果数据都经过Callback函数处理，在Stream销毁前，需要调用一次[svp_acl_rt_synchronize_device](#)接口。

4.6 运行管理资源释放

图 4-16 运行管理资源释放流程



释放运行管理资源时，需按顺序依次释放：**Stream**、**Context**、**Device**。

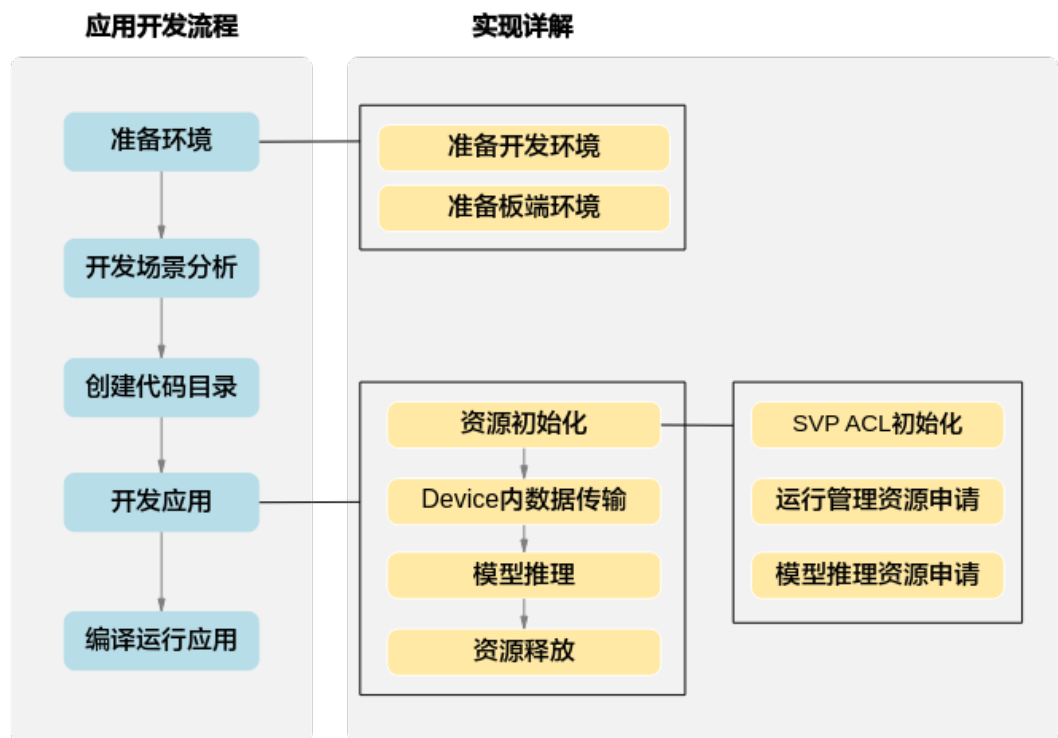
- 显式创建Context和Stream时，需调用[svp_acl_rt_destroy_stream](#)接口释放Stream，再调用[svp_acl_rt_destroy_context](#)接口释放Context。若显式调用[svp_acl_rt_set_device](#)接口指定运算的Device时，还需调用[svp_acl_rt_reset_device](#)接口释放Device上的资源。



- 不显式创建Context和Stream时，仅需调用[svp_acl_rt_reset_device](#)接口释放Device上的资源。

5 开发流程

图 5-1 开发流程（不包含数据预处理）



步骤1 准备环境，包括开发环境和板端环境。

步骤2 开发场景分析。

根据开发场景分析涉及哪些功能（例如，数据传输、模型推理等）的开发，确定功能后，再明确涉及的命令或接口，请参见“[开发场景分析](#)”。

步骤3 创建代码目录。

在开发应用前，您需要先创建目录，存放代码文件、编译脚本、测试图片数据、模型文件等，请参见“[创建代码目录](#)”。

步骤4 开发应用。



1. 资源初始化，包括SVP ACL初始化、运行管理资源申请、模型推理资源申请等，请参见“[资源初始化](#)”。
使用SVP ACL接口开发应用时，必须先调用[svp_acl_init](#)接口进行SVP ACL初始化，否则可能会导致后续系统内部资源初始化出错，进而导致其它业务异常。
2. 数据传输，请参见“[读入图片数据](#)”。
3. 执行模型推理。请参见“[模型推理](#)”。
模型推理结束后，需及时释放相关资源。
4. 若需要处理模型推理的结果，还需要进行数据后处理，例如对于图片分类应用，通过数据后处理从推理结果中查找最大置信度的类别标识。
5. 所有数据处理结束后，需及时释放运行管理资源。

步骤5 编译运行应用，包括模型转换、编译代码、运行应用，请参见“[编译及运行应用](#)”。

----结束



6 准备环境

6.1 准备开发环境

- 您需要参见《驱动和开发环境安装指南》安装开发环境，获取以下文件：
 - 从“SVP ACLlib组件的安装目录/acllib/include/acl”目录下获取调用SVP ACL接口所需的头文件。
 - 从“SVP ACLlib组件的安装目录/acllib/libxx/stub”目录下获取编译SVP ACL接口所需的库文件。
- 在安装开发环境后，您需要参见《驱动和开发环境安装指南》中的“安装后处理”章节安装交叉编译器，并配置对应环境变量。

须知

- 本文以如下安装路径示例来说明操作步骤，实际编译、运行应用前，**请务必**获取这些组件的实际安装路径，以便后续操作时使用，其中，\$HOME表示安装用户的家目录：
以非root用户安装ACLlib组件安装包，安装路径示例为\$HOME/acl，在该路径下，包括“acllib”目录。
- 本文中的操作步骤（包括模型转换、编译代码等）需以运行用户登录开发环境后再执行，**请务必**获取各组件的运行用户，以便后续操作时使用。
- 用户使用export命令在当前终端窗口下声明环境变量，关闭Shell终端或切换用户时环境变量失效。

6.2 准备运行环境

- 如果您需要运行应用，需完成板端环境的配置、软件包的部署等，请参见《驱动和开发环境安装指南》。
- 板端环境的操作系统为Linux时，在部署与调试后，使用过程中，若板端环境上的空间不足时，您可以使用mount命令将NFS服务器上的目录挂载到板端环境的指定目录。



- a. 以root用户登录Linux服务器（Ubuntu操作系统），安装NFS服务并配置共享目录。

确保环境连网且源可用的前提下，安装NFS服务，可参考如下命令安装，如果安装过程中提示已安装NFS服务，则无需重复安装：

```
apt-get install nfs-kernel-server
```

在“/etc/exports”文件中配置共享目录，配置完成后，执行**/etc/init.d/nfs-kernel-server restart**命令重启NFS服务使配置生效。可参考如下配置段（添加在文件的末尾），斜体部分请根据实际情况替换，*服务器绝对路径*表示Linux服务器的共享目录，如不存在，请提前创建。

```
服务器绝对路径*(rw,sync,root_squash,anonuid=id*,anongid=gid*)
```

- b. 命令示例如下，其中，*path*表示板端环境上的目录，需根据实际情况修改，例如：/root。

```
mount -t nfs -o nolock,tcp NFS服务器IP地址:服务器绝对路径 path
```



7 开发首个应用

7.1 开发场景分析

7.1.1 开发场景

开发图片分类应用，对2张分辨率为1024*683的*.jpg图片分类。

7.1.2 场景分析

您可从以下几方面入手分析该应用需包含哪些功能，这些功能点对应哪些SVP ACL接口。

场景列表	实现分析	涉及的命令或关键接口
模型推理	本文介绍的场景是图片分类，因此需要选取开源的分类网络，此处选择的是Caffe resnet-50网络，将开源的resnet-50网络转换为适配图像分析引擎的离线模型（*.om文件），使用该离线模型推理图片所属的类别。 resnet-50网络对输入图片宽高的要求为224*224，且要求输入图片格式为RGB。但当前输入图片是*.jpg格式，因此下文的样例中提供了一个Python脚本先转换图片。	• 使用ATC（Advanced Tensor Compiler）工具将resnet-50网络转换为*.om文件（适配图像分析引擎处理器的离线模型）。请参见《ATC工具使用指南》。 • 模型加载 svp_acl_md1_load_from_mem • 模型执行 svp_acl_md1_execute • 模型卸载 svp_acl_md1_unload

7.2 创建代码目录

在开发应用前，您需要先创建目录，存放代码文件、编译脚本、测试图片数据、模型文件等。



以开发环境的安装用户，在开发环境的任意目录下创建存放应用代码的目录，如下仅是示例，可参考：

```

-App名称
├── caffe_model //该目录下存放模型转换相关的文件
│   └── xxx.caffemodel
├── model //该目录下存放转化后的om模型
│   └── xxx.om
├── data
│   └── xxx.jpg //原始测试数据
├── inc //该目录下存放声明函数的头文件
│   └── xxx.h
├── out //该目录下存放输出结果
├── src //该目录下存放系统初始化的配置文件、编译脚本、函数的实现文件
│   ├── xxx.json //系统初始化的配置文件
│   ├── CMakeLists.txt //编译脚本
│   └── xxx.cpp //实现文件

```

7.3 开发应用

7.3.1 资源初始化

7.3.1.1 SVP ACL 初始化

基本原理：必须调用[svp_acl_init](#)接口初始化SVP ACL，配置文件内容为json格式，当前支持以下配置：

dump信息配置，示例、配置说明及约束请参见《ATC工具使用指南》“online_model_type”章节。默认不启动dump配置。

📖 说明

如果当前的默认配置已满足需求，无需修改，可向svp_acl_init接口中传入NULL，或者可将配置文件配置为空json串（即配置文件中只有{}）。

向svp_acl_init接口中传入空指针的示例如下：

```
svp_acl_error ret = svp_acl_init(NULL);
```

示例代码：可以从[acl_resnet50](#)样例中查看完整样例代码。

调用接口后，增加了异常处理的分支，同时通过ERROR_LOG记录报错日志、通过INFO_LOG记录各动作的提示日志，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```

#include "acl/svp_acl.h"
//.....
//初始化基本配置。
//此处的.表示相对路径，相对可执行文件所在的目录
//例如，编译出来的可执行文件存放在out目录下，此处的..就表示out目录的上一级目录
const char * acl_config_path = "../src/acl.json";
svp_acl_error ret = svp_acl_init(acl_config_path);
//.....

```

7.3.1.2 运行管理资源申请（单进程+单线程+单 Stream）

多线程、多Stream的场景请参见[Stream管理](#)。

基本原理：需要按顺序依次申请如下资源：[Device](#)、[Context](#)、[Stream](#)，确保可以使用这些资源执行运算、管理任务。

- 关于Context和Stream
 - 显式创建Context时，调用[svp_acl_rt_create_context](#)接口，此时需显式调用[svp_acl_rt_destroy_context](#)接口释放。
 - 显式创建Stream时，调用[svp_acl_rt_create_stream](#)接口，此时需显式调用[svp_acl_rt_destroy_stream](#)接口释放。
 - 如果不显式创建Context和Stream，您可以使用[svp_acl_rt_set_device](#)接口隐式创建的默认Context和默认Stream，但默认Context和默认Stream存在如下限制：
 - 一个Device对应一个默认Context，默认Context不能通过[svp_acl_rt_destroy_context](#)接口来释放。
 - 一个Device对应一个默认Stream，默认Stream不能通过[svp_acl_rt_destroy_stream](#)接口来释放。默认Stream作为接口入参时，直接传NULL。
- 关于单进程、单线程、单Stream场景
 - 单进程：一个应用程序对应一个进程。
 - 单线程：不显式创建多个线程时，默认只有一个线程。
 - 单Stream：整个开发的过程中使用同一个Stream。
对于同一个Stream中的异步任务，SVP ACL会按照应用程序中任务的顺序执行任务，确保异步任务执行的顺序。

示例代码：可以从[acl_resnet50](#)样例中查看完整样例代码。

调用接口后，增加了异常处理的分支，同时通过ERROR_LOG记录报错日志、通过INFO_LOG记录各动作的提示日志，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
//1.初始化变量
extern bool g_is_device;
//.....

//2.指定运算的Device，SoC场景下，当前只有一个Device
ret = svp_acl_rt_set_device(device_id);

//3.显式创建一个Context，用于管理Stream对象
ret = svp_acl_rt_create_context(&context, device_id);

//4.显式创建一个Stream
//用于维护一些异步操作的执行顺序，确保按照应用程序中的代码调用顺序执行任务
ret = svp_acl_rt_create_stream(&stream);

//5.获取当前svp_acl软件栈的运行模式，根据不同的运行模式，后续的接口调用方式不同
//SoC场景下,该接口的查询结果为SVP_ACL_DEVICE，表示svp_acl软件栈运行在SoC的CPU上，只需
//申请SoC上的内存
svp_acl_rt_run_mode run_mode;
ret = svp_acl_rt_get_run_mode(&run_mode);
g_is_device = (run_mode == SVP_ACL_DEVICE);

//.....
```

7.3.1.3 模型推理资源申请

基本原理：在模型推理前，需要从离线模型文件（适配SoC的离线模型）中加载模型数据到内存中。

- 关于模型转换

在模型加载前，需要将第三方网络（例如，Caffe ResNet-50网络）转换为适配SoC的离线模型，请参见《ATC工具使用指南》。

- 关于模型加载

加载模型数据方式如下：

svp_acl_mdl_load_from_mem：从内存加载离线模型数据，由系统内部管理内存。

- 关于模型的输入、输出数据

svp_acl_mdl_dataset主要用于描述模型推理时的输入数据/输出数据，模型可能存在多个输入/多个输出，每个输入/输出的内存地址、内存大小用

svp_acl_data_buffer类型的数据来描述。

示例代码：可以从[acl_resnet50](#)样例中查看完整样例代码。

以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
//1.初始化变量。
//此处的..表示相对路径，相对可执行文件所在的目录
//例如，编译出来的可执行文件存放在out目录下，此处的..就表示out目录的上一级目录
const char* om_model_path = "../model/resnet50.om"
//.....

//2.申请用于存储.om文件内存，model_mem_size为.om文件大小。
ret = svp_acl_rt_malloc(&model_mem_ptr, model_mem_size,
SVP_ACL_MEM_MALLOC_HUGE_FIRST);
//.....

//3.加载离线模型文件（适配SoC的离线模型）。
//模型加载成功，返回标识模型的ID。
ret = svp_acl_mdl_load_from_mem(model_mem_ptr, model_mem_size, &model_id);

//4.根据5中加载成功的模型的ID，获取该模型的描述信息。
// model_desc为svp_acl_mdl_desc类型。
model_desc = svp_acl_mdl_create_desc();
ret = svp_acl_mdl_get_desc(model_desc, model_id);

//5.创建svp_acl_model_dataset类型的数据，描述模型推理的输出。
//output为svp_acl_mdl_dataset类型
output = svp_acl_mdl_create_dataset();

//6.1 获取模型的输出个数。
size_t output_size = svp_acl_mdl_get_num_outputs(model_desc);

//6.2 循环为每个输出申请内存，并将每个输出添加到svp_acl_model_dataset类型的数据中。
for (size_t i = 0; i < output_size; ++i) {
    size_t buffer_size = svp_acl_mdl_get_output_size_by_index(model_desc, i);
    size_t output_stride = svp_acl_mdl_get_output_default_stride(model_desc, i);
    void *output_buffer = nullptr;
    svp_acl_error ret = svp_acl_rt_malloc(&output_buffer, buffer_size,
SVP_ACL_MEM_MALLOC_NORMAL_ONLY);
```

```
svp_acl_data_buffer* output_data = svp_acl_create_data_buffer(output_buffer, buffer_size,
output_stride);
ret = svp_acl_md1_add_dataset_buffer(output, output_data);
}
//.....
```

7.3.2 读入图片数据

基本原理：

- 根据“[运行管理资源申请（单进程+单线程+单Stream）](#)”中[svp_acl_rt_get_run_mode](#)接口的查询结果（SVP_ACL_DEVICE，表示软件栈运行在SoC的[图像分析引擎](#)上），在自定义函数ReadBinFile内部，直接调用[svp_acl_rt_malloc](#)申请Device上的内存，再调用C++标准库的read函数将图片数据读入Device的内存中。
- 读入图片数据（即将输入图片数据传输到Device上，用于推理）。

示例代码：可以从[acl_resnet50](#)样例中查看完整样例代码。

以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include <iostream>
#include <fstream>
#include <cstring>
#include "acl/svp_acl.h"
//.....
//以下数据用于描述图片的路径及图片名称picName。
string testFile[] = {
    "../data/dog1_1024_683.bin",
    "../data/dog2_1024_683.bin"
};

//循环处理每张图片
for (size_t index = 0; index < sizeof(testFile) / sizeof(testFile[0]); ++index) {

    //自定义函数，调用C++标准库std::ifstream中的函数读取图片文件。
    //并输出图片文件占用的内存大小fileSize，返回值为图片文件的内存地址inputBuff。

    void *inputBuff = nullptr;
    uint32_t fileSize = 0;
    auto ret = Utils::ReadBinFile(fileName, inputBuff, fileSize);

    //TODO：模型推理
}
//.....
```

7.3.3 模型推理

基本原理：同步执行模型推理。模型推理结束后，需及时释放相关资源。

- 关于模型推理
调用[svp_acl_md1_execute](#)接口实现同步模型推理，该接口的入参之一是模型ID，在“[模型推理资源申请](#)”阶段，如果加载模型成功，则会返回标识模型的ID。
- 关于模型输出数据的处理
“[示例代码](#)”中的代码逻辑是：获取模型推理的输出数据，将输出数据转换为float类型后，再输出其中top5置信度的类别编号。

- 关于资源释放
 - `svp_acl_mdl_dataset`主要用于描述模型推理时的输入数据、输出数据，模型可能存在多个输入、多个输出，每个输入/输出的内存地址、内存大小用 `svp_acl_data_buffer` 类型的数据来描述。在模型推理结束后，需及时调用 `svp_acl_destroy_data_buffer` 接口和 `svp_acl_mdl_destroy_dataset` 接口释放描述模型输入的数据，且先调用 `svp_acl_destroy_data_buffer` 接口，再调用 `svp_acl_mdl_destroy_dataset` 接口。如果存在多个输入，需调用多次 `svp_acl_destroy_data_buffer` 接口。
 - 在模型推理结束后，还需要通过 `svp_acl_mdl_unload` 接口卸载模型。

7.3.3.1 示例代码

您可以从[acl_resnet50](#)样例中查看完整样例代码。

调用接口后，增加了异常处理的分支，同时通过ERROR_LOG记录报错日志、通过INFO_LOG记录各动作的提示日志，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include <iostream>
#include <map>
#include <sstream>
#include <algorithm>
#include "acl/svp_acl.h"
//.....
//以下数据用于描述图片的路径及图片名称pic_name。
string test_file[] = {
    "../data/dog1_1024_683.bin",
    "../data/dog2_1024_683.bin"
};

//循环处理每张图片
for (size_t index = 0; index < sizeof(test_file) / sizeof(test_file[0]); ++index) {
    //1. 创建模型推理的输入数据
    //创建svp_acl_mdl_dataset类型的数据input_dataset，用于描述模型推理时的输入数据（图片数据集）
    //输入的内存地址、内存大小用svp_acl_data_buffer类型的数据来描述
    input_dataset = svp_acl_mdl_create_dataset();
    //示例中的模型只有1个输入，将“读入图片数据”阶段的图片数据作为模型推理的输入
    //input_data_buffer表示图片数据在Device中的内存，buffer_size表示内存大小，input_stride表示内存对齐大小
    svp_acl_data_buffer* input_data = svp_acl_create_data_buffer(input_data_buffer,
buffer_size, input_stride);
    svp_acl_error ret = svp_acl_mdl_add_dataset_buffer(input_dataset, input_data);

    //2. 执行模型推理
    //model_id表示模型ID，在“模型推理资源初始化”时成功加载模型后，会返回标识模型的ID
    //output_dataset表示模型推理的输出数据，在“模型推理资源初始化”时已定义
    svp_acl_error ret = svp_acl_mdl_execute(model_id, input_dataset, output_dataset)

    //获取模型推理的输出，做数据后处理，输出top5置信度的类别编号
    for (size_t i = 0; i < svp_acl_mdl_get_dataset_num_buffers(output_dataset); ++i) {
        //获取每个输出的内存地址和内存大小
        svp_acl_data_buffer* data_buffer = svp_acl_mdl_get_dataset_buffer(output_dataset, i);
        void* data = svp_acl_get_data_buffer_addr(data_buffer);
        size_t len = svp_acl_get_data_buffer_size(data_buffer);

        //将内存中的数据转换为float类型
        float *out_data = (float *) (data);
```

```
//屏显每张图片的置信度的类别编号
for (int j = 0; j < len / sizeof(float); ++j) {
    printf("index:%d value:%f\n", j, outData++);
}
}

//3. 模型推理结束后，释放模型推理的输入内存input_data_buffer
svp_acl_rt_free(input_data_buffer);

//4. 释放资源
//如果模型存在多个输入，则需要多次调用svp_acl_destroy_data_buffer释放描述每个输入的
svp_acl_mdl_dataset类型数据
for (size_t i = 0; i < svp_acl_mdl_get_dataset_num_buffers(input_dataset); ++i) {
    svp_acl_data_buffer* data_buffer = svp_acl_mdl_get_dataset_buffer(input_dataset, i);
    void* data = svp_acl_get_data_buffer_addr(data_buffer);
    (void)svp_acl_rt_free(data);
    svp_acl_destroy_data_buffer(data_buffer);
}

svp_acl_mdl_destroy_dataset(input_dataset);

//.....
}
//TODO: 释放运行管理资源
```

7.3.4 运行管理资源释放与 SVP ACL 去初始化

基本原理：

所有数据处理都结束后，需要释放运行管理资源，包括**Stream**、**Context**、**Device**。释放资源时，需要按顺序释放，先释放Stream，再释放Context，最后再释放Device。

- 如果使用默认Stream，则不能通过**svp_acl_rt_destroy_stream**接口来释放。
- 如果使用默认Context，则不能通过**svp_acl_rt_destroy_context**接口来释放。
- 默认Context、默认Stream，是在调用**svp_acl_rt_reset_device**接口后自动释放。
- 调用**svp_acl_finalize**接口实现SVP ACL去初始化。

示例代码：可以从[acl_resnet50](#)样例中查看完整样例代码。

以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....

svp_acl_error ret = svp_acl_rt_destroy_stream(stream);
ret = svp_acl_rt_destroy_context(context);
ret = svp_acl_rt_reset_device(device_id);
ret = svp_acl_finalize();
//.....
```

7.4 编译及运行应用

编译运行应用的步骤，请参见“[基于Caffe ResNet-50网络实现图片分类（同步推理）](#)”。

相关注意点：

- 模型转换，详细说明请参见《ATC工具使用指南》。



- 编译代码时，编译脚本的修改点如下。
您可以从[acl_resnet50](#)样例中获取编译脚本CMakeLists.txt，在该编译脚本的基础上修改如下参数。
 - include_directories：添加头文件所在的目录。
示例如下：

```
include_directories(  
    directoryPath1  
    directoryPath2  
)
```
 - link_directories：添加库文件所在的目录。
示例如下：

```
link_directories(  
    directoryPath3  
    directoryPath4  
)
```
 - add_executable：修改可执行文件的名称（例如：*main*）、添加*.cpp文件所在的目录。
示例如下：

```
add_executable(main  
    directoryPath5  
    directoryPath6)
```
 - target_link_libraries：修改可执行文件的名称（与 [add_executable](#)中保持一致）、添加可执行文件依赖的库文件。
示例如下：

```
target_link_libraries(main  
    svp_acl  
    libName1  
    libName2)
```

依赖的库文件与接口所在的头文件有关，具体对应关系如下：

表 7-1 头文件与库文件的对应关系

头文件	库文件
Acllib组件的安装目录/acllib/ include/acl/svp_acl_base.h	Acllib组件的安装目录/acllib/libxx/stub/ libsvp_acl.so
Acllib组件的安装目录/acllib/ include/acl/svp_acl.h	
Acllib组件的安装目录/acllib/ include/acl/svp_acl_mdl.h	
Acllib组件的安装目录/acllib/ include/acl/svp_acl_rt.h	
Acllib组件的安装目录/acllib/ include/acl/svp_acl_ext.h	
Acllib组件的安装目录/acllib/ include/acl/svp_acl_prof.h	



📖 说明

- 编译通过后，运行应用时，通过配置环境变量，应用会链接到运行环境上“Acllib组件的安装目录/acllib/libxx”目录下的*.so库，运行时会自动链接到依赖其它组件的*.so库。
- 编译基于SVP ACL接口的代码逻辑时，请按照引用的头文件，依赖对应的so文件，引用多余的so文件可能导致后续版本升级时存在兼容性问题。

编译选项修改可执行文件的名称（与 `add_executable`保持一致），以及可执行文件的安装目录。

示例如下，表示`main`安装在`${CMAKE_INSTALL_PREFIX}/out`目录下，`${CMAKE_INSTALL_PREFIX}`变量定义的路径是相对路径，相对cmake命令执行的路径：

```
set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ".././../out")
install(TARGETS main DESTINATION ${CMAKE_RUNTIME_OUTPUT_DIRECTORY})
```

- 关于cmake参数的详细介绍，请参见<https://cmake.org/cmake/help/latest/guide/tutorial/index.html>，选择对应的版本后查看参数。
- 运行可执行文件时，需将SVP ACL初始化配置文件（`acl.json`）所在的目录、可执行文件所在的目录、测试图片所在的目录、*.om文件所在的目录都上传到板端环境的同一个目录下。

如果在SVP ACL初始化阶段，在`svp_acl_init`接口中传入空指针，则无需将SVP ACL初始化配置文件（`acl.json`）所在的目录上传到板端环境。



8 开发典型功能点的介绍

8.1 Stream 管理

8.1.1 原理介绍

在SVP ACL中，Stream是一个任务队列，应用程序通过Stream来管理任务的并行，一个Stream内部的任务保序执行，即Stream根据发送过来的任务依次执行；不同Stream中的任务并行执行。一个默认Context下会挂一个默认Stream，如果不显式创建Stream，可使用默认Stream。默认Stream作为接口入参时，直接传NULL。

8.1.2 单线程单 Stream

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
//显式创建一个Stream
svp_acl_rt_stream stream;
svp_acl_rt_create_stream(&stream);
uint32_t model_id = 0;

//调用触发任务的接口，例如异步模型推理，任务下发在stream1
svp_acl_md_dataset *input;
svp_acl_md_dataset *output;
svp_acl_md_execute_async(model_id, input, output, stream);
//调用svp_acl_rt_synchronize_stream接口，阻塞应用程序运行，直到指定Stream中的所有任务都完成。
svp_acl_rt_synchronize_stream(stream);

//Stream使用结束后，显式销毁Stream
svp_acl_rt_destroy_stream(stream);
//.....
```

8.1.3 单线程多 Stream

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
int32_t device_id = 0 ;
uint32_t model_id1 = 0;
uint32_t model_id2 = 1;
svp_acl_rt_context context;
svp_acl_rt_stream stream1;
svp_acl_rt_stream stream2;

//如果只创建了一个Context，线程默认将这个Context作为线程当前的Context；
//如果是多个Context，则需要调用svp_acl_rt_set_current_context接口设置当前线程的Context
svp_acl_rt_create_context(&context, device_id);

svp_acl_rt_create_stream(&stream1);
//调用触发任务的接口，例如异步模型推理，任务下发在stream1
svp_acl_md_dataset *input1;
svp_acl_md_dataset *output1;
svp_acl_md_execute_async(model_id1, input1, output1, stream1);

svp_acl_rt_create_stream(&stream2);
//调用触发任务的接口，例如异步模型推理，任务下发在stream2
svp_acl_md_dataset *input2;
svp_acl_md_dataset *output2;
svp_acl_md_execute_async(model_id2, input1, output2, stream2);

// 流同步
svp_acl_rt_synchronize_stream(stream1);
svp_acl_rt_synchronize_stream(stream2);

//释放资源
svp_acl_rt_destroy_stream(stream1);
svp_acl_rt_destroy_stream(stream2);
svp_acl_rt_destroy_context(context);
//....
```

8.1.4 多线程多 Stream

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
void run_thread(svp_acl_rt_stream stream) {
    int32_t device_id = 0 ;
    svp_acl_rt_context context;

    //如果只创建了一个Context，线程默认将这个Context作为线程当前的Context；
    //如果是多个Context，则需要调用svp_acl_rt_set_current_context接口设置当前线程的Context
    svp_acl_rt_create_context(&context, device_id);
    svp_acl_rt_create_stream(&stream);

    //调用触发任务的接口
    //....

    //释放资源
    svp_acl_rt_destroy_stream(stream);
    svp_acl_rt_destroy_context(context);
}

svp_acl_rt_stream stream1;
svp_acl_rt_stream stream2;
```



```
//创建2个线程，每个线程对应一个Stream
std::thread t1(runThread, stream1);
std::thread t2(runThread, stream2);
//显式调用join函数确保结束线程
t1.join();
t2.join();
```

8.2 同步等待

8.2.1 原理介绍

SVP ACL提供以下几种同步机制：

- Event的同步等待：调用[svp_acl_rt_synchronize_event](#)接口，阻塞应用程序运行，等待Event完成。
- Stream内任务的同步等待：调用[svp_acl_rt_synchronize_stream](#)接口，阻塞应用程序运行，直到指定Stream中的所有任务都完成。
- Stream间任务的同步等待：调用[svp_acl_rt_stream_wait_event](#)接口，阻塞指定Stream的运行，直到指定的Event完成。支持多个Stream等待同一个Event的场景。
- Device的同步等待：调用[svp_acl_rt_synchronize_device](#)接口，阻塞应用程序运行，直到正在运算中的Device完成运算。多Device场景下，调用该接口等待的是当前Context对应的Device。

📖 说明

Hi3516CV610不支持“Event的同步等待”和“Stream间任务的同步等待”。

8.2.2 关于 Event 的同步等待

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
//创建一个Event
svp_acl_rt_event event;
svp_acl_rt_create_event(&event);

//创建一个Stream
svp_acl_rt_stream stream;
svp_acl_rt_create_stream(&stream);

//stream末尾添加一个event
svp_acl_rt_record_event(event, stream);

//阻塞应用程序运行，等待event发生，也就是stream执行完成
//stream完成后产生event，唤醒执行应用程序的控制流，开始执行程序
svp_acl_rt_synchronize_event(event);

//显示销毁资源
svp_acl_rt_destroy_stream(stream);
svp_acl_rt_destroy_event(event);
//.....
```

8.2.3 关于 Stream 内任务的同步等待

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
//显式创建一个Stream
svp_acl_rt_stream stream;
svp_acl_rt_create_stream(&stream);
uint32_t model_id = 0;

//调用触发任务的接口，例如异步模型推理，任务下发在stream1
svp_acl_mdl_dataset *input;
svp_acl_mdl_dataset *output;
svp_acl_mdl_execute_async(model_id, input, output, stream);
//调用svp_acl_rt_synchronize_stream接口，阻塞应用程序运行，直到指定Stream中的所有任务都完成。
svp_acl_rt_synchronize_stream(stream);

//Stream使用结束后，显式销毁Stream
svp_acl_rt_destroy_stream(stream);
//.....
```

8.2.4 关于 Stream 间任务的同步等待

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
//创建一个Event
svp_acl_rt_event event;
svp_acl_rt_create_event(&event);

//创建两个Stream
svp_acl_rt_stream s1;
svp_acl_rt_stream s2;
svp_acl_rt_create_stream(&s1);
svp_acl_rt_create_stream(&s2);

//s1末尾添加一个event
svp_acl_rt_record_event(event, s1);

//阻塞s2运行，直到指定event发生，也就是s1执行完成
//s1完成后，唤醒s2，继续执行s2的任务
svp_acl_rt_stream_wait_event(s2, event);

//阻塞应用程序运行，直到指定s2中的所有任务都完成
svp_acl_rt_synchronize_stream(s2);

//显示销毁资源
svp_acl_rt_destroy_stream(s2);
svp_acl_rt_destroy_stream(s1);
svp_acl_rt_destroy_event(event);
//.....
```

8.2.5 关于 Device 的同步等待

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
#include "acl/svp_acl.h"
//.....
//指定device
svp_acl_rt_set_device(0);

//创建context
svp_acl_rt_context ctx;
svp_acl_rt_create_context(&ctx, 0);

//创建stream
svp_acl_rt_stream stream;
svp_acl_rt_create_stream(&stream);

//阻塞应用程序运行，直到正在运算中的Device完成运算
svp_acl_rt_synchronize_device();

//资源销毁
svp_acl_rt_destroy_stream(stream);
svp_acl_rt_destroy_context(ctx);
svp_acl_rt_reset_device(0);
```

8.3 模型推理

8.3.1 单 Batch+固定 shape+单模型同步推理

1. 若涉及色域转换（转换图像格式）、图像归一化（减均值/乘系数）等，在模型加载前，需要先参见《ATC工具使用指南》转换模型。
2. 在模型推理前，需要从离线模型文件（适配SoC的离线模型）中加载模型数据到内存中，并创建svp_acl_md5_dataset类型的数据描述模型的输出。请参见“[模型推理资源申请](#)”。
3. 加载模型后，再同步执行模型推理。请参见“[模型推理](#)”。

8.3.2 异步推理+callback 回调处理

基本原理：

异步推理场景下，关键接口的调用流程如下：

1. 调用[svp_acl_init](#)接口初始化SVP ACL。
2. 按顺序调用[svp_acl_rt_set_device](#)、[svp_acl_rt_create_context](#)、[svp_acl_rt_create_stream](#)接口依次申请运行管理资源：[Device](#)、[Context](#)、[Stream](#)，确保可以使用这些资源执行运算、管理任务。

如果不显式创建Context和Stream，您可以使用[svp_acl_rt_set_device](#)接口隐式创建的默认Context和默认Stream，但默认Context和默认Stream存在如下限制：

- 一个Device对应一个默认Context，默认Context不能通过[svp_acl_rt_destroy_context](#)接口来释放。
- 一个Device对应一个默认Stream，默认Stream不能通过[svp_acl_rt_destroy_stream](#)接口来释放。默认Stream作为接口入参时，直接传NULL。

3. 申请模型推理资源。
 - a. 加载模型。[svp_acl_md5_load_from_mem](#)：从内存加载离线模型数据。



- b. 调用[svp_acl_md1_get_desc](#)接口获取成功加载的模型的描述信息。
- c. 初始化内存，用于存放模型推理的输入数据、输出数据。

此处需要用户可自行定义函数实现以下关键点：

- i. 调用[svp_acl_md1_create_dataset](#)接口创建[svp_acl_md1_dataset](#)类型的数据（描述模型的输入/输出）。
- ii. 模型可能存在多个输入/输出，调用[svp_acl_create_data_buffer](#)接口创建[svp_acl_data_buffer](#)类型的数据（描述每个输入/输出数据的内存地址、内存大小）。

模型输入数据的内存大小，根据实际读入的图片数据的大小来确定。

模型输出数据的内存大小，可调用

[svp_acl_md1_get_output_size_by_index](#)接口根据3.b中模型描述信息获取每个输出需占用的内存大小。

- iii. 调用[svp_acl_md1_add_dataset_buffer](#)接口向[svp_acl_md1_dataset](#)中增加[svp_acl_data_buffer](#)。

- 4. 执行模型异步推理和callback（用于处理模型推理的结果）。

此处需要用户可自行定义函数实现以下关键点：

- a. 创建新线程（例如t1），在线程函数内调用[svp_acl_rt_process_report](#)接口，等待指定时间后，触发回调函数（例如CallBackFunc，用于处理模型推理结果）。
- b. 调用[svp_acl_rt_subscribe_report](#)接口，指定处理Stream上回调函数（CallBackFunc）的线程（t1），回调函数需要提前创建，用于处理模型推理的结果（例如，将推理结果写入文件、从推理结果中获取topn置信度的类别标识等）。
- c. 调用[svp_acl_md1_execute_async](#)接口执行异步模型推理。
- d. 调用[svp_acl_rt_launch_callback](#)接口，在Stream的任务队列中增加一个需要在板端环境上执行的回调函数（CallBackFunc）。
- e. 调用[svp_acl_rt_synchronize_stream](#)接口，阻塞应用程序运行，直到指定Stream中的所有任务都完成。
- f. 调用[svp_acl_rt_unsubscribe_report](#)接口，取消线程注册，Stream上的回调函数（CallBackFunc）不再由指定线程（t1）处理。
- g. 模型推理结束后，调用[svp_acl_md1_unload](#)接口卸载模型。

- 5. 所有数据处理结束后，按顺序调用[svp_acl_rt_destroy_stream](#)、[svp_acl_rt_destroy_context](#)、[svp_acl_rt_reset_device](#)接口依次释放运行管理资源，包括Stream、Context、Device。

- 6. 调用[svp_acl_finalize](#)接口实现SVP ACL去初始化。

示例代码：

您可以从[acl_resnet50_async](#)样例中查看完整样例代码，以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

在[acl_resnet50_async](#)样例中：

– 运行可执行文件，不带参数时：

- 执行模型异步推理的次数默认为10次，对应代码中的g_executeTimes变量；
- callback间隔默认为1，表示1次异步推理后，下发一次callback任务，对应代码中的g_callbackInterval变量；



- 内存池中的内存块的个数默认为10个，对应代码中的 `g_memoryPoolSize`。
- 运行可执行文件，带参数时：
 - 第一个参数表示执行模型异步推理的次数；
 - 第二个参数表示下发callback间隔，参数值为0时表示不下发callback任务，参数值为非0值（例如m）时表示m次异步推理后下发一次callback任务；
 - 第三个参数表示内存池中内存块的个数，内存块个数需大于等于模型异步推理的次数。用户可根据输入图片数量，来调整内存块的个数，例如有2张输入图片、内存块个数为2时，则1张图片1个内存块；例如有3张输入图片、内存块个数为10时，则执行10次循环，每（10/3取整）个内存块对应同一张图片，剩下1个内存块随机对应1张图片。内存块中存放是模型推理的输入数据、输出数据，若多个内存块对应的是同一张图片，则多个内存块中存放的是相同的输入数据、输出数据，用于输入图片少但又想模拟大量图片数据的场景。

```
#include "acl/svp_acl.h"
//.....
//1. SVP ACL初始化
//此处的..表示相对路径，相对可执行文件所在的目录
//例如，编译出来的可执行文件存放在out目录下，此处的..就表示out目录的上一级目录
const char *aclConfigPath = "../src/acl.json";
svp_acl_error ret = svp_acl_init(aclConfigPath);

//2. 申请运行管理资源
extern bool g_isDevice;
ret = svp_acl_rt_set_device(deviceId_);
ret = svp_acl_rt_create_context(&context_, deviceId_);
ret = svp_acl_rt_create_stream(&stream_);
//获取当前svp_acl软件栈的运行模式，根据不同的运行模式，后续的内存申请、内存复制
等接口调用方式不同
svp_acl_rt_run_mode run_mode;
ret = svp_acl_rt_get_run_mode(&runMode);
g_isDevice = (runMode == ACL_DEVICE);

//3. 申请模型推理资源

//此处的..表示相对路径，相对可执行文件所在的目录
//例如，编译出来的可执行文件存放在out目录下，此处的..就表示out目录的上一级目录
const char* omModelPath = "../model/resnet50.om"

//3.1 加载模型
//根据模型文件获取模型执行时所需的权值内存大小、工作内存大小，并申请权值内存、
工作内存
ret = svp_acl_rt_malloc(&modelMemPtr_, modelMemSize_,
SVP_ACL_MEM_MALLOC_NORMAL_ONLY);

//加载离线模型文件，模型加载成功，返回标识模型的ID。
ret = svp_acl_mdload_from_mem(modelPath, &modelId_, modelMemPtr_,
modelMemSize_);

//3.2 根据模型的ID，获取该模型的描述信息
modelDesc_ = svp_acl_mdload_create_desc();
ret = svp_acl_mdload_get_desc(modelDesc_, modelId_);

//3.3 自定义函数InitMemPool，初始化内存池，存放模型推理的输入数据、输出数据
```



```

//-----自定义函数InitMemPool内部的关键实现-----
string testFile[] = {
    "../data/dog1_1024_683.bin",
    "../data/dog2_1024_683.bin"
};
size_t fileNum = sizeof(testFile) / sizeof(testFile[0]);
for (size_t i = 0; i < g_memoryPoolSize; ++i) {
    size_t index = i % (sizeof(testFile) / sizeof(testFile[0]));
    // model process
    uint32_t devBufferSize;
    //自定义函数GetDeviceBufferOfFile, 完成以下功能:
    //获取Host上存放输入图片数据的内存及内存大小、将图片数据从Host传输到
    Device、获取Device上存放输入图片数据的内存及内存大小
    void *picDevBuffer = Utils::GetDeviceBufferOfFile(testFile[index], devBufferSize);
    aclmdlDataset *input = nullptr;
    //自定义函数CreateInput, 创建aclmdlDataset类型的数据input, 用于存放模型推
    理的输入数据
    Result ret = CreateInput(picDevBuffer, devBufferSize, input);
    svp_acl_mdl_dataset *output = nullptr;
    //自定义函数CreateOutput, 创建svp_acl_mdl_dataset类型的数据output, 用于存
    放模型推理的输出数据, modelDesc表示模型的描述信息
    CreateOutput(output, modelDesc);
    {
        std::lock_guard<std::recursive_mutex> lk(freePoolMutex_);
        freeMemoryPool_[input] = output;
    }
}
//-----自定义函数InitMemPool内部的关键实现-----

//4 模型推理
//4.1 创建线程tid, 并将该tid线程指定为处理Stream上回调函数的线程
//其中ProcessCallback为线程函数, 在该函数内调用svp_acl_rt_process_report接口, 等
待指定时间后, 触发回调函数处理
pthread_t tid;
(void)pthread_create(&tid, nullptr, ProcessCallback, &s_isExit);

//4.2 指定处理Stream上回调函数的线程
svp_acl_error aclRt = svp_acl_rt_subscribe_report(tid, stream_);

//4.2 创建回调函数, 用户处理模型推理的结果, 由用户自行定义
void ModelProcess::CallBackFunc(void *arg)
{
    std::map<svp_acl_mdl_dataset *, svp_acl_mdl_dataset *> *dataMap =
        (std::map<svp_acl_mdl_dataset *, svp_acl_mdl_dataset *> *)arg;

    svp_acl_mdl_dataset *input = nullptr;
    svp_acl_mdl_dataset *output = nullptr;
    MemoryPool *memPool = MemoryPool::Instance();

    for (auto& data : *dataMap) {
        ModelProcess::OutputModelResult(data.second);
        memPool->FreeMemory(data.first, data.second);
    }

    delete dataMap;
}

//4.3 自定义函数ExecuteAsync, 执行模型推理
//-----自定义函数ExecuteAsync内部的关键实现-----
bool isCallback = (g_callbackInterval != 0);
size_t callbackCnt = 0;
std::map<svp_acl_mdl_dataset *, svp_acl_mdl_dataset *> *dataMap = nullptr;

```

```
svp_acl_mdl_dataset *input = nullptr;
svp_acl_mdl_dataset *output = nullptr;
MemoryPool *memPool = MemoryPool::Instance();
for (uint32_t cnt = 0; cnt < g_executeTimes; ++cnt) {
    if (memPool->mallocMemory(input, output) != SUCCESS) {
        ERROR_LOG("get free memory failed");
        return FAILED;
    }
    //执行异步推理
    svp_acl_error ret = svp_acl_mdl_execute_async(modelId_, input, output,
stream_);

    if (isCallback) {
        if (dataMap == nullptr) {
            dataMap = new std::map<svp_acl_mdl_dataset *, svp_acl_mdl_dataset *>;
            if (dataMap == nullptr) {
                ERROR_LOG("malloc list failed, modelId is %u", modelId_);
                memPool->FreeMemory(input, output);
                return FAILED;
            }
        }
        (*dataMap)[input] = output;
        callbackCnt++;
        if ((callbackCnt % g_callbackInterval) == 0) {
            //在Stream的任务队列中增加一个需要在Host上执行的回调函数
            ret = svp_acl_rt_launch_callback(CallBackFunc, (void *)dataMap,
SVP_ACL_CALLBACK_BLOCK, stream_);
            if (ret != ACL_SUCCESS) {
                ERROR_LOG("launch callback failed, index=%zu", callbackCnt);
                memPool->FreeMemory(input, output);
                delete dataMap;
                return FAILED;
            }
            dataMap = nullptr;
        }
    }
}
}
//-----自定义函数ExecuteAsync内部的关键实现-----

//4.4 对于异步推理，需阻塞应用程序运行，直到指定Stream中的所有任务都完成
svp_acl_rt_synchronize_stream(stream_);

//4.5 取消线程注册，Stream上的回调函数不再由指定线程处理
aclRt = svp_acl_rt_unsubscribe_report(static_cast<uint64_t>(tid), stream_);
s_isExit = true;
(void)pthread_join(tid, nullptr);

//5 释放运行管理资源
svp_acl_error ret = svp_acl_rt_destroy_stream(stream_);
ret = svp_acl_rt_destroy_context(context_);
ret = svp_acl_rt_reset_device(deviceId_);

//6 SVP ACL去初始化
ret = svp_acl_finalize();
//.....
```

8.3.3 多模型推理

多模型推理的基本流程与单模型类似，请参见[单Batch+固定shape+单模型同步推理](#)。

多模型推理与单模型推理的不同点如下：



- 关于模型加载，如果涉及多个模型，需调用多次模型加载接口。
[svp_acl_mdload_from_mem](#)：从内存加载离线模型数据。
- 关于模型推理，如果涉及多个模型，需调用多次模型推理接口。
调用[svp_acl_mdexecute](#)接口实现同步模型推理。

8.3.4 动态 AIPP

基本原理：

若模型推理时包含动态AIPP特性，在模型推理时，需调用SVP ACL提供的接口设置模型推理时需使用的AIPP配置，完整流程请参见[模型执行](#)。关键原理说明如下：

1. 加载模型。模型加载的详细流程，请参见[模型加载](#)，模型加载成功后，返回标识模型的ID。

对于动态AIPP，模型支持的AIPP模式已提前在构建模型时配置，在[模型加载](#)时，会新增一个输入（下文简称动态AIPP输入），在模型推理时通过该新增的输入提供具体的AIPP参数值。

例如，a输入的AIPP是动态的，在模型加载后，会有与a对应的b输入来描述a的AIPP配置信息。在模型执行时，准备a输入的数据结构请参见[准备模型执行的输入/输出数据](#)，准备b输入的数据结构、设置b输入的数据请依次参见2、3。

2. 创建[svp_acl_mdldataset](#)类型的数据，用于描述模型执行的输入、输出，详细调用流程请参见[准备模型执行的输入/输出数据](#)。

其中，动态AIPP输入的注意点如下：

- a. 申请动态AIPP输入对应的内存前，需要先调用[svp_acl_mdload_get_input_aipp_type](#)接口查询指定模型的指定输入是否有关联的动态AIPP输入，若有，则输出标识动态AIPP输入的index，该参数值可作为[svp_acl_mdload_set_aipp_by_input_index](#)接口的入参之一，来设置对应输入上的动态AIPP参数值；为避免在错误的输入上设置动态AIPP参数，用户可调用[svp_acl_mdload_get_input_name_by_index](#)接口先获取指定输入index的输入名称，根据输入名称所对应的index设置动态AIPP参数。
 - b. 调用[svp_acl_mdload_get_input_size_by_index](#)根据index获取输入内存大小。
 - c. 调用[svp_acl_rt_malloc](#)接口根据b中的大小申请内存。
申请动态AIPP输入对应的内存后，无需用户设置该内存中的数据（否则可能会导致业务异常），用户调用d中的接口后，系统会自动向该内存中填入数据。
 - d. 调用[svp_acl_create_data_buffer](#)接口创建[svp_acl_data_buffer](#)类型的数据，用于存放动态AIPP输入数据的内存地址、内存大小。
 - e. 调用[svp_acl_mdload_create_dataset](#)接口创建[svp_acl_mdldataset](#)类型的数据，并调用[svp_acl_mdload_add_dataset_buffer](#)接口向[svp_acl_mdldataset](#)类型的数据中增加[svp_acl_data_buffer](#)类型的数据。
3. 在成功加载模型之后，执行模型之前，设置动态AIPP参数。
 - a. 根据输入名称（固定为SVP_ACL_DYNAMIC_AIPP_NAME），获取模型中标识动态AIPP输入的index。
 - b. 调用[svp_acl_mdload_create_aipp](#)接口创建[svp_acl_mdload_aipp](#)类型。
 - c. 根据实际需求，调用[svp_acl_mdload_aipp](#)数据类型下的操作接口设置动态AIPP参数值。

- 动态AIPP场景下，`svp_acl_md1_set_aipp_input_format`接口、`svp_acl_md1_set_aipp_src_image_size`接口（设置原始图片的宽和高）必须调用。
- 调用`svp_acl_md1_set_input_aipp`接口设置模型推理时的动态AIPP数据。带有动态AIPP的模型输入必须配置动态AIPP参数。
 - 及时调用`svp_acl_md1_destroy_aipp`接口销毁`svp_acl_md1_aipp`类型。
4. 执行模型。
- 例如，调用`svp_acl_md1_execute`接口（同步接口）执行模型。

须知

- 模型中需要进行输入动态AIPP处理的数据节点，`svp_acl_md1_get_input_size_by_index`接口获取的内存大小是按照图像输入最大值来计算所得；设置动态AIPP输入的参数后，获取内存大小是按设置值计算所得。
- 动态AIPP和动态Batch同时使用时：
 - 动态AIPP只支持Batch为1的配置，不支持对多Batch的输入/输出分别配置动态AIPP参数。
 - 配置的动态AIPP参数同时作用于一次多Batch推理的所有输入/输出图片。
- 动态AIPP和动态分辨率同时使用时：
 - 若在设置动态AIPP参数时，需确保通过AIPP转换之后的宽、高与通过`svp_acl_md1_set_dynamic_hw_size`接口设置的宽、高相等。
 - 模型中需要进行动态AIPP处理的数据节点，其对应的最大输入宽高不能小于动态分辨率最大分辨率（宽、高）。
- 输出的动态AIPP流程和动态AIPP流程类似，输出动态AIPP只支持裁剪功能。

示例代码

调用接口后，需增加异常处理的分支，并记录报错日志、提示日志，此处不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
//1.模型加载，加载成功后，再设置动态AIPP参数值
//.....

//2.准备模型描述信息svp_acl_model_desc，准备模型的输入数据input和模型的输出数据output

//3.自定义函数，设置动态AIPP参数值
int model_set_dynamic_aipp()
{
    //3.1 获取标识动态AIPP输入的index
    size_t index;
    svp_acl_error ret;
    size_t input_num;
    svp_acl_md1_aipp *aipp_set = svp_acl_md1_create_aipp(1);

    //svp_acl_model_desc为svp_acl_md1_create_desc表示模型描述信息，根据1中加载成功的模型的ID，获取该模型
    //的描述信息
    input_num = svp_acl_md1_get_num_inputs(svp_acl_model_desc);

    for (index = 0; index < input_num; index++) {
        //获取输入的AIPP类型，model_id为模型ID，aipp_type为返回的AIPP类型，attached_index为关联的AIPP
        //输入
        ret = svp_acl_md1_get_input_aipp_type(model_id, index, &aipp_type, &attached_index);
        if (aipp_type != SVP_ACL_DATA_WITH_DYNAMIC_AIPP) continue;
        //设置动态AIPP参数值
    }
}
```

```
ret = svp_acl_md5_set_aipp_src_image_size(aipp_set, 256, 224);
ret = svp_acl_md5_set_aipp_input_format(aipp_set, SVP_ACL_YUV420SP_U8);
ret = svp_acl_md5_set_aipp_csc_params(aipp_set, 1, 1192, 2166, 0, 1192, -218, -547, 1192, 0, 1836, 0, 0, 0, 16, 128, 128);
ret = svp_acl_md5_set_aipp_rbuw_swap_switch(aipp_set, 0);
ret = svp_acl_md5_set_aipp_dtc_pixel_mean(aipp_set, 0, 0, 0, 0, 0);
ret = svp_acl_md5_set_aipp_dtc_pixel_min(aipp_set, 0, 0, 0, 0, 0);
ret = svp_acl_md5_set_aipp_pixel_var_reci(aipp_set, 1.0, 1.0, 1.0, 1.0, 0);
ret = svp_acl_md5_set_aipp_crop_params(aipp_set, 1, 2, 2, 224, 224, 0);
//设置对应输入的动态AIPP参数，也可以用svp_acl_md5_set_aipp_by_input_index(model_id, input, index, aipp_set)接口设置
ret = svp_acl_md5_set_input_aipp(model_id, input, attached_index, aipp_set);
}
ret = svp_acl_md5_destroy_aipp(aipp_set);

//.....
}

//4.自定义函数，执行模型
int model_execute(int index)
{
    svp_acl_error ret;
    //4.1 调用自定义函数，设置动态AIPP参数值
    ret = model_set_dynamic_aipp();
    //4.2 执行模型，model_id表示加载成功的模型的ID，input和output分别表示模型的输入和输出
    ret = svp_acl_md5_execute(model_id, input, output);
    //.....
}

//5.处理模型推理结果
//TODO
```

8.3.5 WorkBuf 内存复用

基本原理

在模型推理时，输出临时数据到WorkBuf时，可能输入内存已加载完成或者输出内存还没有使用，这个时候如果复用这部分输入或者输出内存，则可以减少WorkBuf，有利于内存小型化。关键步骤说明如下：

- 步骤1** 加载模型。模型加载的详细流程，请参见[模型加载](#)，模型加载成功后，返回标识模型的ID。
- 步骤2** 创建[svp_acl_md5_dataset](#)类型的数据，用于描述模型执行的输入、输出，详细调用流程请参见[准备模型执行的输入/输出数据](#)。

其中，WorkBuf内存复用的注意点如下：

1. 通过接口[svp_acl_md5_get_reuse_buffer_type](#)获取模型中WorkBuf与输入输出内存Buf复用的类型[svp_acl_buffer_reuse_type](#)。
2. 在申请输入输出内存时，需要根据复用类型[svp_acl_buffer_reuse_type](#)来按要求申请内存。比如说获取到的复用类型SVP_ACL_BUFFER_REUSE_INOUT_CONT，则需要所有非扩展的输入输出内存连续，这时可以按所有非扩展的输入输出内存大小之和来申请一大块内存，再通过各个输入输出大小来偏移来获取内存块，然后填充到输入输出内存集中。
3. 可以通过接口[svp_acl_md5_get_input_reuse_type](#)或者[svp_acl_md5_get_output_reuse_type](#)获取当前输入输出内存的共享类型。

- 步骤3** 执行模型。

例如，调用[svp_acl_md5_execute](#)接口（同步接口）执行模型。



步骤4 释放资源，卸载模型。

释放模型输入/输出信息、内存等资源，卸载模型。如果步骤2.2一次申请一大块连续内存，则释放时也只用一次释放。

----结束

须知

- 模型在离线生成的时候指定WorkBuf和输入输出内存的复用类型，具体操作请参考《ATC工具使用指南》。
- 当输入内存与WorkBuf复用时，模型执行过程中会破坏输入内存源数据，如果需要保证输入源数据的完整性，则模型离线生成过程中需要关闭输入内存复用选项。
- 支持动态AIPP的输入输出不支持与workBuf的内存复用。
- 数据安全场景下，不支持WorkBuf的内存复用。

示例代码

调用接口后，需增加异常处理的分支，并记录报错日志、提示日志，此处不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

```
//1.申请运行管理资源，包括设置用于计算的Device、创建Context、创建Stream
//.....

//2.模型加载，加载成功后，返回标识模型的model_id
//.....

//3.创建svp_acl_mdl_dataset类型的数据，用于描述模型的输入数据input_dataset、输出数据output_dataset
void *virt_addr = null;
size_t input_size[], output_size[]; /* 存放各个输入输出所需内存大小 */
size_t input_stride[], output_stride[]; /* 存放各个输入输出的stride大小 */

//3.1 根据模型描述符model_desc获取WorkBuf复用类型
svp_acl_buffer_reuse_type reuse_type = svp_acl_mdl_get_reuse_buffer_type(model_desc);

//3.2 根据复用类型申请内存,仅提供非扩展输入内存和输出内存都需要连续的样例
// input_num和output_num为非扩展输入输出的个数（可通过svp_acl_mdl_get_input_aipp_type和
svp_acl_mdl_get_output_aipp_type排除扩展的动态AIPP输入和输出）
static void svp_npu_reuse_inout_continue(void)
{
    svp_acl_data_buffer* data_buf = HI_NULL;
    size_t size = 0;
    int i;
    for (i = 0; i < input_num; i++) {
        size += input_size[i];
    }
    for (i = 0; i < output_num; i++) {
        size += output_size[i];
    }
    /* 申请size大小的内存 */
    svp_acl_rt_malloc_cached(&virt_addr, size, SVP_ACL_MEM_MALLOC_NORMAL_ONLY);
    for (i = 0; i < input_num; i++) {
        data_buf = svp_acl_create_data_buffer(virt_addr, input_size[i], input_stride[i]);
        svp_acl_mdl_add_dataset_buffer(input_dataset, data_buf);
        virt_addr += input_size[i];
    }
    for (i = 0; i < output_num; i++) {
        data_buf = svp_acl_create_data_buffer(virt_addr, output_size[i], out_stride[i]);
        svp_acl_mdl_add_dataset_buffer(output_dataset, data_buf);
        virt_addr += output_size[i];
    }
}
```

```
// 申请扩展的输入输出并加入dataset数据集，包括task_buf、work_buf以及可能存在的动态AIPP的输入和输出
// ...
}

if (reuse_type == SVP_ACL_BUFFER_REUSE_IN_CONT) {
    // 非扩展输入内存需要连续
} else if (reuse_type == SVP_ACL_BUFFER_REUSE_OUT_CONT) {
    // 非扩展输出内存需要连续
} else if (reuse_type == SVP_ACL_BUFFER_REUSE_IN_OUT_CONT) {
    // 非扩展输入内存需要连续，非扩展输出内存需要连续
} else if (reuse_type == SVP_ACL_BUFFER_REUSE_INOUT_CONT) {
    // 非扩展输入内存和输出内存都需要连续
    svp_npu_reuse_inout_continue();
} else if (reuse_type == SVP_ACL_BUFFER_REUSE_INOUTWORK_CONT) {
    // 非扩展输入内存、输出内存和workbuf都需要连续
} else {
    // 正常申请，不需要强调各输入输出内存的连续性
}

//4.填充输入并刷Cache
//TODO

//5.执行模型
ret = svp_acl_md_execute(model_id, input, output);

//6.输出内存刷Cache，处理模型推理结果
//TODO

//7.释放内存
svp_acl_rt_free(virt_addr);

//8.释放描述模型输入/输出信息、内存等资源，卸载模型
//.....

//9.释放运行管理资源
//.....
```

8.4 带 cache 属性的内存管理

基本原理：

通过[svp_acl_rt_malloc_cached](#)接口申请的内存支持cache缓存，对于频繁使用的内存，建议使用本接口分配内存，这样可以提高cpu的读写效率，提升系统性能，但需要用户处理cpu与硬件设备（如[图像分析引擎](#)）之间的数据一致性问题，SVP ACL提供了[svp_acl_rt_mem_flush](#)接口将cache中的数据刷新到ddr中，从而确保硬件设备（如[图像分析引擎](#)）可以从ddr中获取最新数据进行处理，SVP ACL还提供了[svp_acl_rt_mem_invalidate](#)接口将cache中的数据置为无效（如果cache被CPU写操作，调用该接口功能会将数据刷到ddr），从而确保cpu可以获取到ddr中的经[图像分析引擎](#)处理后的结果数据。

如果任务（例如，模型加载）不涉及cpu与硬件设备（如[图像分析引擎](#)）之间的数据交互，则不涉及两者之间的数据一致性问题，因此就不需要调用[svp_acl_rt_mem_flush](#)接口和[svp_acl_rt_mem_invalidate](#)接口。

示例代码：

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

示例中，运行管理资源申请与释放请参见“[运行管理资源申请](#)”和“[运行管理资源释放](#)”，模型加载的接口调用流程请参见“[模型加载](#)”，模型推理的接口调用流程、准备模型推理的输入/输出数据的接口调用流程请参见“[模型执行](#)”。



```
//1.申请运行管理资源，包括设置用于计算的Device、创建Context、创建Stream
//.....

//2.模型加载，加载成功后，返回标识模型的model_id
//.....

//3.创建svp_acl_mdl_dataset类型的数据，用于描述模型的输入数据input、输出数据output
void *input_dev_buffer = null;
void *output_dev_buffer = null;
size_t input_size;
size_t output_size;
size_t input_stride, output_stride;

//3.1 申请支持cache缓存的内存：input_dev_buffer用于存放模型推理的输入数据
svp_acl_error ret = svp_acl_rt_malloc_cached(&input_dev_buffer, input_size,
SVP_ACL_MEM_MALLOC_NORMAL_ONLY);
svp_acl_mdl_dataset *input = svp_acl_mdl_create_dataset();
svp_acl_data_buffer* input_data = svp_acl_create_data_buffer(input_dev_buffer, input_size, input_stride);
ret = svp_acl_mdl_add_dataset_buffer(input, input_data);
//将输入数据读入内存中，该自定义函数ReadFile由用户实现
read_file(file_name, input_dev_buffer, input_size);

//3.2 申请支持cache缓存的内存：output_dev_buffer用于存放模型推理的输出数据
ret = svp_acl_rt_malloc_cached(&output_dev_buffer, output_size,
SVP_ACL_MEM_MALLOC_NORMAL_ONLY);
svp_acl_mdl_dataset *output = svp_acl_mdl_create_dataset();
svp_acl_data_buffer* output_data = svp_acl_create_data_buffer(output_dev_buffer, output_size,
output_stride);
ret = svp_acl_mdl_add_dataset_buffer(output, output_data);

//4.把对应cache中的数据刷新到ddr
//示例以获取第一个输入为例，如果有多个输入，可调用svp_acl_mdl_get_dataset_num_buffers接口获取输入个数
svp_acl_data_buffer *input_data_buffer = svp_acl_mdl_get_dataset_buffer(input, 0);
void *input_data_buffer_addr = svp_acl_get_data_buffer_addr(input_data_buffer);
size_t input_data_buffer_size = svp_acl_get_data_buffer_size(input_data_buffer);
ret = svp_acl_rt_mem_flush(input_data_buffer_addr, input_data_buffer_size);

//5.执行模型
ret = svp_acl_mdl_execute(model_id, input, output);

//6.执行svp_acl_rt_mem_invalidate接口，把对应cache中的数据置为无效，确保cpu获取到的是ddr中的数据
//示例以获取第一个输出为例，如果有多个输出，可调用svp_acl_mdl_get_dataset_num_buffers接口获取输出个数
svp_acl_data_buffer *out_data_buffer = svp_acl_mdl_get_dataset_buffer(output, 0);
void *output_data_buffer_addr = svp_acl_get_data_buffer_addr(out_data_buffer);
size_t output_data_buffer_size = svp_acl_get_data_buffer_size(out_data_buffer);
ret = svp_acl_rt_mem_invalidate(output_data_buffer_addr, output_data_buffer_size);

//7.处理模型推理结果
//TODO

//8.释放内存
svp_acl_rt_free(input_dev_buffer);
svp_acl_rt_free(output_dev_buffer);

//9.释放描述模型输入/输出信息、内存等资源，卸载模型
//.....

//10.释放运行管理资源
//.....
```

8.5 推理使用模式识别 CPU 算子

基本原理：

SoC推理场景下，如果模型中有 *模式识别* Core不支持的算子，需要通过 *模式识别* CPU算子来实现，这时，异步推理场景下可以调用 `svp_acl_ext_process_aicpu_task` 接口来接收 *模式识别* CPU算子并在CPU上执行，同步推理场景下无需用户做特殊处理。下面就异步推理关于该接口的使用方式进行进一步介绍。

模式识别 CPU算子在CPU侧执行时可能会引入一些问题，因此用户需要关注以下几点：

1. 为了不影响整个推理业务，需要创建 *模式识别* CPU工作线程对 *模式识别* CPU算子进行处理，确保 *模式识别* CPU算子处理和推理业务同时在CPU侧运行，且可保证各自的执行效率。
2. 需要用户在 *模式识别* CPU工作线程内，调用 `svp_acl_ext_process_aicpu_task` 接口来接收 *模式识别* CPU算子并执行。
3. 用户场景差异比较大，需要根据自己业务需求来部署 *模式识别* CPU工作线程，例如线程数量，是否绑核。

示例代码：

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

示例中，异步推理流程请参见“[异步推理+callback回调处理](#)”。

```
//1.实现模式识别CPU工作线程处理函数
static bool thread_exit_flag = false;
void *cpu_func_async_test(void *param)
{
    while (thread_exit_flag == false) {
        svp_acl_ext_process_aicpu_task(1000);
    }
    return NULL;
}

//2.主线程创建模式识别CPU工作线程
static pthread_t g_tid;
pthread_create(&g_tid, NULL, cpu_func_async_test, NULL);

//3.异步推理流程
//.....

//4.释放线程
thread_exit_flag = true;
pthread_join(g_tid, NULL);
```

须知

- 编译代码的时候，需要匹配对应的头文件 `svp_acl_ext.h`。
- 含有模式识别CPU算子的模型在离线生成时，可配置在CPU处理时AI Core是否处于忙等。如果配置为忙等，则AI Core不会释放，等到CPU处理完成后继续AI Core任务；如果不是忙等，则NPU会在当前模型处理CPU任务时切换到其它Stream上处理其它模型的AI Core任务。具体配置可参考《ATC工具开发指南》的“功能配置选项”章节。
- 含有模式识别CPU算子并且配置为非忙等的模型在执行时，有一些上下文信息保存在模型的work buf中，所以不建议此模型的work buf和其它不在同一个流上的模型之间共享，否则可能会执行错误。

8.6 任务优先级

优先级调度：硬件是基于任务的优先级来实现调度的。调度特点：

- 任务优先级分为0~7级，数值越低，优先级越高，0的优先级最高。
- 不同优先级任务的调度策略发生在不同的Stream之间，以Stream当前运行任务的优先级为准。
- 高优先级任务先于低优先级任务进行调度。
- 同等优先级情况下，等待时间长的任务优先调度。
- 同等优先级同等等待时长下，任务所在任务流的SQ编号越小的优先调度（SQ编号可通过[Proc信息说明](#)中的“sq_id”字段获取）。

须知

- 由于硬件记录等待时长的时间精度为10.6ms，所以会出现两个下发时间差不超过10.6ms的任务被认定为等待相同时长的情况。
- 由于硬件记录的最大等待时长为339.2ms，所以会出现两个等待时间超过339.2ms的任务被认定为等待相同时长的情况。

优先级抢占：优先级抢占是为了保障高优先级的任务能实时优先运行。优先级抢占的特点如下：

- 优先级抢占发生在不同的Stream之间，高优先级的任务可以打断正在运行的低优先级任务。
- 低优先级任务被打断的时间边界是模型融合层执行完成，被打断后会保存现场并释放执行权。
- 被打断的任务会在高优先级任务都完成后重新被调度。

须知

- 优先级抢占场景下，高优先级任务需要开启优先级抢占使能，可参考[svp_acl_md1_config_attr](#)中SVP_ACL_MD1_PRIORITY_PREEMP_EN_BOOL的配置说明；
- 优先级抢占场景下，被抢占的低优先级模型需要在模型生成的时候开启优先级抢占使能，允许被高优先级任务打断，具体配置可参考《ATC工具使用指南》。
- 模型在被抢占时，有一些上下文信息保存在模型的work buf中，所以不建议被抢占模型的work buf和其它不在同一个流上的模型之间共享，否则可能会执行错误。
- 由于低优先级任务并不是被实时打断的，而是在低优先级模型融合层执行完后才被打断，所以对高优先级任务的实时性会有影响。减少低优先级模型层间融合可以一定程度缓解高优先级任务的实时性影响。
- 优先级抢占不支持模型Dump业务场景。
- 优先级抢占不支持模型Profiling业务场景，发生抢占时生成的Profiling数据可能在工具侧解析失败。

优先级提升：优先级提升是为了保障低优先级任务在等待一定时间后获得高优先级从而能得到任务执行权。优先级提升的特点如下：

- 优先级提升是基于任务的，开启优先级提升并设置超时后，任务在超时时间内没有被调度优先级会自动提升。
- 优先级提升有两种模式：优先级逐步提升和优先级最高提升；两种模式不能同时使用。
- 优先级逐步提升是指任务在超时时间没有得到调度，优先级会提升一级，一直到任务被调度结束。
- 优先级最高提升是指任务在超时时间没有得到调度，优先级会提升到最高优先级（0）。

须知

- 低优先级任务在算力够的情况下可以不用设置优先级提升，在算力不够的情况下可以设置合理的优先级逐步提升超时时间。
- 有强时间片要求的任务可以设置合理的优先级最高提升的超时时间（根据任务预期帧率）。
- 优先级提升目的是保障低优先级任务的执行权，如果有其它强时间片的任务，低优先级任务优先级提升可能对强时间片的任务的执行产生波动，请谨慎使用。
- 优先级为0的任务不支持优先级提升。
- 优先级提升场景任务达到超时提升时间时硬件会清空等待时间；任务优先级提升到0后仍得不到调度的任务会受提升超时时间影响，不建议多个优先级提升到0的任务满负荷运行。

模型优先级说明：

- 模型支持设置不同优先级/是否开启抢占低优先级任务/任务等待超时优先级提升一级/任务等待超时优先级提升到最高。
- 模型通过`svp_acl_mdload_from_mem`接口加载时，默认优先级为3，支持优先级抢占，不支持优先级提升。
- 可以通过`svp_acl_mdload_with_config`接口加载模型，设置模型优先级配置。
- 可以通过`svp_acl_mdload_set_model_config`接口动态设置模型优先级配置。

8.7 Profiling 性能数据采集

基本原理：

该章节下的接口用于Profiling采集性能数据，实现方式支持以下两种：

- 方式一：将采集到的Profiling数据写入文件，再使用Profiling工具解析该文件。
- 方式二：将采集到的Profiling数据解析后写入管道，由用户读入内存，再由用户调用ACL的接口获取性能数据。

实现方式详细说明请参考“[功能及约束说明](#)”。

示例代码1：

调用接口后，需增加异常处理的分支，并记录报错日志、提示日志，此处不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。



示例中，运行管理资源申请与释放、模型加载的接口调用流程、模型推理的接口调用流程、准备模型推理的输入/输出数据的接口调用流程请参见“[异步推理+callback回调处理](#)”。

```
//1.SVP ACL资源初始化、模型加载、数据创建

//2.创建管道（UNIX操作系统下需要引用C++标准库头文件unistd.h），用于读取以及写入模型订阅的数据
int sub_fd[2];
// 读管道指针指向sub_fd[0]，写管道指针指向sub_fd[1]
pipe(sub_fd);

//3.创建模型订阅的配置并且进行模型订阅
svp_acl_prof_subscribe_config *config = svp_acl_prof_create_subscribe_config(1, 0, &sub_fd[1]);
//模型订阅需要传入模型的model_id
svp_acl_prof_model_subscribe(model_id, config);

//4.实现管道读取订阅数据的函数
//4.1 自定义函数，实现从用户内存中读取订阅数据的函数
void get_model_info(void *data, uint32_t len) {
    uint32_t op_number = 0;
    uint32_t data_len = 0;
    //通过SVP ACL接口读取算子信息个数
    svp_acl_prof_get_op_num(data, len, &op_number);
    //遍历用户内存的算子信息
    for (int32_t i = 0; i < op_number; i++){
        //获取算子的模型id
        uint32_t model_id = svp_acl_prof_get_model_id(data, len, i);
        //获取算子的类型名称长度
        size_t op_type_len = 0;
        svp_acl_prof_get_op_type_len(data, len, i, &op_type_len);
        //获取算子的类型名称
        char op_type[op_type_len];
        svp_acl_prof_get_op_type(data, len, i, op_type, op_type_len);
        //获取算子的详细名称长度
        size_t op_name_len = 0;
        svp_acl_prof_get_op_name_len(data, len, i, &op_name_len);
        //获取算子的详细名称
        char op_name[op_name_len];
        svp_acl_prof_get_op_name(data, len, i, op_name, op_name_len);
        //获取算子的执行开始时间
        uint64_t op_start = svp_acl_prof_get_op_start(data, len, i);
        //获取算子的执行结束时间
        uint64_t op_end = svp_acl_prof_get_op_end(data, len, i);
        uint64_t op_duration = svp_acl_prof_get_op_duration(data, len, i);
    }
}

//4.2 自定义函数，实现从管道中读取数据到用户内存的函数
void *prof_data_read(void *fd) {
    //设置每次从管道中读取的算子信息个数
    uint64_t N = 10;
    //获取单位算子信息的大小（Byte）
    uint64_t buffer_size = 0;
    svp_acl_prof_get_op_desc_size(&buffer_size);
    //计算存储算子信息的内存的大小，并且申请内存
    uint64_t read_buf_len = buffer_size * N;
    char *read_buf = (char *)malloc(sizeof(char) * read_buf_len);
    //从管道中读取数据到申请的内存中，读取到的实际数据大小data_len可能小于buffer_size * N，如果管道中没有数据，默认会阻塞直到读取到数据为止
    uint32_t data_len = read(*(int*)fd, read_buf, read_buf_len);
    //读取数据到read_buf成功
    while (data_len > 0) {
        //调用4.1实现的函数解析内存中的数据
        get_model_info(read_buf, data_len);
        memset(read_buf, 0, buffer_size);
        data_len = read(*(int*)fd, read_buf, read_buf_len);
    }
    free(read_buf);
}
```



```
//5. 启动线程读取管道数据并解析
pthread_t sub_tid = 0;
pthread_create(&sub_tid, NULL, prof_data_read, &sub_fd[0]);

//6.执行模型
ret = svp_acl_mdl_execute(model_id, input, output);

//7.处理模型推理结果、释放资源、卸载模型、等待线程数据获取完毕
//8.取消订阅，释放订阅相关资源
svp_acl_prof_model_unsubscribe(model_id);
close(sub_fd[1]);
pthread_join(sub_tid, NULL);
close(sub_fd[0]);
//释放config指针
svp_acl_prof_destroy_subscribe_config(config);
//9. 释放运行管理资源，SVP ACL去初始化
//.....
```

示例代码2:

调用接口后，需增加异常处理的分支，示例代码中不一一列举。以下是关键步骤的代码示例，不可以直接拷贝编译运行，仅供参考。

示例中，运行管理资源申请与释放、模型加载的接口调用流程、模型推理的接口调用流程、准备模型推理的输入/输出数据的接口调用流程请参见“[异步推理+callback回调处理](#)”。

```
//1.SVP ACL资源初始化、模型加载、数据创建
//2.profilng初始化
//设置数据落盘路径
const char *prof_path = "/";
svp_acl_prof_init(prof_path, strlen(prof_path));

//3.进行profilng配置
uint32_t device_id_list[1] = {0};
//创建配置结构体
svp_acl_prof_config *config = svp_acl_prof_create_config(device_id_list, 1, 0, NULL,
SVP_ACL_PROF_AICORE_METRICS | SVP_ACL_PROF_AICPU);
svp_acl_prof_start(config);

//4.执行模型
ret = svp_acl_mdl_execute(model_id, input, output);

//5.处理模型推理结果、释放资源、卸载模型

//6.关闭profilng配置，释放配置资源，释放profilng组件资源
svp_acl_prof_stop(config);
svp_acl_prof_destroy_config(config);
svp_acl_prof_finalize();

//7.释放运行管理资源，SVP ACL去初始化.
//.....
```

JSON文件格式:

通过**svp_acl_init**指定带有Profiling配置的json文件也可以实现将采集到的Profiling数据写入文件的功能，文件格式如下:

```
{
  "profiler": {
    "switch": "on",
    "output": "output",
    "interval": "20",
    "aic_metrics": "ArithmeticUtilization",
    "aicpu": "on",
    "acl_api": "on"
  }
}
```

profiler参数配置说明：

- switch: Profiling开关, 取值on或off。可选参数。
on表示开启Profiling, off表示关闭Profiling; 如果缺失该参数或参数值不为on, 则表示关闭Profiling。
- output: Profiling性能数据在板端环境上的落盘路径。可选参数。
Profiling采集结束后, 在该目录下生成JOB开头目录, 存放Profiling采集的性能原始数据。支持配置绝对路径或相对路径(相对执行命令行时的当前路径):
 - 绝对路径配置以“/”开头, 例如: /home/output。
 - 相对路径配置直接以目录名开始, 例如: output。
 - 如果该处设置的目录不存在, 默认存放采集结果数据到应用工程可执行文件所在目录(确保安装时配置的运行用户具有该目录的读写权限)。

 注意

该参数指定的目录需要提前创建且确保安装时配置的运行用户具有读写权限。

- interval: Profiling采集间隔, 代表每interval次推理保存一次Profiling数据, 此配置建议不要设置太小, 否则可能由于落盘数据太慢而引起数据丢失。
- aic_metrics: 模式识别Core采集事件, 配置为ArithmeticUtilization代表采集模式识别Core性能数据, 否则为不采集。
- aicpu: 模式识别CPU采集事件, 配置为on代表采集模式识别CPU性能数据, 否则为不开启。
- acl_api: ACL API采集事件, 配置为on代表采集主要ACL API接口性能数据, 否则为不开启。

 注意

多次采集的场景, acl_api会采集每次的数据然后取平均值, aic的core性能数据会根据interval采样取平均, 可能会出现acl_api时间比aic时间短的现象(ms级别以下)。



9 SVP ACL API 参考

9.1 接口列表

一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持	
			Hi3519DV500/ Hi3516DV500	Hi3516C V610
系统配置	svp_acl_init	-	支持	支持
	svp_acl_finalize	-	支持	支持
Device管理	svp_acl_rt_set_device	-	支持	支持
	svp_acl_rt_reset_device	-	支持	支持
	svp_acl_rt_get_device	-	支持	支持
	svp_acl_rt_get_run_mode	-	支持	支持
	svp_acl_rt_get_device_count	-	支持	支持
Context管理	svp_acl_rt_create_context	-	支持	支持
	svp_acl_rt_destroy_context	-	支持	支持
	svp_acl_rt_set_current_context	-	支持	支持
	svp_acl_rt_get_current_context	-	支持	支持
Stream管理	svp_acl_rt_create_stream	-	支持	支持
	svp_acl_rt_destroy_stream	-	支持	支持
同步等待	svp_acl_rt_synchronize_device	-	支持	支持



一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持	
			Hi3519DV500/ Hi3516DV500	Hi3516C V610
	svp_acl_rt_synchronize_stream	-	支持	支持
	svp_acl_rt_subscribe_report	-	支持	支持
	svp_acl_rt_launch_callback	-	支持	支持
	svp_acl_rt_process_report	-	支持	支持
	svp_acl_rt_unsubscribe_report	-	支持	支持
	svp_acl_rt_set_op_wait_timeout	-	支持	支持
	svp_acl_rt_create_event	-	支持	不支持
	svp_acl_rt_create_event_with_flag	-	支持	不支持
	svp_acl_rt_destroy_event	-	支持	不支持
	svp_acl_rt_record_event	-	支持	不支持
	svp_acl_rt_reset_event	-	支持	不支持
	svp_acl_rt_query_event	-	支持	不支持
	svp_acl_rt_query_event_wait_status	-	支持	不支持
	svp_acl_rt_synchronize_event	-	支持	不支持
	svp_svp_acl_rt_event_elapsed_time	-	支持	不支持
	svp_acl_rt_stream_wait_event	-	支持	不支持
内存管理	svp_acl_rt_malloc	-	支持	支持
	svp_acl_rt_malloc_cached	-	支持	支持
	svp_acl_rt_mem_flush	-	支持	支持
	svp_acl_rt_mem_invalidate	-	支持	支持
	svp_acl_rt_free	-	支持	支持
	svp_acl_rt_malloc_host	-	支持	支持



一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持	
			Hi3519DV500/ Hi3516DV500	Hi3516C V610
	svp_acl_rt_free_host	-	支持	支持
	svp_acl_rt_malloc_sec	-	支持	不支持
	svp_acl_rt_free_sec	-	支持	不支持
模型加载与执行	svp_acl_mdl_load_from_mem	-	支持	支持
	svp_acl_mdl_execute	-	支持	支持
	svp_acl_mdl_execute_async	-	支持	支持
	svp_acl_mdl_unload	-	支持	支持
	svp_acl_mdl_set_dynamic_batch_size	-	支持	支持
	svp_acl_mdl_set_total_time	-	支持	支持
	svp_acl_mdl_init_dump	-	支持	支持
	svp_acl_mdl_set_dump	-	支持	支持
	svp_acl_mdl_finalize_dump	-	支持	支持
	svp_acl_mdl_set_config_opt	-	支持	支持
	svp_acl_mdl_get_config_opt	-	支持	支持
	svp_acl_mdl_load_with_config	-	支持	支持
	svp_acl_mdl_set_model_config	-	支持	支持
	svp_acl_mdl_set_input_aipp	-	支持	支持
	svp_acl_mdl_set_aipp_by_input_index	-	支持	支持
	svp_acl_mdl_set_output_aipp	-	支持	不支持
	svp_acl_mdl_get_first_aipp_info	-	支持	支持
	svp_acl_mdl_get_last_aipp_info	-	支持	不支持



一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持	
			Hi3519DV500/ Hi3516DV500	Hi3516C V610
	svp_acl_mdl_get_input_ainn_type	-	支持	支持
	svp_acl_mdl_get_output_ainn_type	-	支持	不支持
	svp_acl_mdl_get_dynamic_batch	-	支持	支持
	svp_acl_mdl_set_dynamic_hw_size	-	支持	支持
Profiling配置	svp_acl_prof_init	-	支持	支持
	svp_acl_prof_start	-	支持	支持
	svp_acl_prof_stop	-	支持	支持
	svp_acl_prof_finalize	-	支持	支持
	svp_acl_prof_model_subscribe	-	支持	支持
	svp_acl_prof_model_unsubscribe	-	支持	支持
	svp_acl_prof_get_op_desc_size	-	支持	支持
	svp_acl_prof_get_op_num	-	支持	支持
	svp_acl_prof_get_op_type_len	-	支持	支持
	svp_acl_prof_get_op_type	-	支持	支持
	svp_acl_prof_get_op_name_len	-	支持	支持
	svp_acl_prof_get_op_name	-	支持	支持
	svp_acl_prof_get_op_start	-	支持	支持
	svp_acl_prof_get_op_stop	-	支持	支持
	svp_acl_prof_get_op_duration	-	支持	支持
	svp_acl_prof_get_model_id	-	支持	支持
获取数据大小	svp_acl_data_type_size	-	支持	支持



一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持	
			Hi3519DV500/ Hi3516DV500	Hi3516C V610
SoC扩展接口	svp_acl_ext_process_aicpu_task	-	支持	支持
	svp_acl_ext_get_md1_aicpu_task_num	-	支持	支持
	svp_acl_ext_get_md1_net_type	-	支持	支持
数据类型及其操作接口	svp_acl_error	-	支持	支持
	svp_acl_data_type	-	支持	支持
	svp_acl_float16	-	支持	支持
	svp_acl_format	-	支持	支持
	svp_acl_rt_context	-	支持	支持
	svp_acl_rt_stream	-	支持	支持
	svp_acl_md1_io_dims	-	支持	支持
	svp_acl_data_buffer	svp_acl_create_data_buffer	支持	支持
		svp_acl_destroy_data_buffer	支持	支持
		svp_acl_get_data_buffer_addr	支持	支持
		svp_acl_get_data_buffer_size	支持	支持
		svp_acl_get_data_buffer_stride	支持	支持
		svp_acl_update_data_buffer	支持	支持
	svp_acl_md1_dataset	svp_acl_md1_create_dataset	支持	支持
		svp_acl_md1_destroy_dataset	支持	支持
		svp_acl_md1_add_dataset_buffer	支持	支持
		svp_acl_md1_get_dataset_num_buffers	支持	支持



一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持	
			Hi3519DV500/ Hi3516DV500	Hi3516C V610
	svp_acl_mdl_desc	svp_acl_mdl_get_dataset_buffer	支持	支持
		svp_acl_mdl_create_desc	支持	支持
		svp_acl_mdl_destroy_desc	支持	支持
		svp_acl_mdl_get_desc	支持	支持
		svp_acl_mdl_get_num_inputs	支持	支持
		svp_acl_mdl_get_num_outputs	支持	支持
		svp_acl_mdl_get_input_size_by_index	支持	支持
		svp_acl_mdl_get_output_size_by_index	支持	支持
		svp_acl_mdl_get_input_dims	支持	支持
		svp_acl_mdl_get_output_dims	支持	支持
		svp_acl_mdl_get_input_name_by_index	支持	支持
		svp_acl_mdl_get_output_name_by_index	支持	支持
		svp_acl_mdl_get_input_format	支持	支持
		svp_acl_mdl_get_output_format	支持	支持
		svp_acl_mdl_get_input_data_type	支持	支持
		svp_acl_mdl_get_output_data_type	支持	支持



一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持	
			Hi3519DV500/ Hi3516DV500	Hi3516C V610
		svp_acl_mdl_get_input_index_by_name	支持	支持
		svp_acl_mdl_get_output_index_by_name	支持	支持
		svp_acl_mdl_get_input_default_stride	支持	支持
		svp_acl_mdl_get_output_default_stride	支持	支持
		svp_acl_mdl_get_dynamic_hw	支持	支持
		svp_acl_mdl_get_input_class_id_by_index	支持	不支持
		svp_acl_mdl_get_output_class_id_by_index	支持	不支持
		svp_acl_mdl_get_reuse_buffer_type	支持	支持
		svp_acl_mdl_get_input_reuse_type	支持	支持
		svp_acl_mdl_get_output_reuse_type	支持	支持
	svp_acl_prof_config	svp_acl_prof_create_config	支持	支持
		svp_acl_prof_destroy_config	支持	支持
	svp_acl_prof_subscribe_config	svp_acl_prof_create_subscribe_config	支持	支持
		svp_acl_prof_destroy_subscribe_config	支持	支持



一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持	
			Hi3519DV500/ Hi3516DV500	Hi3516C V610
	svp_acl_mdl_config_attr	-	支持	支持
	svp_acl_mdl_config_handle	svp_acl_mdl_create_config_handle	支持	支持
		svp_acl_mdl_destroy_config_handle	支持	支持
	svp_acl_aipp_input_format svp_acl_aipp_info	svp_acl_mdl_get_first_aipp_info	支持	支持
	svp_acl_mdl_aipp	svp_acl_mdl_create_aipp	支持	支持
		svp_acl_mdl_set_aipp_input_format	支持	支持
		svp_acl_mdl_set_aipp_csc_params	支持	支持
		svp_acl_mdl_set_aipp_rbuvs_swap_switch	支持	支持
		svp_acl_mdl_set_aipp_ax_swap_switch	支持	支持
		svp_acl_mdl_set_aipp_src_image_size	支持	支持
		svp_acl_mdl_set_aipp_scf_params	支持	支持
		svp_acl_mdl_set_aipp_crop_params	支持	支持
		svp_acl_mdl_set_aipp_padding_params	支持	支持
		svp_acl_mdl_set_aipp_dtc_pixel_mean	支持	支持



一级目录	二级目录或接口或数据类型	三级目录或接口	是否支持	
			Hi3519DV500/ Hi3516DV500	Hi3516C V610
		svp_acl_md5_set_aipp_dtc_pixel_min	支持	支持
		svp_acl_md5_set_aipp_pixel_var_reci	支持	支持
		svp_acl_md5_destroy_aipp	支持	支持
	svp_acl_status	svp_acl_query_status	支持	支持

 **注意**

接口中涉及到创建与销毁的接口，如果创建的对象为指针类型则需要创建销毁成对出现。因为创建的接口会在接口内部申请内存，销毁的接口会释放内存，不允许在销毁后不给指针对象赋空的情况下重复销毁。

9.2 系统配置

9.2.1 svp_acl_init

函数功能：SVP ACL初始化函数，同步接口。

约束说明：

- 一个进程内可以多次调用**svp_acl_init**接口，但需要与**svp_acl_finalize**成对调用。
- 使用SVP ACL接口开发应用时，必须先调用**svp_acl_init**接口，进行内部资源初始化，否则可能会导致业务异常。
- 进程退出时需要调用**svp_acl_finalize**接口释放内部资源。

函数原型：

svp_acl_error svp_acl_init(const char *config_path)

参数说明：



参数名	输入/输出	说明
config_path	输入	配置文件所在的路径，包含文件名，配置文件内容为json格式（json文件内的“{”的层级最多为10，“[”的层级最多为10）。如果以下的默认配置已满足需求，无需修改，可向aclInit接口中传入NULL，或者可将配置文件配置为空json串（即配置文件中只有{}）。 配置文件格式为json格式，当前支持以下配置： • dump信息配置，示例、配置说明及约束请参见《ATC工具使用指南》“online_model_type”章节。默认不启动dump配置。dump功能开启除了配置json文件外，OM需要选择开启dump模式，如果OM开启dump后，json未配置dump，运行会报错，开启dump模式请参考《ATC工具使用指南》。 • profiling信息配置，配置说明请参见“Profiling性能数据采集”。默认不启动Profiling配置。

返回值说明：返回0表示成功，返回其它值表示失败。

9.2.2 svp_acl_finalize

函数功能：SVP ACL去初始化函数，用于释放进程内的SVP ACL内部管理的相关资源，同步接口。

约束说明：

- 用户申请的资源需要在调用svp_acl_finalize接口前通过调用对应的释放接口释放，防止未释放的资源造成内存泄露或者后续被误用而导致业务异常。
- 应用的进程退出前，应显式调用该接口实现SVP ACL去初始化，否则可能会导致异常，例如应用进程退出时有异常报错。
- 不建议在析构函数中调用svp_acl_finalize接口，否则在进程退出时可能由于单例析构顺序未知而导致进程异常退出的问题。

函数原型：

```
svp_acl_error svp_acl_finalize()
```

参数说明：无

返回值说明：返回0表示成功，返回其它值表示失败。

9.3 Device 管理



9.3.1 svp_acl_rt_set_device

函数功能：

- 指定用于运算的Device，同时隐式创建默认Context，该默认Context中包含1个默认Stream，同步接口。
- 如果多次调用svp_acl_rt_set_device接口而不调用[svp_acl_rt_reset_device](#)接口释放本进程使用的Device资源，功能上不会有问题，因为在进程退出时也会释放本进程使用的Device资源。建议svp_acl_rt_set_device接口和[svp_acl_rt_reset_device](#)接口配对使用，在不使用Device上资源时，通过调用[svp_acl_rt_reset_device](#)接口及时释放本进程使用的Device资源。

支持以下使用场景：

- 在不同进程或线程中可指定同一个Device用于运算。
- 在某一进程中指定Device，该进程内的多个线程可共用此Device显式创建Context（[svp_acl_rt_create_context](#)接口）。
- 多Device场景下，可在进程中通过svp_acl_rt_set_device接口切换到其它Device。但利用Context切换（调用[svp_acl_rt_set_current_context](#)接口）来切换Device，比使用svp_acl_rt_set_device接口效率高。

函数原型：

```
svp_acl_error svp_acl_rt_set_device(int32_t device_id)
```

参数说明：

参数名	输入/输出	说明
device_id	输入	Device ID。 用户调用 svp_acl_rt_get_device_count 接口获取可用的Device数量后，这个Device ID的取值范围：[0, (可用的Device数量-1)]

返回值说明：返回0表示成功，返回其它值表示失败。

9.3.2 svp_acl_rt_reset_device

函数功能：复位当前运算的Device，释放Device上的资源，包括默认Context、默认Stream以及默认Context下创建的所有Stream，同步接口。若默认Context或默认Stream下的任务还未完成，会删除任务，不再执行。

约束说明：

- 若要复位的Device上存在显式创建的Context、Stream、Event，在复位前，建议遵循如下接口调用顺序，否则可能会导致业务异常。
- **接口调用顺序：**调用[svp_acl_rt_destroy_event](#)接口释放Event/调用[svp_acl_rt_destroy_stream](#)接口释放显式创建的Stream-->调用[svp_acl_rt_destroy_context](#)释放显式创建的Context-->调用svp_acl_rt_reset_device接口

函数原型：

```
svp_acl_error svp_acl_rt_reset_device(int32_t device_id)
```



参数说明：

参数名	输入/输出	说明
device_id	输入	Device ID。

返回值说明：返回0表示成功，返回其它值表示失败。

9.3.3 svp_acl_rt_get_device

函数功能：获取当前正在使用的Device的ID，同步接口。

约束说明：如果没有调用**svp_acl_rt_set_device**接口显式指定Device或没有调用**svp_acl_rt_create_context**接口隐式指定Device，则调用svp_acl_rt_get_device接口时，返回错误。

函数原型：

```
svp_acl_error svp_acl_rt_get_device(int32_t *device_id)
```

参数说明：

参数名	输入/输出	说明
device_id	输出	Device ID。

返回值说明：返回0表示成功，返回其它值表示失败。

9.3.4 svp_acl_rt_get_run_mode

函数功能：获取当前svp_acl软件栈的运行模式，同步接口。

函数原型：

```
svp_acl_error svp_acl_rt_get_run_mode(svp_acl_rt_run_mode *run_mode)
```

参数说明：

参数名	输入/输出	说明
run_mode	输出	表示运行模式。 typedef enum svp_acl_rt_run_mode { //软件栈运行在SoC的CPU上 SVP_ACL_DEVICE, //SoC不支持 SVP_ACL_HOST, } svp_acl_rt_run_mode;

返回值说明：返回0表示成功，返回其它值表示失败。



9.3.5 svp_acl_rt_get_device_count

函数功能：获取可用Device的数量，同步接口。

函数原型：

```
svp_acl_error svp_acl_rt_get_device_count(uint32_t *count)
```

参数说明：

参数名	输入/输出	说明
count	输出	Device数量。

返回值说明：返回0表示成功，返回其它值表示失败。

9.4 Context 管理

9.4.1 svp_acl_rt_create_context

函数功能：显式创建一个Context，该Context中包含1个默认Stream，同步接口。

支持以下使用场景：

- 若不调用svp_acl_rt_create_context接口显式创建Context，那系统会使用默认Context，该默认Context是在调用[svp_acl_rt_set_device](#)接口时隐式创建的。
 - 隐式创建Context：适合简单、无复杂交互逻辑的应用，但缺点在于，在多线程编程中，执行结果取决于线程调度的顺序。
 - 显式创建Context：**推荐显式**，适合大型、复杂交互逻辑的应用，且便于提高程序的可读性、可维护性。
- 若在某一进程内创建多个Context（Context的数量与Stream相关，Stream数量有限制，请参见[svp_acl_rt_create_stream](#)），当前线程在同一时刻内只能使用其中一个Context，建议通过[svp_acl_rt_set_current_context](#)接口明确指定当前线程的Context，增加程序的可维护性。

函数原型：

```
svp_acl_error svp_acl_rt_create_context(svp_acl_rt_context *context, int32_t device_id)
```

参数说明：

参数名	输入/输出	说明
device_id	输入	指定需要创建context的Device的ID。
context	输出	创建的context的指针。

返回值说明：返回0表示成功，返回其它值表示失败。



9.4.2 svp_acl_rt_destroy_context

函数功能：销毁一个Context，释放Context的资源，同步接口。只能销毁通过[svp_acl_rt_create_context](#)接口创建的Context。

函数原型：

```
svp_acl_error svp_acl_rt_destroy_context(svp_acl_rt_context context)
```

参数说明：

参数名	输入/输出	说明
context	输入	指定需要销毁的context。

返回值说明：返回0表示成功，返回其它值表示失败。

9.4.3 svp_acl_rt_set_current_context

函数功能：设置线程的Context，同步接口。

支持以下场景：

- 如果在某线程（例如：thread1）中调用[svp_acl_rt_create_context](#)接口显式创建一个Context（例如：ctx1），则可以不调用[svp_acl_rt_set_current_context](#)接口指定该线程的Context，系统默认将ctx1作为thread1的Context。
- 如果没有调用[svp_acl_rt_create_context](#)接口显式创建Context，则系统将默认Context作为线程的Context，此时，不能通过[svp_acl_rt_destroy_context](#)接口来释放默认Context。
- 如果多次调用[svp_acl_rt_set_current_context](#)接口设置线程的Context，以最后一次为准。

约束说明：

- 若给线程设置的Context所对应的Device已经被复位，则不能将该Context设置为线程的Context，否则会导致业务异常。
- 推荐在某一线程中创建的Context，在该线程中使用。若在线程A中调用[svp_acl_rt_create_context](#)接口创建Context，在线程B中使用该Context，则需由用户自行保证两个线程中同一个Context下同一个Stream中任务执行的顺序。

函数原型：

```
svp_acl_error svp_acl_rt_set_current_context(svp_acl_rt_context context)
```

参数说明：

参数名	输入/输出	说明
context	输入	线程当前的context。

返回值说明：返回0表示成功，返回其它值表示失败。



9.4.4 svp_acl_rt_get_current_context

函数功能：

- 获取线程的Context，同步接口。
- 如果用户多次调用[svp_acl_rt_set_current_context](#)接口设置当前线程的Context，则获取的是最后一次设置的Context。

函数原型：

```
svp_acl_error svp_acl_rt_get_current_context(svp_acl_rt_context *context)
```

参数说明：

参数名	输入/输出	说明
context	输出	线程当前的context。

返回值说明：返回0表示成功，返回其它值表示失败。

9.5 Stream 管理

9.5.1 svp_acl_rt_create_stream

函数功能：创建一个Stream，同步接口。

约束说明：

- SoC-cs形态，硬件资源最多支持128个Stream(Hi3516CV610最多支持4个Stream)，如果已存在多个默认Stream，只能显式创建N个Stream（N=最大stream数目-默认Stream个数），例如在Hi3519DV500/Hi3516DV500平台，若已存在一个默认Stream，则只能显式创建127个Stream。
- 每个Context对应一个默认Stream，该默认Stream是调用[svp_acl_rt_set_device](#)接口或[svp_acl_rt_create_context](#)接口隐式创建的。推荐调用svp_acl_rt_create_stream接口显式创建Stream。
- 隐式创建Stream：适合简单、无复杂交互逻辑的应用，但缺点在于，在多线程编程中，执行结果取决于线程调度的顺序。
- 显式创建Stream：**推荐显式**，适合大型、复杂交互逻辑的应用，且便于提高程序的可读性、可维护性。

函数原型：

```
svp_acl_error svp_acl_rt_create_stream(svp_acl_rt_stream *stream)
```

参数说明：

参数名	输入/输出	说明
stream	输出	新创建的stream的指针。



返回值说明：返回0表示成功，返回其它值表示失败。

9.5.2 svp_acl_rt_destroy_stream

函数功能：销毁指定Stream，只能销毁通过[svp_acl_rt_create_stream](#)接口创建的Stream，同步接口。

约束说明：

- 在调用svp_acl_rt_destroy_stream接口销毁指定Stream前，需要先调用[svp_acl_rt_synchronize_stream](#)接口确保Stream中的任务都已完成。
- 调用svp_acl_rt_destroy_stream接口销毁指定Stream时，需确保该Stream在当前Context下。
- 在调用svp_acl_rt_destroy_stream接口销毁指定Stream时，需确保其它接口没有正在使用该Stream。

函数原型：

```
svp_acl_error svp_acl_rt_destroy_stream(svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
stream	输入	待销毁的stream。

返回值说明：返回0表示成功，返回其它值表示失败。

9.6 同步等待

9.6.1 svp_acl_rt_synchronize_device

函数功能：

- 阻塞应用程序运行，直到正在运算中的Device完成运算，同步接口。
- 多Device场景下，调用该接口等待的是当前Context对应的Device。

约束说明：一个进程内多线程场景下，不支持多线程并发调用svp_acl_rt_synchronize_device接口。

函数原型：

```
svp_acl_error svp_acl_rt_synchronize_device(void)
```

参数说明：无

返回值说明：返回0表示成功，返回其它值表示失败。

须知

Device同步会向所有有任务的Stream上发送流同步任务，请确保Stream上节点资源足够。



9.6.2 svp_acl_rt_synchronize_stream

函数功能：阻塞应用程序运行，直到指定Stream中的所有任务都完成，同步接口。

函数原型：

```
svp_acl_error svp_acl_rt_synchronize_stream(svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
stream	输入	指定需要完成所有任务的stream，如果为NULL则调用默认stream。

返回值说明：返回0表示成功，返回其它值表示失败。

须知

Stream同步也算Stream上的一个任务，请确保Stream上节点资源足够。

9.6.3 svp_acl_rt_subscribe_report

函数功能：注册处理Stream上回调函数的线程。同步接口。

约束说明：

- 支持多次调用svp_acl_rt_subscribe_report接口给多个Stream（仅支持同一Device内的多个Stream）注册同一个处理回调函数的线程；
- 为确保Stream内的任务按调用顺序执行，不支持调用svp_acl_rt_subscribe_report接口给同一个Stream注册多个处理回调函数的线程；
- 单进程内调用svp_acl_rt_subscribe_report接口注册的线程数量如果超过128个，则接口返回失败；
- 考虑操作系统的线程切换性能开销，建议调用svp_acl_rt_subscribe_report接口注册的线程数量控制在32个以下（包括32）；
- 同一个进程内，在不同的Device上注册回调函数的线程时，不能指定同一个线程ID。

函数原型：

```
svp_acl_error svp_acl_rt_subscribe_report(uint64_t thread_id, svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
threadId	输入	指定线程的ID。
stream	输入	指定Stream。



返回值说明：返回0表示成功，返回其它值表示失败。

9.6.4 svp_acl_rt_launch_callback

函数功能：在Stream的任务队列中增加一个需要在板端环境中执行的回调函数。异步接口。

函数原型：

```
svp_acl_error svp_acl_rt_launch_callback(svp_acl_rt_callback fn, void *user_data,
svp_acl_rt_callback_block_type blockType, svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
fn	输入	指定要增加的回调函数。 回调函数的函数原型为： typedef void (*svp_acl_rt_callback)(void *user_data)
user_data	输入	待传递给回调函数的用户数据，若fn不需要入参，user_data可以为空。
block_type	输入	指定回调函数调用是否阻塞Device运行。 typedef enum svp_acl_rt_callback_block_type { SVP_ACL_CALLBACK_NO_BLOCK, //非阻塞 SVP_ACL_CALLBACK_BLOCK, //阻塞 } svp_acl_rt_callback_block_type;
stream	输入	指定Stream，如果为NULL则调用默认stream。

约束说明：用户需要保证传入的fn和user_data的正确性，以及user_data所指向内存空间大小是否足够，否则处理回调函数时会导致程序异常。

返回值说明：返回0表示成功，返回其它值表示失败。

9.6.5 svp_acl_rt_process_report

函数功能：

- 在timeout时间内，触发回调处理，由svp_acl_rt_subscribe_report接口指定的线程处理回调。同步接口。
- 用户需新建一个线程，在线程函数内调用svp_acl_rt_process_report接口。

函数原型：

```
svp_acl_error svp_acl_rt_process_report(int32_t timeout)
```

参数说明：



参数名	输入/输出	说明
timeout	输入	超时时间，单位为ms。 取值范围： • -1：表示无限等待 • 大于0（不包含0）：表示等待的时间

返回值说明：返回0表示成功，返回其它值表示失败。

9.6.6 svp_acl_rt_unsubscribe_report

函数功能：取消线程注册，Stream上的回调函数不再由指定线程处理。同步接口。

函数原型：

svp_acl_error svp_acl_rt_unsubscribe_report(uint64_t thread_id, **svp_acl_rt_stream** stream)

参数说明：

参数名	输入/输出	说明
threadId	输入	指定线程的ID。
stream	输入	指定Stream，如果为NULL则调用默认stream。

返回值说明：返回0表示成功，返回其它值表示失败。

9.6.7 svp_acl_rt_set_op_wait_timeout

函数功能：

- 设置等待超时时间。同步接口。
- 对于需要等待Event、Stream、Device上任务完成的场景，可以调用本接口设置等待超时时间。设置超时时间后，如果等待任务完成的时间超过所设置的时间，则SVP ACL会返回报错。不调用本接口设置等待超时时间，则默认超时时间是120秒。

函数原型：

svp_acl_error svp_acl_rt_set_op_wait_timeout(uint32_t timeout)

参数说明：

参数名	输入/输出	说明
timeout	输入	设置超时时间，单位为秒。 将该参数设置为0时，表示不超时。 取值范围: [0, 4000000]。



返回值说明：返回0表示成功，返回其它值表示失败。

9.6.8 svp_acl_rt_create_event

函数功能：

创建一个Event。同步接口。

约束说明：

- 通过该接口创建的Event，Event数量有限，创建出来的Event可用于统计两个Event之间的耗时、多Stream场景下的同步等待、Event内的同步等待等场景。
- 单个Device上最多支持128个Event。

函数原型：

```
svp_acl_error svp_acl_rt_create_event(svp_acl_rt_event *event)
```

参数说明：

参数名	输入/输出	说明
event	输出	新创建的Event的指针。

返回值说明：返回0表示成功，返回其它值表示失败。

注：Hi3516CV610、仿真不支持该接口。

9.6.9 svp_acl_rt_create_event_with_flag

函数功能：

创建一个Event。同步接口。

约束说明：

- 通过该接口创建Event时，由于当前仅支持将flag设置为宏SVP_ACL_EVENT_TIME_LINE，表示创建出来的Event仅用于统计两个Event之间的耗时。
- 单个Device上创建的带flag的Event数量个数不超出4294967295个。

函数原型：

```
svp_acl_error svp_acl_rt_create_event_with_flag(svp_acl_rt_event *event, uint32_t flag)
```

参数说明：

参数名	输入/输出	说明
event	输出	新创建的Event的指针。



参数名	输入/输出	说明
flag	输入	Event指针的flag。 当前仅支持将flag设置为宏 SVP_ACL_EVENT_TIME_LINE。 #define SVP_ACL_EVENT_TIME_LINE 0x00000008u

返回值说明：返回0表示成功，返回其它值表示失败。

注：Hi3516CV610、仿真不支持该接口。

9.6.10 svp_acl_rt_destroy_event

函数功能：

销毁一个Event，只能销毁通过[svp_acl_rt_create_event](#)/
[svp_acl_rt_create_event_with_flag](#)接口创建的Event，同步接口。

约束说明：

- 在调用[svp_acl_rt_destroy_event](#)接口销毁指定Event时，需确保其它接口没有正在使用该Event。
- 调用[svp_acl_rt_create_event](#)接口创建的Event，在销毁时，用户需确保等待[svp_acl_rt_synchronize_event](#)接口或[svp_acl_rt_stream_wait_event](#)接口涉及的任务都结束后，再销毁。

函数原型：

```
svp_acl_error svp_acl_rt_destroy_event(svp_acl_rt_event event)
```

参数说明：

参数名	输入/输出	说明
event	输入	待销毁的Event。

返回值说明：返回0表示成功，返回其它值表示失败。

注：Hi3516CV610、仿真不支持该接口。

9.6.11 svp_acl_rt_record_event

函数功能：

在stream中记录一个Event，同步接口。

约束说明：

- [svp_acl_rt_record_event](#)接口与[svp_acl_rt_stream_wait_event](#)接口配合使用时，主要用于多Stream之间同步的场景，在调用[svp_acl_rt_record_event](#)接口时，系统内部会申请Event资源，在调用[svp_acl_rt_stream_wait_event](#)接口之后，请及时调用[svp_acl_rt_reset_event](#)接口释放Event资源。



- 接口调用顺序：[svp_acl_rt_create_event](#)-->[svp_acl_rt_record_event](#)-->[svp_acl_rt_stream_wait_event](#)-->[svp_acl_rt_reset_event](#)。

函数原型：

```
svp_acl_error svp_acl_rt_record_event(svp_acl_rt_event event, svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
event	输入	待记录的Event。
stream	输入	将指定Event记录在指定的Stream中。 如果使用默认Stream，此处设置为NULL。

返回值说明：返回0表示成功，返回其它值表示失败。

注：Hi3516CV610、仿真不支持该接口。

9.6.12 svp_acl_rt_reset_event

函数功能：

复位一个Event。用户需确保等待Stream中的任务都完成后，再复位Event，同步接口。

约束说明：

- 在复位Event时，涉及重置Event的状态，Event的状态是在调用[svp_acl_rt_record_event](#)接口时记录，因此在调用[svp_acl_rt_reset_event](#)接口前，需要先调用[svp_acl_rt_record_event](#)接口。
- 接口调用顺序：[svp_acl_rt_create_event](#)-->[svp_acl_rt_record_event](#)-->[svp_acl_rt_reset_event](#)。

函数原型：

```
svp_acl_error svp_acl_rt_reset_event(svp_acl_rt_event event, svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
event	输入	待复位的Event。
stream	输入	指定Event所在的Stream。

返回值说明：返回0表示成功，返回其它值表示失败。

注：Hi3516CV610、仿真不支持该接口。



9.6.13 svp_acl_rt_query_event

函数功能：

查询**svp_acl_rt_record_event**接口中记录的Event是否执行完成，同步接口。

函数原型：

```
svp_acl_error svp_acl_rt_query_event(svp_acl_rt_event event, svp_acl_rt_event_status *status)
```

参数说明：

参数名	输入/输出	说明
event	输入	指定待查询的Event。
status	输出	Event的状态。

返回值说明：返回0表示成功，返回其它值表示失败。

注：Hi3516CV610、仿真不支持该接口。

9.6.14 svp_acl_rt_query_event_wait_status

函数功能：

调用**svp_acl_rt_stream_wait_event**接口后，查询stream是否接收到指定Event。如果多个Stream等待同一个Event，则多个Stream都接收到Event，通过该接口查询的Event状态才是已完成。同步接口。

函数原型：

```
svp_acl_error svp_acl_rt_query_event_wait_status(svp_acl_rt_event event,  
svp_acl_rt_event_wait_status *status)
```

参数说明：

参数名	输入/输出	说明
event	输入	指定待查询的Event。
status	输出	Event的状态。

返回值说明：返回0表示成功，返回其它值表示失败。

注：Hi3516CV610、仿真不支持该接口

9.6.15 svp_acl_rt_synchronize_event

函数功能：

阻塞应用程序运行，等待Event完成，同步接口。

函数原型：

```
svp_acl_error svp_acl_error svp_acl_rt_synchronize_event(svp_acl_rt_event event)
```



参数说明：

参数名	输入/输出	说明
event	输入	指定需等待的Event。

返回值说明：返回0表示成功，返回其它值表示失败。

注：Hi3516CV610、仿真不支持该接口

9.6.16 svp_acl_rt_event_elapsed_time

函数功能：

统计两个Event之间的耗时，同步接口。

约束说明：

接口调用顺序：调用[svp_acl_rt_create_event](#)/[svp_acl_rt_create_event_with_flag](#)接口创建Event-->调用[svp_acl_rt_record_event](#)接口在同一个Stream中记录起始Event、结尾Event-->调用[svp_acl_rt_synchronize_stream](#)接口阻塞应用程序运行，直到指定Stream中的所有任务都完成-->调用[svp_acl_rt_event_elapsed_time](#)接口统计两个Event之间的耗时。

函数原型：

```
svp_acl_error svp_acl_rt_event_elapsed_time(float *ms, svp_acl_rt_event start_event, svp_acl_rt_event end_event)
```

参数说明：

参数名	输入/输出	说明
ms	输出	表示两个Event之间的耗时，单位为毫秒。
start_event	输入	起始Event。
end_event	输入	结尾Event。

返回值说明：返回0表示成功，返回其它值表示失败。

注：Hi3516CV610、仿真不支持该接口。

9.6.17 svp_acl_rt_stream_wait_event

函数功能：

阻塞指定Stream的运行，直到指定的Event的完成，同步接口。支持多个Stream等待同一个Event的场景。

约束说明：

- 只有通过[svp_acl_rt_create_event](#)接口创建的Event，才能使用此接口功能。
- [svp_acl_rt_record_event](#)接口与[svp_acl_rt_stream_wait_event](#)接口配合使用时，主要用于多Stream之间同步的场景，在调用[svp_acl_rt_record_event](#)接口



时，系统内部会申请Event资源，在调用[svp_acl_rt_stream_wait_event](#)接口之后，请及时调用[svp_acl_rt_reset_event](#)接口释放Event资源。

- 接口调用顺序：[svp_acl_rt_create_event](#)-->[svp_acl_rt_record_event](#)-->[svp_acl_rt_stream_wait_event](#)-->[svp_acl_rt_reset_event](#)。

函数原型：

[svp_acl_error](#) svp_acl_rt_stream_wait_event([svp_acl_rt_stream](#) stream, [svp_acl_rt_event](#) event)

参数说明：

参数名	输入/输出	说明
stream	输入	指定需要等待Event完成的Stream。如果使用默认Stream，此处设置为NULL。
event	输入	需等待的Event。

返回值说明：返回0表示成功，返回其它值表示失败。

注：Hi3516CV610、仿真不支持该接口。

9.7 内存管理

9.7.1 svp_acl_rt_malloc

函数功能：

- 申请Device上的内存，同步接口。
- 在Device上申请size大小的线性内存，通过*dev_ptr返回已分配内存的虚拟地址指针。
- 通过该接口申请的Device内存不支持cache缓存，如果用户需要申请支持cache缓存的内存，请参见[svp_acl_rt_malloc_cached](#)。

约束说明：

- 使用svp_acl_rt_malloc接口申请的内存，需要通过[svp_acl_rt_free](#)接口释放内存。
- 频繁调用svp_acl_rt_malloc接口申请内存、调用[svp_acl_rt_free](#)接口释放内存，会损耗性能，建议用户提前做内存预先分配或二次管理，避免频繁申请/释放内存。

函数原型：

[svp_acl_error](#) svp_acl_rt_malloc(void **dev_ptr, size_t size, svp_acl_rt_mem_malloc_policy policy)

参数说明：



参数名	输入/输出	说明
size	输入	申请内存的大小，单位Byte。 size不能为0。
policy	输入	内存分配规则。 typedef enum svp_acl_rt_mem_malloc_policy { SVP_ACL_MEM_MALLOC_HUGE_FIRST, // SOC不支持申请大页内存，使用该内存分配 规则，申请的是普通页的内存 SVP_ACL_MEM_MALLOC_HUGE_ONLY, // SOC不支持，使用该内存分配规则，申请的 是普通页的内存 SVP_ACL_MEM_MALLOC_NORMAL_ONLY , // 仅申请普通页 } svp_acl_rt_mem_malloc_policy;
dev_ptr	输出	指向Device上已分配内存的指针的指 针。

返回值说明：返回0表示成功，返回其它值表示失败。

9.7.2 svp_acl_rt_malloc_cached

函数功能：

- 申请Device上的内存，同步接口，该接口在任何场景下申请的内存都是支持cache缓存。
- 在Device上申请size大小的线性内存，通过*dev_ptr返回已分配内存的虚拟地址指针。
- 通过本接口分配的内存支持cache缓存，对于频繁使用的内存，建议使用本接口分配内存，这样可以提高cpu的读写效率，提升系统性能。当cpu访问此接口申请的内存时，会将内存中的数据放在cache中，而硬件设备（如图像分析引擎）只能访问物理内存，不能访问cache中内容，对于这种cpu和硬件会共同操作的内存，需要调用**svp_acl_rt_mem_flush**接口做好数据同步。

约束说明：其它约束与**svp_acl_rt_malloc**接口相同，但使用本接口申请的支持cache缓存的内存，需要用户显式处理cpu与**图像分析引擎**之间cache的一致性。

函数原型：

```
svp_acl_error svp_acl_rt_malloc_cached(void **dev_ptr, size_t size,  
svp_acl_rt_mem_malloc_policy policy)
```

参数说明：

参数名	输入/输出	说明
dev_ptr	输出	指向Device上已分配内存的指针的指 针。



参数名	输入/输出	说明
size	输入	申请内存的大小，单位Byte。 size不能为0。
policy	输入	内存分配规则。 typedef enum svp_acl_rt_mem_malloc_policy { SVP_ACL_MEM_MALLOC_HUGE_FIRST, // SOC不支持申请大页内存，使用该内存分配规则，申请的是普通页的内存 SVP_ACL_MEM_MALLOC_HUGE_ONLY, // SOC不支持, 使用该内存分配规则，申请的是普通页的内存 SVP_ACL_MEM_MALLOC_NORMAL_ONLY, // / 仅申请普通页 } svp_acl_rt_mem_malloc_policy;

返回值说明：返回0表示成功，返回其它值表示失败。

9.7.3 svp_acl_rt_mem_flush

函数功能：

- 将cache中的数据刷新到ddr中，并将cache中的内容设置成无效。
- 当cache里的数据为最新数据时，为了保证不能直接访问cache的硬件（如图像分析引擎）在访问内存时能够得到正确的数据，此时需要先调用本接口将cache里的内容更新到内存，这样，当硬件访问内存时，保证了数据的一致性和正确性。
- 此接口应与**svp_acl_rt_malloc_cached**接口配套使用。

函数原型：

```
svp_acl_error svp_acl_rt_mem_flush(const void *dev_ptr, size_t size)
```

参数说明：

参数名	输入/输出	说明
dev_ptr	输入	要flush的ddr内存起始地址。
size	输入	要flush的ddr内存大小，单位Byte。 size不能为0。

返回值说明：返回0表示成功，返回其它值表示失败。

9.7.4 svp_acl_rt_mem_invalidate

函数功能：

- 将cache中的数据设置成无效。



- 针对硬件输出的内存，如果此时cache中有预读引起的脏数据，硬件计算完成后，为了保证CPU后处理能获取到正确的数据，此时需要将cache中预读数据置无效。如果CPU对该内存执行过写操作，调用该接口会将cache中的数据同步到ddr。
- 此接口应与[svp_acl_rt_malloc_cached](#)接口配套使用。

函数原型：

```
svp_acl_error svp_acl_rt_mem_invalidate(const void *dev_ptr, size_t size)
```

参数说明：

参数名	输入/输出	说明
devPtr	输入	需要指定ddr内存对应的cache无效。
size	输入	ddr内存大小，单位Byte。 size不能为0。

返回值说明：返回0表示成功，返回其它值表示失败。

9.7.5 svp_acl_rt_free

函数功能：

- 释放Device上的内存，同步接口。
- svp_acl_rt_free接口只能释放通过[svp_acl_rt_malloc](#)接口或[svp_acl_rt_malloc_cached](#)接口申请的内存。如果应用在Devcie侧使用[svp_acl_rt_malloc_host](#)接口申请内存，也可以通过svp_acl_rt_free接口释放。

函数原型：

```
svp_acl_error svp_acl_rt_free(const void *dev_ptr)
```

参数说明：

参数名	输入/输出	说明
dev_ptr	输入	待释放内存的指针。

返回值说明：返回0表示成功，返回其它值表示失败。

9.7.6 svp_acl_rt_malloc_host

函数功能：申请Host或Device上的内存，Device上的内存按普通页申请。同步接口。

约束说明：

- 使用svp_acl_rt_malloc_host接口申请的内存，需要通过[svp_acl_rt_free_host](#)接口释放内存。
- 应用在Host上运行时，调用该接口申请的是Host内存，由系统保证内存首地址64字节对齐。应用在Device上运行时，调用该接口申请的是Device内存，由系统保证内存首地址64字节对齐。



- 频繁调用`svp_acl_rt_malloc_host`接口申请内存、调用`svp_acl_rt_free_host`接口释放内存，会损耗性能，建议用户提前做内存预先分配或二次管理，避免频繁申请/释放内存。

函数原型：

```
svp_acl_error svp_acl_rt_malloc_host(void **host_ptr, size_t size)
```

参数说明：

参数名	输入/输出	说明
size	输入	申请内存的大小，单位Byte。 size不能为0。
host_ptr	输出	“已分配内存的指针”的指针。

返回值说明：返回0表示成功，返回其它值表示失败。

9.7.7 svp_acl_rt_free_host

函数功能：

- 释放Host或Device上的内存，同步接口。
- 应用在Host侧，`svp_acl_rt_free_host`接口只能释放通过`svp_acl_rt_malloc_host`接口申请的内存。应用在Device侧，`svp_acl_rt_free_host`接口既可以释放`svp_acl_rt_malloc_host`接口申请的内存，也可以释放`svp_acl_rt_malloc`接口申请的内存。

函数原型：

```
svp_acl_error svp_acl_rt_free_host(const void *host_ptr)
```

参数说明：

参数名	输入/输出	说明
host_ptr	输入	待释放内存的指针。

返回值说明：返回0表示成功，返回其它值表示失败。

9.7.8 svp_acl_rt_malloc_sec

函数功能：申请Device上的安全内存。同步接口。

约束说明：

- 使用`svp_acl_rt_malloc_sec`接口申请的内存，需要通过`svp_acl_rt_free_sec`接口释放内存。
- 频繁调用`svp_acl_rt_malloc_sec`接口申请内存、调用`svp_acl_rt_free_sec`接口释放内存，会损耗性能，建议用户提前做内存预先分配或二次管理，避免频繁申请/释放内存。



- 该接口获取到的安全内存虚拟地址为安全内存句柄，不支持在REE侧进行访问，不支持flush等动作。在REE侧强行尝试读写存在设备挂死风险。在REE侧使用安全内存时必须整块使用，不支持内部分割处理。
- 该接口仅用于开启数据安全后存在模型要求输入/输出使用安全内存的场景。class_id说明详见《安全开发指南》。

函数原型：

```
svp_acl_error svp_acl_rt_malloc_sec(void **dev_ptr, size_t size, size_t class_id)
```

参数说明：

参数名	输入/输出	说明
size	输入	申请安全内存的大小，单位Byte。 size不能为0。
class_id	输入	需要申请的安全内存区域的class id。 class_id不能为0（非安全内存）。
dev_ptr	输出	“已分配内存的指针”的指针。

返回值说明：返回0表示成功，返回其它值表示失败。

注：仿真不支持该接口

9.7.9 svp_acl_rt_free_sec

函数功能：

- 释放Device上的安全内存，同步接口。
- svp_acl_rt_free_sec接口只能释放通过[svp_acl_rt_malloc_sec](#)接口申请的内存。
- 该接口仅用于开启数据安全后存在模型要求输入/输出使用安全内存的场景。

函数原型：

```
svp_acl_error svp_acl_rt_free_sec(const void *dev_ptr)
```

参数说明：

参数名	输入/输出	说明
dev_ptr	输入	待释放内存的指针。

返回值说明：返回0表示成功，返回其它值表示失败。

注：仿真不支持该接口。

**须知**

- 用户通过接口[svp_acl_rt_malloc](#)、接口[svp_acl_rt_malloc_cached](#)以及接口[svp_acl_rt_malloc_host](#)申请的内存为MMZ内存，可以通过系统的proc信息（`cat /proc/umap/media-mem`）查看内存情况。这些接口申请的内存统一以"svp_npu"作为MMZ的名字，如果需要加以区分，建议采用MMZ专用内存申请接口申请。

9.8 模型加载与执行

9.8.1 svp_acl_mdl_load_from_mem

函数功能：

- 从内存加载离线模型数据，加载的模型是独立的，不依赖于device，contxt等资源，可以被下发到不同的stream上执行，同步接口。
- 系统完成模型加载后，返回的模型ID，作为后续操作时用于识别模型的标志。

约束说明：

- 最多可加载64(Hi3519DV500)/32(Hi3516CV610)个模型。
- 输入/输出所有维度乘积值不能超过0xFFFFFFFF。
- 生成.om文件时配置的[internal_stride]项（详细参见《ATC工具使用指南》）必须大于0且为16的倍数（倍数须为2的非负整数次幂），最大不能超过256。
- Recurrent网络生成.om文件时配置的[recurrent_max_total_t]项（详细参见《ATC工具使用指南》）必须大于0，最大不能超过1024。
- 输入/输出默认stride按照生成.om文件时配置的[internal_stride]的值对齐后不能超过0xFFFFFFFF（详细输入/输出默认stride计算方法请参见[svp_acl_mdl_get_input_default_stride](#)和[svp_acl_mdl_get_output_default_stride](#)）。
- 输入/输出按照默认stride计算得到的单batch内存大小不能超过0xFFFFFFFF（详细输入/输出内存计算方法请参见[svp_acl_mdl_get_input_size_by_index](#)和[svp_acl_mdl_get_output_size_by_index](#)）。
- Recurrent网络data层个数范围：[2, 5]。
- Recurrent网络不支持前后接模式识别CPU算子。
- 模型每个执行段data层和report层个数总和不能超过32。

函数原型：

```
svp_acl_error svp_acl_mdl_load_from_mem(const void* model,size_t model_size, uint32_t* model_id)
```

参数说明：



参数名	输入/输出	说明
model	输入	模型数据的内存地址。用户要保证内存中存储的模型数据的完整性和正确性，在调用 svp_acl_mdload 接口卸载模型之前内存不能被释放且内存中的模型数据不能被修改。
model_size	输入	内存中的模型数据长度，单位Byte。
model_id	输出	系统完成模型加载后生成的模型ID。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.2 svp_acl_mdload_execute

函数功能：执行模型推理，直到返回推理结果，同步接口。

约束说明：

- input中的task_buf（见“[准备模型执行的输入/输出数据](#)”）在任务模型推理完成前不能被改写也不能被其他任务使用。
- input中的work_buf（见“[准备模型执行的输入/输出数据](#)”）不能被其他stream上的任务使用。
- 在模型推理完成之前，不能调用[svp_acl_mdload_destroy_dataset](#)接口释放input或output，不能调用[svp_acl_update_data_buffer](#)接口更新input或者output。

函数原型：

```
svp_acl_error svp_acl_mdload_execute(uint32_t model_id, const svp_acl_mdload_dataset *input, const svp_acl_mdload_dataset *output)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	指定需要执行推理的模型的ID。 调用 svp_acl_mdload_load_from_mem 接口加载模型成功后，会返回模型ID。
input	输入	模型推理的输入数据。 若用户使用 svp_acl_rt_malloc 或 svp_acl_rt_malloc_host 接口申请大块内存并自行划分、管理内存时，用户在管理内存时，模型输入数据的内存有对齐和补齐要求，首地址和stride需要16字节对齐。



参数名	输入/输出	说明
output	输出	模型推理的输出数据。 若用户使用 svp_acl_rt_malloc 接口申请大块内存并自行划分、管理内存时，用户在管理内存时，模型输出数据的内存有对齐和补齐要求，首地址和stride需要16字节对齐。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.3 svp_acl_mdl_execute_async

函数功能：执行模型推理，异步接口。

约束说明：

- input中的task_buf（见“[准备模型执行的输入/输出数据](#)”）在任务模型推理完成前不能被改写也不能被其他任务使用。
- input中的work_buf（见“[准备模型执行的输入/输出数据](#)”）不能被其他stream上的任务使用。
- 在模型推理完成之前，不能调用[svp_acl_mdl_destroy_dataset](#)接口释放input或output，不能调用[svp_acl_update_data_buffer](#)接口更新input或者output。

函数原型：

```
svp_acl_error svp_acl_mdl_execute_async(uint32_t model_id, const svp_acl_mdl_dataset *input, const svp_acl_mdl_dataset *output, svp_acl_rt_stream stream)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	指定需要执行推理的模型的ID。 调用 svp_acl_mdl_load_from_mem 接口加载模型成功后，会返回模型ID。
input	输入	模型推理的输入数据。 若用户使用 svp_acl_rt_malloc 或 svp_acl_rt_malloc_host 接口申请大块内存并自行划分、管理内存时，用户在管理内存时，模型输入数据的内存有对齐和补齐要求，首地址和stride需要16字节对齐。
output	输出	模型推理的输出数据。 若用户使用 svp_acl_rt_malloc 接口申请大块内存并自行划分、管理内存时，用户在管理内存时，模型输出数据的内存有对齐和补齐要求，首地址和stride需要16字节对齐。



参数名	输入/输出	说明
stream	输入	指定stream。stream为NULL则使用默认stream。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.4 svp_acl_mdl_unload

函数功能：系统完成模型推理后，可调用该接口卸载模型，释放资源，同步接口。

约束说明：

- 在调用svp_acl_mdl_unload接口卸载指定模型时，需确保其它接口没有正在使用该模型。
- 不建议在析构函数中调用svp_acl_mdl_unload接口，否则在进程退出时可能由于单例析构顺序未知而导致进程异常退出的问题。

函数原型：

```
svp_acl_error svp_acl_mdl_unload(uint32_t model_id)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	指定需卸载的模型的ID。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.5 svp_acl_mdl_set_dynamic_batch_size

函数功能：用于设置模型推理时的批量大小Batch（每次处理图片的数量）。

约束说明：

- 配置batch_size时，batch_size需大于0，若输入为图片类型，不能超过256；若输入为非图片数据，不能超过5000。如果执行模型前没有调用该接口配置batch_size到dataset，则将按照dataset中默认的batch值（默认值为1）执行。
- batch_size值约束请查看《ATC工具使用指南》中batch_num参数说明。

函数原型：

```
svp_acl_error svp_acl_mdl_set_dynamic_batch_size(uint32_t model_id, svp_acl_mdl_dataset *dataset, size_t index, uint64_t batch_size)
```

参数说明：



参数名	输入/输出	说明
model_id	输入	模型ID。 调用 svp_acl_mdl_load_from_mem 接口加载模型成功后，会返回模型ID。
dataset	输入&输出	表示模型的输入数据。 使用svp_acl_mdl_dataset类型的数据描述模型推理时的输入数据，输入的内存地址、内存大小用 svp_acl_data_buffer 类型的数据来描述。
index	输入	标识动态Batch输入的输入index，需调用 svp_acl_mdl_get_input_index_by_name 接口获取，输入名称固定为SVP_ACL_DYNAMIC_TENSOR_NAME。
batch_size	输入	指定模型推理时的批量大小Batch。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.6 svp_acl_mdl_set_total_t

函数功能：用于设置Recurrent模型推理时要处理的总帧数。

约束说明：配置的total_t不能为0，也不能超过编译om时[recurrent_max_total_t]项配置的值（详细参见《ATC工具使用指南》）。执行Recurrent网络前可以调用该接口将total_t配置到dataset中，如果没有配置，将把编译om时[recurrent_max_total_t]项配置的值作为T维度长度进行内存校验和网络执行。

函数原型：

```
svp_acl_error svp_acl_mdl_set_total_t(uint32_t model_id, svp_acl_mdl_dataset *dataset, uint64_t total_t)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	模型ID。 调用 svp_acl_mdl_load_from_mem 接口加载模型成功后，会返回模型ID。



参数名	输入/输出	说明
dataset	输入&输出	表示模型的输入数据。 使用svp_acl_mdl_dataset类型的数据描述模型推理时的输入数据，输入的内存地址、内存大小用 svp_acl_data_buffer 类型的数据来描述。
total_t	输入	指定Recurrent推理时的总帧数。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.7 svp_acl_mdl_init_dump

函数功能：

- Dump初始化。
- svp_acl_mdl_init_dump接口需要与[svp_acl_mdl_set_dump](#)接口、[svp_acl_mdl_finalize_dump](#)接口配合使用，用于配置Dump信息。
- 接口的调用顺序如下，以配置两个模型的Dump信息为例：[svp_acl_init](#)接口-->[svp_acl_mdl_init_dump](#)接口-->[svp_acl_mdl_set_dump](#)接口-->模型1加载-->[svp_acl_mdl_finalize_dump](#)接口-->[svp_acl_mdl_init_dump](#)接口-->[svp_acl_mdl_set_dump](#)接口-->模型2加载-->[svp_acl_mdl_finalize_dump](#)接口-->执行其它任务-->模型卸载-->[svp_acl_finalize](#)接口

约束说明：

- 如果已经通过[svp_acl_init](#)接口配置了dump信息，则调用svp_acl_mdl_init_dump接口时会返回失败。
- 必须在调用[svp_acl_init](#)接口之后、模型加载接口之前调用svp_acl_mdl_init_dump接口。

函数原型：

```
svp\_acl\_error svp_acl_mdl_init_dump()
```

参数说明：无

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.8 svp_acl_mdl_set_dump

函数功能：

- 设置dump参数。
- svp_acl_mdl_set_dump接口需要与[svp_acl_mdl_init_dump](#)接口、[svp_acl_mdl_finalize_dump](#)接口配合使用，用于配置Dump信息。
- 接口的调用顺序如下，以配置两个模型的Dump信息为例：[svp_acl_init](#)接口-->[svp_acl_mdl_init_dump](#)接口-->[svp_acl_mdl_set_dump](#)接口-->模型1加载-->[svp_acl_mdl_finalize_dump](#)接口-->[svp_acl_mdl_init_dump](#)接口-->[svp_acl_mdl_set_dump](#)接口-->模型2加载-->[svp_acl_mdl_finalize_dump](#)接口-->执行其它任务-->模型卸载-->[svp_acl_finalize](#)接口



约束说明：

- 只有在调用本接口之后加载模型，配置的Dump信息有效。在调用本接口之前已经加载的模型不受影响，除非用户在调用本接口后重新加载该模型。
例如以下接口调用顺序中，加载的模型1不受影响，配置的Dump信息仅对加载的模型2有效：
`svp_acl_md1_init_dump`接口-->模型1加载-->`svp_acl_md1_set_dump`接口-->模型2加载-->`svp_acl_md1_finalize_dump`接口
- 多次调用本接口对同一个模型配置了Dump信息，系统内处理时会采用覆盖策略。
例如以下接口调用顺序中，第二次调用本接口配置的Dump信息会覆盖第一次配置的Dump信息：
`svp_acl_md1_init_dump`接口-->`svp_acl_md1_set_dump`接口-->`svp_acl_md1_set_dump`接口-->模型1加载-->`svp_acl_md1_finalize_dump`接口
- Dump功能开启除了配置json文件，OM需要选择开启Dump模式的OM，开启Dump模式请参考《ATC使用指南》“online_model_type”的相关描述。

函数原型：

`svp_acl_error` `svp_acl_md1_set_dump(const char *dump_cfg_path)`

参数说明：

参数名	输入/输出	说明
dump_cfg_path	输入	配置文件所在的路径，包含文件名。 配置文件格式为json格式，当前可配置dump数据的相关信息，示例请参见 配置文件示例 ，详细配置说明请参见《精度比对工具使用指导》中的比对数据准备>准备离线模型Dump数据；如果不涉及配置信息，需向 <code>svp_acl_md1_set_dump</code> 接口中传入NULL。

配置文件示例：

以Caffe ResNet-50网络为例，若需要比对Caffe ResNet-50网络与基于Caffe ResNet-50转换成的适配SoC的离线模型中某些层算子的输出结果，可以在配置文件中配置如下内容：

```
{
  "dump":{
    "dump_list":[
      {
        "model_name":"ResNet-50",
        "layer":[
          "conv1conv1_relu",
          "res2a_branch2ares2a_branch2a_relu",
          "res2a_branch1",
          "pool1"
        ]
      }
    ],
    "dump_path":"/MyApp20/dump",
  }
}
```



```
        "dump_mode":"output"  
    }  
}
```

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.9 svp_acl_mdl_finalize_dump

函数功能：

- Dump去初始化。
- svp_acl_mdl_finalize_dump接口需要与[svp_acl_mdl_init_dump](#)接口、[svp_acl_mdl_set_dump](#)接口配合使用，用于配置Dump信息。
- 接口的调用顺序如下，以配置两个模型的Dump信息为例：[svp_acl_init](#)接口-->[svp_acl_mdl_init_dump](#)接口-->[svp_acl_mdl_set_dump](#)接口-->模型1加载-->[svp_acl_mdl_finalize_dump](#)接口-->[svp_acl_mdl_init_dump](#)接口-->[svp_acl_mdl_set_dump](#)接口-->模型2加载-->[svp_acl_mdl_finalize_dump](#)接口-->执行其它任务-->模型卸载-->[svp_acl_finalize](#)接口

函数原型：

```
svp_acl_error svp_acl_mdl_finalize_dump()
```

参数说明：无

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.10 svp_acl_mdl_set_config_opt

函数功能：

设置模型加载的配置对象中的各属性的取值，包括模型执行的优先级、模型的内存地址、内存大小等。

约束说明：

按如下接口调用顺序可实现模型加载的功能：

1. 调用[svp_acl_mdl_create_config_handle](#)接口创建模型加载的配置对象。
2. 多次调用[svp_acl_mdl_set_config_opt](#)接口设置配置对象中每个属性的值。
3. 调用[svp_acl_mdl_load_with_config](#)接口指定模型加载时需要的配置信息，并进行模型加载。
4. 模型加载成功后，调用[svp_acl_mdl_destroy_config_handle](#)接口销毁。
5. 值得注意的是通过[svp_acl_mdl_load_with_config](#)接口加载的模型同样需要调用[svp_acl_mdl_unload](#)接口进行模型卸载。

函数原型：

```
svp_acl_error svp_acl_mdl_set_config_opt(svp\_acl\_mdl\_config\_handle *handle,  
svp\_acl\_mdl\_config\_attr attr, const void *attr_value, size_t value_size)
```

参数说明：



参数名	输入/输出	说明
handle	输入	指定模型加载的配置对象。需提前调用 svp_acl_mdl_create_config_handle 接口创建该对象。
attr	输入	指定需设置的属性。
attr_value	输入	指向属性值的指针，attr对应的属性取值。如果属性值本身是指针，则传入该指针的地址。
value_size	输入	attr_value部分的数据长度。 用户可使用C/C++标准库的函数 <code>sizeof(*attr_value)</code> 查询数据长度。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.11 svp_acl_mdl_get_config_opt

函数功能：

获取模型加载的配置对象中的各属性的取值，包括模型执行的优先级、模型的内存地址、内存大小等。

函数原型：

```
svp_acl_error svp_acl_mdl_get_config_opt(svp\_acl\_mdl\_config\_handle *handle,  
svp\_acl\_mdl\_config\_attr attr, const void *attr_value, size_t value_size)
```

参数说明：

参数名	输入/输出	说明
handle	输入	指定模型加载的配置对象。需提前调用 svp_acl_mdl_create_config_handle 接口创建该对象。
attr	输入	指定需获取的属性。
attr_value	输出	指向属性值的指针，attr对应的属性取值。
value_size	输入	attr_value部分的数据类型长度。 用户需要根据属性的类型设置合理的长度。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.12 svp_acl_mdl_load_with_config

函数功能：

指定模型加载时需要的配置信息，并进行模型加载。



约束说明：

1. 调用[svp_acl_mdl_create_config_handle](#)接口创建模型加载的配置对象。
2. 多次调用[svp_acl_mdl_set_config_opt](#)接口设置配置对象中每个属性的值。
3. 调用[svp_acl_mdl_load_with_config](#)接口指定模型加载时需要的配置信息，并进行模型加载。
4. 模型加载成功后，调用[svp_acl_mdl_destroy_config_handle](#)接口销毁。如果后续需要通过接口[svp_acl_mdl_set_model_config](#)动态设置模型相关配置，可以不销毁handle。

函数原型：

```
svp_acl_error svp_acl_mdl_load_with_config(const svp\_acl\_mdl\_config\_handle *handle,  
uint32_t *model_id)
```

参数说明：

参数名	输入/输出	说明
handle	输入	指定模型加载的配置对象。需提前调用 svp_acl_mdl_create_config_handle 接口创建该对象，与 svp_acl_mdl_set_config_opt 中的handle保持一致。
model_id	输出	系统完成模型加载后生成的模型ID。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.13 svp_acl_mdl_set_model_config

函数功能：

动态设置模型的配置信息，包括优先级值、是否开启优先级抢占、优先级提升超时值。

约束说明：

1. 调用[svp_acl_mdl_create_config_handle](#)接口创建模型关联的配置对象。
2. 多次调用[svp_acl_mdl_set_config_opt](#)接口设置配置对象中每个属性的值；只有SVP_ACL_MDL_PRIORITY_INT32 / SVP_ACL_MDL_PRIORITY_PREEMP_EN_BOOL / SVP_ACL_MDL_PRIORITY_UP_STEP_TIMEOUT_UINT32 / SVP_ACL_MDL_PRIORITY_UP_TOP_TIMEOUT_UINT32的配置可动态设置，其它配置动态设置无效。
3. 调用[svp_acl_mdl_set_model_config](#)接口设置模型配置成功后，可以调用[svp_acl_mdl_destroy_config_handle](#)接口销毁配置对象。

函数原型：

```
svp\_acl\_error svp_acl_mdl_set_model_config(uint32_t model_id, const  
svp\_acl\_mdl\_config\_handle *handle)
```

参数说明：



参数名	输入/输出	说明
model_id	输入	系统完成模型加载后生成的模型ID。
handle	输入	指定模型运行时的配置对象。需提前调用 svp_acl_md1_create_config_handle 接口创建该对象，与 svp_acl_md1_set_config_opt 中的handle保持一致。

返回值说明：返回0表示成功，返回其它值表示失败。

注：仿真不支持该接口

9.8.14 svp_acl_md1_set_input_aipp

函数功能：

动态AIPP场景下，根据指定的动态AIPP输入的输入index，设置模型推理时的AIPP参数值，同步接口。

动态AIPP支持的几种操作的计算方式及其计算顺序会根据缩放模式的不同而有所差异，顺序如下：

- 最近邻插值模式缩放：抠图->padding->缩放->色域转换->减均值/归一化；
- 双线性插值模式缩放：抠图->padding->色域转换->减均值/归一化->缩放。

约束说明：

- 模型中支持动态AIPP的输入必须配置动态AIPP参数。
- 经过动态AIPP处理后的图像的宽、高必须与原始模型中输入Shape中的宽、高保持一致。
- 支持动态AIPP的输入在[svp_acl_md1_set_input_aipp](#)接口调用前，通过接口[svp_acl_md1_get_input_size_by_index](#)接口和[svp_acl_md1_get_input_default_stride](#)获取的大小和跨度是按动态AIPP模型提供的最大值计算的，[svp_acl_md1_set_input_aipp](#)接口调用成功后，通过接口获取的大小和跨度是通过动态AIPP配置值计算出的。
- 多Batch场景下，每个Batch的动态AIPP配置是一样的。计算输出图片宽、高的计算公式如[表9-1](#)。
- 抠图或者缩放或者padding之后，对图片宽、高的校验规则如下，其中，aipp_output_width、aipp_output_height分别表示AIPP输出图片的宽、高，其它参数是[svp_acl_md1_set_aipp_src_image_size](#)、[svp_acl_md1_set_aipp_scf_params](#)、[svp_acl_md1_set_aipp_crop_params](#)、[svp_acl_md1_set_aipp_padding_params](#)接口的入参。



表 9-1 输出图片宽、高计算公式

抠图	补边 (padding)	缩放	动态AIPP输出图片的宽、高
否	否	否	aipp_output_width=src_image_size_width, aipp_output_height=src_image_size_height
是	否	否	aipp_output_width=crop_size_width, aipp_output_height=crop_size_height
是	否	是	aipp_output_width=scf_output_size_width, aipp_output_height=scf_output_size_height
是	是	否	aipp_output_width=crop_size_width + padding_size_left + padding_size_right, aipp_output_height=crop_size_height + padding_size_top + padding_size_bottom
否	是	否	aipp_output_width=src_image_size_width + padding_size_left + padding_size_right, aipp_output_height=src_image_size_height + padding_size_top + padding_size_bottom
否	是	是	aipp_output_width=scf_output_size_width, aipp_output_height=scf_output_size_height
否	否	是	aipp_output_width=scf_output_size_width, aipp_output_height=scf_output_size_height
是	是	是	aipp_output_width=scf_output_size_width, aipp_output_height=scf_output_size_height

函数原型:

svp_acl_error svp_acl_md5_set_input_aipp(uint32_t model_id, **svp_acl_md5_dataset** *dataset, size_t index, const **svp_acl_md5_aipp** *aipp_set)

参数说明:

参数名	输入/输出	说明
model_id	输入	系统完成模型加载后生成的模型ID。
dataset	输入	表示模型的输入数据。 使用 svp_acl_md5_dataset 类型的数据描述模型推理时的输入数据，输入的内存地址、内存大小用 svp_acl_data_buffer 类型的数据来描述。



参数名	输入/输出	说明
index	输入	标识动态AIPP输入的输入index。 多个动态AIPP输入的场景下，用户可调用 svp_acl_md1_get_input_aipp_type 接口获取指定模型输入所关联的动态AIPP输入的输入index。 为保证向前兼容，如果明确只有一个动态AIPP输入，可调用 svp_acl_md1_get_input_index_by_name 接口获取，输入名称固定为SVP_ACL_DYNAMIC_AIPP_NAME。
aipp_set	输入	动态AIPP参数对象。 提前调用 svp_acl_md1_create_aipp 接口创建 svp_acl_md1_aipp 类型的数据。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.15 svp_acl_md1_set_aipp_by_input_index

函数功能：

动态AIPP场景下，根据指定的模型输入的输入index，设置模型推理时的AIPP参数值，同步接口。

动态AIPP支持的几种操作的计算方式及其计算顺序会根据缩放模式的不同而有所差异，顺序如下：

- 最近邻插值模式缩放：抠图->padding->缩放->色域转换->减均值/归一化；
- 双线性插值模式缩放：抠图->padding->色域转换->减均值/归一化->缩放。

约束说明：

svp_acl_md1_set_aipp_by_input_index接口约束说明请参考[svp_acl_md1_set_input_aipp](#)接口。

函数原型：

[svp_acl_error](#) svp_acl_md1_set_aipp_by_input_index(uint32_t model_id, [svp_acl_md1_dataset](#) *dataset, size_t index, const [svp_acl_md1_aipp](#) *aipp_set)

参数说明：

参数名	输入/输出	说明
modelId	输入	系统完成模型加载后生成的模型ID。
dataset	输入	表示模型的输入数据。 使用 svp_acl_md1_dataset 类型的数据描述模型推理时的输入数据，输入的内存地址、内存大小用 svp_acl_data_buffer 类型的数据来描述。



参数名	输入/输出	说明
index	输入	表示在第几个输入上设置动态AIPP参数。如果该输入没有关联动态AIPP输入，则返回报错。 可调用 svp_acl_md1_get_input_aipp_type 查询指定模型的指定输入是否有关联的动态AIPP输入，若有，则本接口的index参数值与 svp_acl_md1_get_input_aipp_type 接口的index参数值保持一致；若无，则无需设置动态AIPP参数。
aipp_set	输入	动态AIPP参数对象。 提前调用 svp_acl_md1_create_aipp 接口创建 svp_acl_md1_aipp 类型的数据。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.16 svp_acl_md1_set_output_aipp

函数功能：

动态AIPP场景下，根据指定的动态AIPP输出的输出index，设置模型推理时的AIPP参数值，同步接口。

约束说明：

- 模型中支持动态AIPP的输出必须配置动态AIPP参数。
- 输出的动态AIPP只支持抠图，抠图时源图像的宽、高必须与原始模型中输出Shape中的宽、高保持一致。
- 支持动态AIPP的输出在svp_acl_md1_set_output_aipp接口调用前，通过接口[svp_acl_md1_get_output_size_by_index](#)接口和[svp_acl_md1_get_output_default_stride](#)获取的大小和跨度是按动态AIPP模型提供的最大值计算的，svp_acl_md1_set_output_aipp接口调用成功后，通过接口获取的大小和跨度是通过动态AIPP配置值计算出的。
- 多Batch场景下，每个Batch的动态AIPP配置是一样的。开启了抠图后的输出宽高就是[svp_acl_md1_set_aipp_crop_params](#)设置的输出宽高。

函数原型：

```
svp_acl_error svp_acl_md1_set_output_aipp(uint32_t model_id, svp_acl_md1_dataset *dataset, size_t index, const svp_acl_md1_aipp *aipp_set)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	系统完成模型加载后生成的模型ID。



参数名	输入/输出	说明
dataset	输入	表示模型的输入数据。 使用 svp_acl_mdl_dataset 类型的数据描述模型推理时的输入数据，输入的内存地址、内存大小用 svp_acl_data_buffer 类型的数据来描述。
index	输入	标识动态AIPP输出的输出index。 多个动态AIPP输入的场景下，用户可调用 svp_acl_mdl_get_output_aipp_type 接口获取指定模型输出所关联的动态AIPP输出的输出index。为保证向前兼容，如果明确只有一个动态AIPP输出，可调用 svp_acl_mdl_get_output_index_by_name 接口获取，输入名称固定为SVP_ACL_DYNAMIC_AIPP_NAME。
aipp_set	输入	动态AIPP参数对象。 提前调用 svp_acl_mdl_create_aipp 接口创建 svp_acl_mdl_aipp 类型的数据。

返回值说明：返回0表示成功，返回其它值表示失败。

注意：Hi3516CV610不支持output的aipp功能。

9.8.17 svp_acl_mdl_get_first_aipp_info

函数功能：

获取模型输入AIPP的配置信息。

约束说明：

使用本接口获取模型中动态AIPP的信息，只能获取[svp_acl_aipp_info](#) 结构体中如下参数值：src_format、src_data_type、src_dim_num、shape_count、out_dims，其他的值无法获取。

函数原型：

svp_acl_error svp_acl_mdl_get_first_aipp_info(uint32_t model_id, size_t index, [svp_acl_aipp_info](#) *aipp_info)

参数说明：

参数名	输入/输出	说明
model_id	输入	系统完成模型加载后生成的模型ID。
index	输入	模型中输入的index。不支持获取task_buf与work_buf的aipp配置信息。
aipp_info	输出	获取指定输入上AIPP的配置信息。



返回值说明：返回0表示成功，返回其它值表示失败。

9.8.18 svp_acl_md1_get_last_aipp_info

函数功能：

获取模型输出AIPP的配置信息。

约束说明：

使用本接口获取模型中动态AIPP的信息，只能获取**svp_acl_aipp_info** 结构体中如下参数值：src_format、src_data_type、src_dim_num、shape_count、out_dims，其他的值无法获取。

函数原型：

```
svp_acl_error svp_acl_md1_get_last_aipp_info(uint32_t model_id, size_t index,  
svp_acl_aipp_info *aipp_info)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	系统完成模型加载后生成的模型ID。
index	输入	模型中输出的index。
aipp_info	输出	获取指定输出上AIPP的配置信息。

返回值说明：返回0表示成功，返回其它值表示失败。

注意：Hi3516CV610不支持output的aipp功能。

9.8.19 svp_acl_md1_get_input_aipp_type

函数功能：

获取指定模型的指定输入所支持的AIPP类型（动态AIPP或静态AIPP）及动态AIPP输入对应的index值。

函数原型：

```
svp_acl_error svp_acl_md1_get_input_aipp_type(uint32_t model_id, size_t index,  
svp_acl_md1_aipp_type *type, size_t *attached_index)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	系统完成模型加载后生成的模型ID。
index	输入	模型中输入的index。
type	输出	表示指定模型输入的AIPP类型。



参数名	输入/输出	说明
attached_index	输出	当type不为SVP_ACL_DATA_WITH_DYNAMIC_AIPP时，该值返回0xFFFFFFFF，表示无效。 当type为SVP_ACL_DATA_WITH_DYNAMIC_AIPP时，该值返回动态AIPP输入（用于配置动态AIPP参数）的index。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.20 svp_acl_mdl_get_output_aipp_type

函数功能：

获取指定模型的指定输出所支持的AIPP类型（动态AIPP或静态AIPP）及动态AIPP输出对应的index值。

函数原型：

```
svp_acl_error svp_acl_mdl_get_output_aipp_type(uint32_t model_id, size_t index,  
svp_acl_mdl_aipp_type *type, size_t *attached_index)
```

参数说明：

参数名	输入/输出	说明
modelId	输入	系统完成模型加载后生成的模型ID。
index	输入	模型中输出的index。
type	输出	表示指定模型输出的AIPP类型。
attached_index	输出	当type不为SVP_ACL_DATA_WITH_DYNAMIC_AIPP时，该值返回0xFFFFFFFF，表示无效。 当type为SVP_ACL_DATA_WITH_DYNAMIC_AIPP时，该值返回动态AIPP输出（用于配置动态AIPP参数）的index。

返回值说明：返回0表示成功，返回其它值表示失败。

注意：Hi3516CV610不支持output的aipp功能。

9.8.21 svp_acl_mdl_get_dynamic_batch

函数功能：

根据模型描述信息获取模型生成时配置的Batch信息。同步接口。

函数原型：



```
svp_acl_error svp_acl_mdl_get_dynamic_batch(const svp_acl_mdl_desc *model_desc,
svp_acl_mdl_batch *batch);
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。
batch	输出	<pre>#define SVP_ACL_MAX_BATCH_NUM 1 typedef struct svp_acl_mdl_batch { size_t batch_count; /* batch array count, only support 1 */ uint64_t batch[SVP_ACL_MAX_BATCH_NUM]; /* batch data array */ } svp_acl_mdl_batch;</pre> 当前只支持获取生成模型时配置的Batch信息。Batch值约束请查看《ATC工具使用指南》中batch_num参数说明。

返回值说明：返回0表示成功，返回其它值表示失败。

9.8.22 svp_acl_mdl_set_dynamic_hw_size

函数功能：

动态分辨率场景下，在模型执行前调用本接口设置模型推理时输入图片的高和宽。

函数原型：

```
svp_acl_error svp_acl_mdl_set_dynamic_hw_size(uint32_t model_id, svp_acl_mdl_dataset
*dataset, size_t index, uint64_t height, uint64_t width);
```

参数说明：

参数名	输入/输出	说明
model_id	输入	系统完成模型加载后生成的模型ID。
dataset	输入&输出	表示模型的输入数据。 使用svp_acl_mdl_dataset类型的数据描述模型推理时的输入数据，输入的内存地址、内存大小用 svp_acl_data_buffer 类型的数据来描述。
index	输入	标识动态HW输入的输入index，需调用 svp_acl_mdl_get_input_index_by_name 接口获取，输入名称固定为SVP_ACL_DYNAMIC_TENSOR_NAME。



参数名	输入/输出	说明
height	输入	需设置的H值。 此处设置的分辨率只能是模型构建时设置的分辨率档位中的某一档，模型构建的详细说明请参见《ATC工具使用指南》--dynamic_image_size参数。 可以调用 svp_acl_md5_get_dynamic_hw 接口获取指定模型支持的分辨率档位数以及每一档中的宽、高。
width	输入	需设置的W值。 此处设置的分辨率只能是模型构建时设置的分辨率档位中的某一档，模型构建的详细说明请参见《ATC工具使用指南》--dynamic_image_size参数。 可以调用 svp_acl_md5_get_dynamic_hw 接口获取指定模型支持的分辨率档位数以及每一档中的宽、高。

返回值说明：返回0表示成功，返回其它值表示失败。

9.9 Profiling 配置

9.9.1 功能及约束说明

功能：

该章节下的接口用于Profiling采集性能数据，实现方式支持以下两种：

- 方式一：将采集到的Profiling数据写入文件，再使用Profiling工具解析该文件，并展示性能分析数据

包括以下两种接口调用方式：

- [svp_acl_prof_init](#)接口、[svp_acl_prof_start](#)接口、[svp_acl_prof_stop](#)接口、[svp_acl_prof_finalize](#)接口配合使用，实现该方式的性能数据采集。该方式可获取SVP ACL的接口性能数据、*模式识别*Core上算子的执行时间、*模式识别*CPU算子的执行时间等。目前这些接口为进程级控制，表示在进程内任意线程调用该接口，其它线程都会生效。

一个进程内，可以根据需求多次调用这些接口，基于不同的Profiling采集配置，采集数据。

- 调用[svp_acl_init](#)接口，在SVP ACL初始化阶段，通过*.json文件传入要采集的Profiling数据。该方式可获取SVP ACL的接口性能数据、*模式识别*Core上算子的执行时间、*模式识别*CPU算子的执行时间等。

一个进程内，只能调用一次[svp_acl_init](#)接口，如果要修改Profiling采集配置，需修改*.json文件中的配置。详细使用说明请参见接口处的说明，不在本章节描述。

- 方式二：将采集到的Profiling数据解析后写入管道，由用户读入内存，再由用户调用SVP ACL的接口获取性能数据

[svp_acl_prof_model_subscribe](#)接口、[svp_acl_prof_get*](#)接口、[svp_acl_prof_model_unsubscribe](#)接口配合使用，实现该方式的性能数据采集，



当前支持获取网络模型中模式识别Core算子的性能数据，包括算子名称、算子类型名称、算子执行时间等。

总体约束：两种方式的Profiling性能数据采集接口不能交叉调用，当前只能在单进程中使用profiling功能。

- `sva_acl_prof_init`接口和`sva_acl_prof_finalize`接口之间不能调用`sva_acl_prof_model_subscribe`接口、`sva_acl_prof_get*`接口、`sva_acl_prof_model_unsubscribe`接口；
- `sva_acl_prof_model_subscribe`接口和`sva_acl_prof_model_unsubscribe`接口之间不能调用`sva_acl_prof_init`接口、`sva_acl_prof_start`接口、`sva_acl_prof_stop`接口、`sva_acl_prof_finalize`接口。
- 通过方式二实现的Profiling采集不支持Recurrent、RPN、ROI、CPU的网络，这些网络建议通过MindStudio工具采集并解析Profiling数据。
- 开启profiling进行数据采集时，由于第一帧推理需要配置profiling的使能/内存，并且第一帧推理时硬件会额外处理页表等动作，所以第一帧推理采集的数据会偏高。

方式一的接口约束说明

- 调用接口要求：
 - `sva_acl_prof_init`接口必须在`sva_acl_init`接口之后、模型加载之前调用。如果已经通过`sva_acl_init`接口配置了Profiling信息，则调用`sva_acl_prof_init`接口、`sva_acl_prof_start`接口、`sva_acl_prof_stop`接口、`sva_acl_prof_finalize`时，会返回报错。如果没有调用`sva_acl_prof_init`接口，调用`sva_acl_prof_start`接口、`sva_acl_prof_stop`接口、`sva_acl_prof_finalize`时，会返回报错。
 - `sva_acl_prof_start`接口在模型执行之前调用，若在模型执行过程中调用`sva_acl_prof_start`接口，Profiling采集到的数据为调用`sva_acl_prof_start`接口之后的数据，可能导致数据不完整。调用`sva_acl_prof_start`接口时，可以指定从一个Device上采集性能数据，也可以指定从多个Device上采集性能数据。一个用户APP进程内，如果连续调用多次`sva_acl_prof_start`接口，指定重复的Profiling配置，或指定的Device重复，会返回报错。
 - 在用户APP的进程生命周期内，`sva_acl_prof_init`接口与`sva_acl_prof_finalize`接口配对使用，建议只调用一次，如该组合多次调用可以改变保存性能数据的文件的路径。
 - `sva_acl_prof_start`接口与`sva_acl_prof_stop`接口需配对使用。
- 接口调用顺序：
 - 建议的接口调用顺序如下，以“一个用户APP进程内采集多个模型推理时的性能数据”为例：
`sva_acl_init`接口-->`sva_acl_prof_init`接口-->`sva_acl_prof_start`接口-->模型1加载-->模型1执行-->模型2加载-->模型2执行-->模型卸载-->`sva_acl_prof_stop`接口(与`sva_acl_prof_start`接口的`sva_acl_prof_config`数据保持一致)-->`sva_acl_prof_finalize`接口-->执行其它任务-->`sva_acl_finalize`接口
 - 错误的接口调用顺序示例如下，以“一个用户APP进程内，如果连续调用多次`sva_acl_prof_start`接口，指定的Device重复”为例：
`sva_acl_init`接口-->`sva_acl_prof_init`接口-->`sva_acl_prof_start`接口(指定Device 0)-->`sva_acl_prof_start`接口(指定Device 0)-->模型1加载-->模型1



执行-->模型2加载-->模型2执行-->svp_acl_prof_stop接口--
>svp_acl_prof_stop接口-->svp_acl_prof_finalize-->执行其它任务-->模型
卸载-->svp_acl_finalize接口

方式二的接口约束说明

- 接口调用要求：
 - **svp_acl_prof_model_subscribe**接口在模型执行之前调用，若在模型执行过程中调用**svp_acl_prof_model_subscribe**接口，Profiling采集到的数据为调用**svp_acl_prof_model_subscribe**接口之后的数据，可能导致数据不完整。
 - **svp_acl_prof_model_subscribe**接口需与**svp_acl_prof_model_unsubscribe**接口配对使用，不能在调用接口前，多次调用**svp_acl_prof_model_subscribe**接口重复订阅相同的模型。
 - 不能调用**svp_acl_prof_model_subscribe**接口订阅不存在的模型ID。
 - 不能调用**svp_acl_prof_model_unsubscribe**接口取消订阅不存在的模型ID或未订阅过的模型ID。
 - 如果在同一个Device上加载了多个模型，只能对多个模型下发同样的订阅配置。
- 接口调用顺序：
 - 建议的接口调用顺序如下：
模型加载-->**svp_acl_prof_model_subscribe**接口--
>**svp_acl_prof_get_op_desc_size**接口-->**svp_acl_prof_get_op_num**接口--
>**svp_acl_prof_get_op_type/svp_acl_prof_get_op_name/
svp_acl_prof_get_op_start/svp_acl_prof_get_op_end/
svp_acl_prof_get_op_duration/svp_acl_prof_get_model_id**接口--
>**svp_acl_prof_model_unsubscribe**接口
 - 错误的接口调用顺序示例如下，以重复定义同一个模型为例：
模型1加载-->**svp_acl_prof_model_subscribe**接口(指定模型1)--
>**svp_acl_prof_model_subscribe**接口(指定模型1)--
>**svp_acl_prof_model_unsubscribe**接口

📖 说明

Hi3516CV610平台在模型使能打断功能时（preempted_en），如果模型执行过程中被其他模型打断，会造成采集数据无效。建议在使用profiling采集性能时，不能同时运行多个可相互打断的模型。

9.9.2 svp_acl_prof_init

函数功能：初始化Profiling，目前用于设置保存性能数据的文件的路径。同步接口。

函数原型：

```
svp_acl_error svp_acl_prof_init(const char *result_path, size_t length)
```

参数说明：

参数名	输入/输出	说明
result_path	输入	指定保存性能数据的文件的路径，此路径为绝对路径。



参数名	输入/输出	说明
length	输入	result_path的长度，单位为Byte，最大长度不超过4096字节。

返回值说明：返回0表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.3 svp_acl_prof_start

函数功能：

- 下发Profiling请求，使能对应数据的采集。同步接口。
- 用户可根据需要，在模型执行过程中按需调用svp_acl_prof_start接口，Profiling采集到的数据为调用该接口之后的数据，且只会获取一次模型Profiling数据。

函数原型：

svp_acl_errors svp_acl_prof_start (const **svp_acl_prof_config** *profiler_config)

参数说明：

参数名	输入/输出	说明
profiler_config	输入	指定Profiling配置数据。 需提前调用 svp_acl_prof_create_config 接口创建svp_acl_prof_config类型的数据。

返回值说明：返回0表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.4 svp_acl_prof_stop

函数功能：停止Profiling数据采集，同步接口。

函数原型：

svp_acl_errors svp_acl_prof_stop(const **svp_acl_prof_config** *profiler_config)

参数说明：

参数名	输入/输出	说明
profiler_config	输入	指定停止Profiling数据采集的配置。 与 svp_acl_prof_start 接口中的 svp_acl_prof_config 类型数据保持一致。



返回值说明：返回0表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.5 svp_acl_prof_finalize

函数功能：结束Profiling。同步接口。

函数原型：

```
svp_acl_error svp_acl_prof_finalize()
```

参数说明：无

返回值说明：返回0表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.6 svp_acl_prof_model_subscribe

函数功能：网络场景下，订阅算子的基本信息，包括算子名称、算子类型、算子执行耗时等。同步接口。需要与[svp_acl_prof_model_unsubscribe](#)接口配对使用。

函数原型：

```
svp_acl_error svp_acl_prof_model_subscribe(uint32_t model_id, const  
svp_acl_prof_subscribe_config *prof_subscribe_config)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	待订阅的网络模型的ID。 调用 svp_acl_mdload_from_mem 接口加载模型成功后，会返回模型ID。
prof_subscribe_config	输入	待订阅的配置信息。 需提前调用 svp_acl_prof_create_subscribe_config 接口创建 svp_acl_prof_subscribe_config 类型的数据。

返回值说明：返回0表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.7 svp_acl_prof_model_unsubscribe

函数功能：网络场景下，取消订阅算子的基本信息，包括算子名称、算子类型、算子执行耗时等。同步接口。需要与[svp_acl_prof_model_subscribe](#)接口配对使用。

函数原型：

```
svp_acl_error svp_acl_prof_model_unsubscribe(uint32_t model_id)
```

参数说明：



参数名	输入/输出	说明
model_id	输入	已订阅的模型的ID。

返回值说明：返回0表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.8 svp_acl_prof_get_op_desc_size

函数功能：

- 获取单个算子数据结构的大小，单位为Byte。当前版本中约定每个算子数据结构的大小是一样的。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
svp_acl_error svp_acl_prof_get_op_desc_size(size_t *op_desc_size)
```

参数说明：

参数名	输入/输出	说明
op_desc_size	输出	算子数据结构的大小。

返回值说明：返回0表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.9 svp_acl_prof_get_op_num

函数功能：

- 获取指定内存中算子的数量。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
svp_acl_error svp_acl_prof_get_op_num(const void *op_info, size_t op_info_len, uint32_t *op_number)
```

参数说明：



参数名	输入/输出	说明
op_info	输入	指定算子信息的内存地址。 调用 svp_acl_prof_get_op_desc_size 接口获取到单个算子数据结构的大小后，用户需按照“单个算子数据结构的大小*整数系数”得到的数值申请内存，用于存放Profiling采集到的算子信息数据，作为本接口的输入。
op_info_len	输入	算子信息的长度。
op_number	输出	算子的数量。

返回值说明：返回0表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.10 svp_acl_prof_get_op_type_len

函数功能：

- 获取算子类型的字符串长度，用于内存申请。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
svp_acl_error_t svp_acl_prof_get_op_type_len(const void *op_info, size_t op_info_len, uint32_t index, size_t *op_type_len)
```

参数说明：

参数名	输入/输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子的算子类型名称。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个index的取值范围：[0, (算子数量-1)]
op_type_len	输出	op_type的长度。

返回值说明：返回0表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.11 svp_acl_prof_get_op_type

函数功能：



- 获取指定算子的算子类型名称，同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
svp_acl_error_t svp_acl_prof_get_op_type(const void *op_info, size_t op_info_len, uint32_t index, char *op_type, size_t op_type_len)
```

参数说明：

参数名	输入/输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子的算子类型名称。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个index的取值范围：[0, (算子数量-1)]
op_type	输出	算子类型名称。
op_type_len	输入	op_type的实际内存申请长度。取值范围建议不小于 svp_acl_prof_get_op_type_len ，否则内容会有截断。

返回值说明：返回0表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.12 svp_acl_prof_get_op_name_len

函数功能：

- 获取算子名称的字符串长度，用于内存申请。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
svp_acl_error_t svp_acl_prof_get_op_name_len(const void *op_info, size_t op_info_len, uint32_t index, size_t *op_name_len)
```

参数说明：

参数名	输入/输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。



参数名	输入/输出	说明
index	输入	指定获取第几个算子的算子名称长度。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个index的取值范围：[0, (算子数量-1)]
op_name_len	输出	op_name的实际内存申请长度。

返回值说明：返回0表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.13 svp_acl_prof_get_op_name

函数功能：

- 获取指定算子的算子名称。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

svp_acl_error svp_acl_prof_get_op_name(const void *op_info, size_t op_info_len, uint32_t index, char *op_name, size_t op_name_len)

参数说明：

参数名	输入/输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子的算子名称。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个index的取值范围：[0, (算子数量-1)]
op_name	输出	算子名称。
op_name_len	输入	op_name的实际内存申请长度。取值范围建议不小于svp_acl_prof_get_op_name_len。

返回值说明：返回0表示成功，返回其它值表示失败。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.14 svp_acl_prof_get_op_start

函数功能：



- 获取算子执行的开始时间，单位为ns。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
uint64_t svp_acl_prof_get_op_start(const void *op_info, size_t op_info_len, uint32_t index)
```

参数说明：

参数名	输入/输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子执行的开始时间。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个index的取值范围：[0, (算子数量-1)]

返回值说明：算子执行的开始时间。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.15 svp_acl_prof_get_op_end

函数功能：

- 获取算子执行的结束时间，单位为ns。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
uint64_t svp_acl_prof_get_op_end(const void *op_info, size_t op_info_len, uint32_t index)
```

参数说明：

参数名	输入/输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子执行的结束时间。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个index的取值范围：[0, (算子数量-1)]

返回值说明：算子执行结束时间。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。



9.9.16 svp_acl_prof_get_op_duration

函数功能：

- 获取算子执行的耗时时间，单位为ns。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
uint64_t svp_acl_prof_get_op_duration(const void *op_info, size_t op_info_len, uint32_t index)
```

参数说明：

参数名	输入/输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子执行的耗时时间。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个index的取值范围：[0, (算子数量-1)]

返回值说明：算子执行的耗时时间。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.9.17 svp_acl_prof_get_model_id

函数功能：

- 获取指定算子所在模型的ID。同步接口。
- 建议用户新建一个线程，在新线程内调用该接口，否则可能阻塞主线程中的其它任务调度。

函数原型：

```
size_t svp_acl_prof_get_model_id(const void *op_info, size_t op_info_len, uint32_t index)
```

参数说明：

参数名	输入/输出	说明
op_info	输入	包含算子信息的地址。
op_info_len	输入	算子信息的长度。
index	输入	指定获取第几个算子所在模型的ID。 用户调用 svp_acl_prof_get_op_num 接口获取算子数量后，这个index的取值范围：[0, (算子数量-1)]



返回值说明：模型的ID。

参考资源：接口调用示例，参见“[Profiling性能数据采集](#)”。

9.10 获取数据大小

9.10.1 svp_acl_data_type_size

函数功能：获取svp_acl_data_type数据的大小，单位bit，同步接口。

函数原型：

```
size_t svp_acl_data_type_size(svp_acl_data_type data_type)
```

参数说明：

参数名	输入/输出	说明
data_type	输入	指定要获取大小的svp_acl_data_type数据。

返回值说明：返回svp_acl_data_type数据的大小，单位bit。

9.11 SoC 扩展接口

9.11.1 svp_acl_ext_process_aicpu_task

函数功能：

- 获取模式识别CPU任务并执行。同步接口。
- 用户需新建一个线程，在线程函数内调用svp_acl_ext_process_aicpu_task接口。

约束说明：如果当前进程没有创建任何stream，会返回错误。

函数原型：

```
svp_acl_error svp_acl_ext_process_aicpu_task(int32_t timeout)
```

参数说明：

参数名	输入/输出	说明
timeout	输入	超时时间，单位为ms。 取值范围： • -1：表示无限等待 • 大于等于0：表示等待的时间

返回值说明：返回0表示成功，返回其它值表示失败。



9.11.2 svp_acl_ext_get_mdl_aicpu_task_num

函数功能：获取模型中模式识别CPU任务个数。同步接口。

函数原型：

```
svp_acl_error svp_acl_ext_get_mdl_aicpu_task_num(uint32_t model_id, uint32_t *num)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	指定需要执行推理的模型的ID。 调用 svp_acl_mdl_load_from_mem 接口加载模型成功后，会返回模型ID。
num	输出	获取对应模型所含有模式识别CPU任务个数。

返回值说明：返回0表示成功，返回其它值表示失败。

9.11.3 svp_acl_ext_get_mdl_net_type

函数功能：获取模型中的网络类型

函数原型：

```
svp_acl_error svp_acl_ext_get_mdl_net_type(uint32_t model_id, uint32_t *net_type)
```

参数说明：

参数名	输入/输出	说明
model_id	输入	指定需要执行推理的模型的ID。 调用 svp_acl_mdl_load_from_mem 接口加载模型成功后，会返回模型ID。
net_type	输出	获取对应模型中的网络类型，当前模型中有RECURRENT网络会返回SVP_ACL_NET_TYPE_RECURRENT类型，其余的模型返回SVP_ACL_NET_TYPE_CNN类型。 #define SVP_ACL_NET_TYPE_CNN 0x1 #define SVP_ACL_NET_TYPE_RECURRENT (0x1 << 1)

返回值说明：返回0表示成功，返回其它值表示失败。

9.12 数据类型及其操作接口

9.12.1 svp_acl_error

```
typedef int svp_acl_error;
```



表 9-2 返回码列表

返回码	含义	可能原因及解决方法
#define SVP_ACL_SUCCESS 0	执行成功	-
#define SVP_ACL_ERROR_NONE 0	执行成功。	-
#define SVP_ACL_ERROR_INVALID_PARAMETER 100000	参数校验失败。	请检查接口的入参值是否正确。
#define SVP_ACL_ERROR_UNINITIALIZED 100001	未初始化。	请检查是否已调用 svp_acl_init 接口进行初始化，请确保已调用接口，且在其它SVP ACL接口之前调用。
#define SVP_ACL_ERROR_REPEAT_INITIALIZE 100002	重复初始化或重复加载。	请检查是否调用对应的接口重复初始化或重复加载。
#define SVP_ACL_ERROR_INVALID_FILE 100003	无效的文件。	请检查文件是否存在、文件是否能被访问等。
#define SVP_ACL_ERROR_WRITE_FILE 100004	写文件失败。	请检查文件路径是否存在、文件是否有写权限等。
#define SVP_ACL_ERROR_INVALID_FILE_SIZE 100005	无效的文件大小。	请检查文件大小是否符合接口要求。
#define SVP_ACL_ERROR_PARSE_FILE 100006	解析文件失败。	请检查文件内容是否合法。
#define SVP_ACL_ERROR_FILE_MISSING_ATTR 100007	文件缺失参数。	请检查文件内容是否完整。
#define SVP_ACL_ERROR_FILE_ATTR_INVALID 100008	文件参数无效。	请检查文件中参数值是否正确。
#define SVP_ACL_ERROR_INVALID_DUMP_CONFIG 100009	无效的Dump配置。	请检查Dump配置是否正确，详细配置请参见《ATC工具使用指南》“online_model_type”章节。
#define SVP_ACL_ERROR_INVALID_PROFILING_CONFIG 100010	无效的Profiling配置。	请检查Profiling配置是否正确。



返回码	含义	可能原因及解决方法
#define SVP_ACL_ERROR_INVALID_MODEL_ID 100011	无效的模型ID。	请检查模型ID是否正确、模型是否正确加载。
#define SVP_ACL_ERROR_DESERIALIZE_MODEL 100012	反序列化模型失败。	模型可能与当前版本不匹配，请参见《ATC工具使用指南》重新转换模型。
#define SVP_ACL_ERROR_PARSE_MODEL 100013	解析模型失败。	模型可能与当前版本不匹配，请参见《ATC工具使用指南》重新转换模型。
#define SVP_ACL_ERROR_READ_MODEL_FAILURE 100014	读取模型失败。	请检查模型文件是否存在、模型文件是否能被访问等。
#define SVP_ACL_ERROR_MODEL_SIZE_INVALID 100015	无效的模型大小。	模型文件无效，请参见《ATC工具使用指南》重新转换模型。
#define SVP_ACL_ERROR_MODEL_MISSING_ATTR 100016	模型缺少参数。	模型可能与当前版本不匹配，请参见《ATC工具使用指南》重新转换模型。
#define SVP_ACL_ERROR_MODEL_INPUT_NOT_MATCH 100017	模型的输入不匹配。	请检查模型的输入是否正确。
#define SVP_ACL_ERROR_MODEL_OUTPUT_NOT_MATCH 100018	模型的输出不匹配。	请检查模型的输出是否正确。
#define SVP_ACL_ERROR_MODEL_NOT_DYNAMIC 100019	非动态模型。	请检查当前模型是否支持动态场景，如不支持，请参见《ATC工具使用指南》重新转换模型。
#define SVP_ACL_ERROR_OP_TYPE_NOT_MATCH 100020	单算子类型不匹配。	请检查算子类型是否正确。
#define SVP_ACL_ERROR_OP_INPUT_NOT_MATCH 100021	单算子的输入不匹配。	请检查算子的输入是否正确。
#define SVP_ACL_ERROR_OP_OUTPUT_NOT_MATCH 100022	单算子的输出不匹配。	请检查算子的输出是否正确。
#define SVP_ACL_ERROR_OP_ATTR_NOT_MATCH 100023	单算子的属性不匹配。	请检查算子的属性是否正确。



返回码	含义	可能原因及解决方法
#define SVP_ACL_ERROR_OP_NOT_FOUND 100024	单算子未找到。	请检查算子类型是否支持。
#define SVP_ACL_ERROR_OP_LOAD_FAILED 100025	单算子加载失败。	模型可能与当前版本不匹配，请参见《ATC工具使用指南》重新转换模型。
#define SVP_ACL_ERROR_UNSUPPORTED_DATA_TYPE 100026	不支持的数据类型。	请检查数据类型是否存在或当前是否支持。
#define SVP_ACL_ERROR_FORMAT_NOT_MATCH 100027	Format不匹配。	请检查Format是否正确。
#define SVP_ACL_ERROR_BIN_SELECTOR_NOT_REGISTERED 100028	使用二进制选择方式编译算子接口时，算子未注册选择器。	请检查是否注册算子选择器。
#define SVP_ACL_ERROR_KERNEL_NOT_FOUND 100029	编译算子时，算子Kernel未注册。	请检查是否注册算子Kernel。
#define SVP_ACL_ERROR_BIN_SELECTOR_ALREADY_REGISTERED 100030	使用二进制选择方式编译算子接口时，算子重复注册。	请检查是否重复注册算子选择器。
#define SVP_ACL_ERROR_KERNEL_ALREADY_REGISTERED 100031	编译算子时，算子Kernel重复注册。	请检查是否重复注册算子Kernel。
#define SVP_ACL_ERROR_INVALID_QUEUE_ID 100032	无效的队列ID。	请检查队列ID是否正确。
#define SVP_ACL_ERROR_REPEAT_SUBSCRIBE 100033	重复订阅。	请检查针对同一个Stream，是否重复调用 svp_acl_rt_unsubscribe_report 接口。
#define SVP_ACL_ERROR_STREAM_NOT_SUBSCRIBE 100034	Stream未订阅。	请检查是否已调用 svp_acl_rt_subscribe_report 接口。
#define SVP_ACL_ERROR_THREAD_NOT_SUBSCRIBE 100035	线程未订阅。	请检查是否已调用 svp_acl_rt_subscribe_report 接口。



返回码	含义	可能原因及解决方法
#define SVP_ACL_ERROR_WAIT_CALLBACK_TIMEOUT 100036	等待callback超时。	请检查是否已调用 svp_acl_rt_launch_callback 接口下发callback任务； 请检查 svp_acl_rt_process_report 接口中超时时间是否合理； 请检查callback任务是否已经处理完成，如果已处理完成，但还调用 svp_acl_rt_process_report 接口，则需优化代码逻辑。
#define SVP_ACL_ERROR_REPEAT_FINALIZE 100037	重复去初始化。	请检查是否重复调用 svp_acl_finalize 接口进行去初始化。
#define SVP_ACL_ERROR_NOT_STATIC_AIPP 100038	静态AIPP配置信息不存在。	请传入正确的index值。
#define SVP_ACL_ERROR_COMPILING_STUB_MODE 100039	运行应用前配置的动态库路径是编译桩的路径，不是正确的动态库路径。	请检查动态库路径的配置，确保使用运行模式的动态库。
#define SVP_ACL_ERROR_GROUP_NOT_SET 100040	未设置Group。	请检查是否已调用配置Group接口。
#define SVP_ACL_ERROR_GROUP_NOT_CREATE 100041	未创建对应的Group。	请检查调用接口时设置的Group ID是否在支持的范围内，Group ID的取值范围：[0, (Group数量-1)]
#define SVP_ACL_ERROR_PROF_ALREADY_RUN 100042	已存在采集Profiling数据的任务。	- 请检查在调用prof接口配置Profiling性能数据采集信息前，是否已调用svp_acl_init接口配置Profiling信息，如是，请调整代码逻辑，保留一种方式配置Profiling信息即可。 - 请检查是否对同一个Device重复下发了多次Profiling配置。



返回码	含义	可能原因及解决方法
#define SVP_ACL_ERROR_PROF_NOT_RUN 100043	未进行Profiling初始化。	请检查接口调用顺序。
#define SVP_ACL_ERROR_DUMP_ALREADY_RUN 100044	已存在获取Dump数据的任务。	请检查在调用Dump接口配置Dump信息前，是否已调用 svp_acl_init 接口配置Dump信息，如是，请调整代码逻辑，保留一种方式配置Dump信息即可。
#define SVP_ACL_ERROR_DUMP_NOT_RUN 100045	未先进行Dump初始化。	请检查获取Dump数据的接口调用顺序。
#define SVP_ACL_ERROR_PROF_REPEAT_SUBSCRIBE 148046	重复订阅同一个模型。	请检查接口调用顺序。
#define SVP_ACL_ERROR_PROF_API_CONFLICT 148047	采集性能数据的接口调用冲突。	两种方式的Profiling性能数据采集接口不能交叉调用。
#define SVP_ACL_ERROR_INVALID_MAX_OPQUEUE_NUM_CONFIG 148048	无效的算子缓存信息老化配置。	请检查算子缓存信息老化配置，参考 svp_acl_init 处的配置说明及示例。
#define SVP_ACL_ERROR_BAD_ALLOC 200000	申请资源失败。	请检查硬件环境上的内存剩余情况或者Stream、Context等资源剩余情况。
#define SVP_ACL_ERROR_API_NOT_SUPPORTED 200001	接口不支持。	请检查调用的接口当前是否支持。
#define SVP_ACL_ERROR_INVALID_DEVICE 200002	无效的Device。	请检查Device是否存在。
#define SVP_ACL_ERROR_MEMORY_ADDRESS_UNALIGNED 200003	内存地址未对齐。	请检查内存地址是否符合接口要求。
#define SVP_ACL_ERROR_RESOURCE_NOT_MATCH 200004	资源不匹配。	请检查调用接口时，是否传入正确的Stream、Context等资源。
#define SVP_ACL_ERROR_INVALID_RESOURCE_HANDLE 200005	无效的资源句柄。	请检查调用接口时，传入的Stream、Context等资源是否已被销毁或占用。



返回码	含义	可能原因及解决方法
#define SVP_ACL_ERROR_FEATURE_UN SUPPORTED 200006	特性不支持。	请根据日志报错排查问 题，或联系软件工程师。
#define SVP_ACL_ERROR_PROF_MOD ULES_UNSUPPORTED 200007	下发了不支持的 Profiling配置。	请检查Profiling的配置是 否正确。
#define SVP_ACL_ERROR_STORAGE_ OVER_LIMIT 300000	超出存储上限。	请检查硬件环境上的存储 剩余情况。
#define SVP_ACL_ERROR_INTERNAL_ ERROR 500000	未知内部错误。	请根据日志报错排查问 题，或联系软件工程师。
#define SVP_ACL_ERROR_FAILURE 500001	系统内部SVP ACL的 错误。	请根据日志报错排查问 题，或联系软件工程师。
#define SVP_ACL_ERROR_GE_FAILUR E 500002	系统内部GE的错误。	请根据日志报错排查问 题，或联系软件工程师。
#define SVP_ACL_ERROR_RT_FAILURE 500003	系统内部RUNTIME的 错误。	请根据日志报错排查问 题，或联系软件工程师。
#define SVP_ACL_ERROR_DRV_FAILU RE 500004	系统内部DRV (Driver) 的错误。	请根据日志报错排查问 题，或联系软件工程师。
#define SVP_ACL_ERROR_PROFILING _FAILURE 500005	Profiling相关错误。	请根据日志报错排查问 题，或联系软件工程师。
#define SVP_ACL_ERROR_RT_REPORT _TIMEOUT 507012	获取task超时。	请根据日志报错排查问 题，或联系软件工程师。

📖 说明

返回码定义规则：

- 规则1：开发人员的环境异常或者代码逻辑错误，可以通过优化环境或代码逻辑的方式解决问题，此时返回码定义为：1XXXXX。
- 规则2：资源不足（Stream、内存等）、开发人员编程时使用的接口或参数与当前硬件不匹配，可以通过在编程时合理使用资源的方式解决，此时返回码定义为：2XXXXX。
- 规则3：业务功能异常，比如队列满、队列空等，此时返回码定义为3XXXXX。
- 规则4：软硬件内部异常，包括软件内部错误、Device执行失败等，用户无法解决问题，需将问题反馈给软件工程师的，此时返回码定义为：5XXXXX。
- 规则5：无法识别的错误，当前都映射为500000。

9.12.2 svp_acl_data_type

```
typedef enum {  
    SVP_ACL_DT_UNDEFINED = -1, //未知数据类型，默认值。  
    SVP_ACL_FLOAT = 0,  
    SVP_ACL_FLOAT16 = 1,  
    SVP_ACL_INT8 = 2,  
    SVP_ACL_INT32 = 3,  
    SVP_ACL_UINT8 = 4,  
    SVP_ACL_INT16 = 6,  
    SVP_ACL_UINT16 = 7,  
    SVP_ACL_UINT32 = 8,  
    SVP_ACL_INT64 = 9,  
    SVP_ACL_UINT64 = 10,  
    SVP_ACL_DOUBLE = 11,  
    SVP_ACL_BOOL = 12,  
    SVP_ACL_STRING = 13,  
    SVP_ACL_INT10 = 100,  
    SVP_ACL_UINT10 = 101,  
    SVP_ACL_INT12 = 102,  
    SVP_ACL_UINT12 = 103,  
    SVP_ACL_INT14 = 104,  
    SVP_ACL_UINT14 = 105,  
    SVP_ACL_INT24 = 106,  
    SVP_ACL_UINT24 = 107,  
    SVP_ACL_UINT4 = 108,  
} svp_acl_data_type;
```

9.12.3 svp_acl_float16

该数据类型仅用于数据传递，不能用于加减乘除等运算。

```
typedef uint16_t svp_acl_float16;
```

9.12.4 svp_acl_format

```
typedef enum {  
    SVP_ACL_FORMAT_UNDEFINED = -1,  
    SVP_ACL_FORMAT_NCHW = 0,  
    SVP_ACL_FORMAT_NHWC = 1,  
    SVP_ACL_FORMAT_ND = 2,  
    SVP_ACL_FORMAT_NC1HWC0 = 3,  
    SVP_ACL_FORMAT_FRACTAL_Z = 4,  
    SVP_ACL_FORMAT_NC1HWC0_C04 = 12,  
    SVP_ACL_FORMAT_FRACTAL_NZ = 29,  
} svp_acl_format;
```

- UNDEFINED：未知格式，默认值。
- ND：表示支持任意格式，仅有Square、Tanh等这些单输入对自身处理的算子外，其它需要慎用。

- NCHW: NCHW格式。
- NHWC: NHWC格式。
- NC1HWC0: 自研的5维数据格式。其中, C0与微架构强相关, 该值等于cube单元的size, 例如16; C1是将C维度按照C0切分: $C1=C/C0$, 若结果不整除, 最后一份数据需要padding到C0。
- FRACTAL_Z: 卷积的权重的格式。
- NC1HWC0_C04: 自研的5维数据格式。其中, C0固定为4, C1是将C维度按照C0切分: $C1=C/C0$, 若结果不整除, 最后一份数据需要padding到C0。当前版本不支持。
- FRACTAL_NZ: 内部格式, 用户目前无需使用。

9.12.5 svp_acl_rt_context

```
typedef void *svp_acl_rt_context;
```

9.12.6 svp_acl_rt_stream

```
typedef void *svp_acl_rt_stream;
```

9.12.7 svp_acl_rt_event

```
typedef void *svp_acl_rt_event;
```

注意: Hi3516CV610不支持。

9.12.8 svp_acl_rt_event_status

```
typedef enum svp_acl_rt_event_status {  
    SVP_ACL_EVENT_STATUS_COMPLETE = 0, /* 完成 */  
    SVP_ACL_EVENT_STATUS_NOT_READY = 1, /* 未完成 */  
    SVP_ACL_EVENT_STATUS_RESERVED = 2, /* 预留 */  
} svp_acl_rt_event_status;
```

注意: Hi3516CV610不支持。

9.12.9 svp_acl_rt_event_wait_status

```
typedef enum svp_acl_rt_event_wait_status {  
    SVP_ACL_EVENT_WAIT_STATUS_COMPLETE = 0, /* 完成 */  
    SVP_ACL_EVENT_WAIT_STATUS_NOT_READY = 1, /* 未完成 */  
    SVP_ACL_EVENT_WAIT_STATUS_RESERVED = 0xFFFF, /* 预留 */  
} svp_acl_rt_event_wait_status;
```

注意: Hi3516CV610不支持。

9.12.10 svp_acl_rt_run_mode

```
typedef enum svp_acl_rt_run_mode {  
    SVP_ACL_DEVICE, /* 软件栈运行在Device的Control CPU或板端环境中 */  
    SVP_ACL_HOST, /* 软件栈运行在Host侧 */  
} svp_acl_rt_run_mode;
```

9.12.11 svp_acl_mdl_io_dims

```
#define SVP_ACL_MAX_DIM_CNT 128  
#define SVP_ACL_MAX_TENSOR_NAME_LEN 128  
typedef struct svp_acl_mdl_io_dims {  
    char name[SVP_ACL_MAX_TENSOR_NAME_LEN]; /* tensor name */  
}
```



```
size_t dim_count; /* Shape中的维度个数，如果为标量，则维度个数为0 */
int64_t dims[SVP_ACL_MAX_DIM_CNT]; /* 维度信息 */
} svp_acl_md_io_dims;
```

9.12.12 svp_acl_buffer_reuse_type

表示模型使用的WorkBuf与输入输出Buf的复用类型。

```
typedef enum {
    SVP_ACL_BUFFER_REUSE_UNDEFINED = -1, /* 无定义 */
    SVP_ACL_BUFFER_REUSE_NONE = 0,      /* 不支持复用 */
    SVP_ACL_BUFFER_REUSE_DISCONT = 1,   /* 支持复用，不需要输入/输出内存连续 */
    SVP_ACL_BUFFER_REUSE_IN_CONT = 2,    /* 支持复用，需要所有非扩展输入内存连续，输出不需要连续，输入内存需要按照index顺序排布 */
    SVP_ACL_BUFFER_REUSE_OUT_CONT = 3,    /* 支持复用，需要所有非扩展输出内存连续，输入不需要连续，输出内存需要按照index顺序排布 */
    SVP_ACL_BUFFER_REUSE_IN_OUT_CONT = 4, /* 支持复用，需要所有非扩展输入内存连续，需要所有非扩展输出内存连续，输入内存需要按照index顺序排布，输出内存按照index顺序排布 */
    SVP_ACL_BUFFER_REUSE_INOUT_CONT = 5, /* 支持复用，需要所有非扩展输入以及所有非扩展输出内存全部连续，输入内存需要按照index顺序排布，输出内存需要按照index顺序排布，并且需要按照输入/输出顺序排布 */
    SVP_ACL_BUFFER_REUSE_INOUTWORK_CONT = 6, /* 支持复用，需要所有非扩展输入、所有非扩展输出、workbuf的内存全部连续，输入内存需要按照index顺序排布，输出内存需要按照index顺序排布，并且需要按照输入/输出/workbuf顺序排布 */
    SVP_ACL_BUFFER_REUSE_OUTWORK_CONT = 7, /* 支持复用，需要所有非扩展输出、workbuf的内存全部连续，输出内存需要按照index顺序排布，并且需要按照输出/workbuf顺序排布 */
} svp_acl_buffer_reuse_type;
```

说明：非扩展输入输出表示该输入输出为源数据，不包括TaskBuf、WorkBuf以及动态AIPP扩展的输入输出。

9.12.13 svp_acl_inout_reuse_type

表示模型的输入输出内存与WorkBuf的复用类型。

```
typedef enum {
    SVP_ACL_INOUT_REUSE_UNDEFINED = -1, /* 无定义 */
    SVP_ACL_INOUT_REUSE_NONE = 0,      /* 当前输入/输出的内存不与Workbuf复用 */
    SVP_ACL_INOUT_REUSE_DISCONT = 1,   /* 当前输入/输出的内存与Workbuf复用，与其它输入/输出内存不需要连续 */
    SVP_ACL_INOUT_REUSE_CONT = 2,      /* 当前输入/输出的内存与Workbuf复用，与其它输入/输出内存需要连续 */
} svp_acl_inout_reuse_type;
```

9.12.14 svp_acl_prof_aicore_metrics

```
typedef enum {
    SVP_ACL_AICORE_ARITHMETIC_UTILIZATION = 0, /* 各种计算类性能数据统计，包括MAC、VEC等 */
    SVP_ACL_AICORE_NONE = 0xFF
} svp_acl_prof_aicore_metrics;
```

9.12.15 svp_acl_data_buffer

9.12.15.1 svp_acl_create_data_buffer

函数功能：创建svp_acl_data_buffer类型的数据，该数据类型用于描述内存地址、大小等内存信息。同步接口。

函数原型：

```
svp_acl_data_buffer *svp_acl_create_data_buffer(void *data, size_t size, size_t stride)
```

参数说明：



参数名	输入/输出	说明
data	输入	存放数据的内存地址。 该内存需由用户自行管理，调用 svp_acl_rt_malloc 接口/ svp_acl_rt_free 接口申请/释放内存，或调用 svp_acl_rt_malloc_host 接口/ svp_acl_rt_free_host 接口申请/释放内存。
size	输入	内存大小，单位Byte。 Hi3519DV500/Hi3516DV500 size大小：(0, 0xFFFFFFFF] Hi3516CV610 size大小：(0, 0x80000000] 可参照 svp_acl_mdl_get_input_size_by_index 或 svp_acl_mdl_get_output_size_by_index 中描述的计算方法计算获取。
stride	输入	跨度大小，用于逻辑快速跳转到下一行，要求16字节对齐，单位Byte。 stride大小：(0, 0xFFFFFFFF) 可参照 svp_acl_mdl_get_input_default_stride 或 svp_acl_mdl_get_output_default_stride 中的计算方法计算获得。

返回值说明：返回svp_acl_data_buffer类型的指针。

9.12.15.2 svp_acl_destroy_data_buffer

函数功能：

- 销毁svp_acl_data_buffer类型的数据。同步接口。
- 此处仅销毁svp_acl_data_buffer类型的数据，调用[svp_acl_create_data_buffer](#)接口创建svp_acl_data_buffer类型数据时传入的data的内存需由用户自行释放。

函数原型：

```
svp_acl_error svp_acl_destroy_data_buffer(const svp\_acl\_data\_buffer *data_buffer)
```

参数说明：

参数名	输入/输出	说明
data_buffer	输入	待销毁的svp_acl_data_buffer类型的指针。

返回值说明：返回0表示成功，返回非0表示失败。



9.12.15.3 svp_acl_get_data_buffer_addr

函数功能：获取svp_acl_data_buffer类型中的数据的内存地址。同步接口。

函数原型：

```
void *svp_acl_get_data_buffer_addr(const svp_acl_data_buffer *data_buffer)
```

参数说明：

参数名	输入/输出	说明
data_buffer	输入	svp_acl_data_buffer的地址。

返回值说明：返回svp_acl_data_buffer类型中的数据的内存地址。

9.12.15.4 svp_acl_get_data_buffer_size

函数功能：获取svp_acl_data_buffer类型中数据的内存大小，单位Byte。同步接口。

函数原型：

```
size_t svp_acl_get_data_buffer_size(const svp_acl_data_buffer *data_buffer)
```

参数说明：

参数名	输入/输出	说明
data_buffer	输入	svp_acl_data_buffer的地址。

返回值说明：svp_acl_data_buffer类型中数据的内存大小。

9.12.15.5 svp_acl_get_data_buffer_stride

函数功能：获取svp_acl_data_buffer类型中stride的大小，单位Byte。同步接口。

函数原型：

```
size_t svp_acl_get_data_buffer_stride(const svp_acl_data_buffer *data_buffer)
```

参数说明：

参数名	输入/输出	说明
data_buffer	输入	svp_acl_data_buffer的地址。

返回值说明：svp_acl_data_buffer类型中stride的大小。

9.12.15.6 svp_acl_update_data_buffer

函数功能：



- 更新svp_acl_data_buffer中数据的内存及大小。同步接口。
- 更新svp_acl_data_buffer后，之前svp_acl_data_buffer中存放数据的内存如果不使用，需及时释放，否则可能会导致内存泄漏。

函数原型：

```
svp_acl_error_t  svp_acl_update_data_buffer(svp_acl_data_buffer_t *data_buffer, void *data, size_t size, size_t stride)
```

参数说明：

参数名	输入/输出	说明
data_buffer	输入	svp_acl_data_buffer的地址。
data	输入	存放数据的内存地址。 该内存需由用户自行管理，调用 svp_acl_rt_malloc 接口/ svp_acl_rt_free 接口申请/释放内存，或调用 svp_acl_rt_malloc_host 接口/ svp_acl_rt_free_host 接口申请/释放内存。
size	输入	内存大小，单位Byte。 size大小：(0, 0xFFFFFFFF]。
stride	输入	跨度大小，用于逻辑快速跳转到下一行，需要16字节对齐，单位Byte。 stride大小：(0, 0xFFFFFFFF)。 可参照 svp_acl_md5_get_input_default_stride 或 svp_acl_md5_get_output_default_stride 参照中的计算方法计算获得。

返回值说明：返回0表示成功，返回非0表示失败。

9.12.16 svp_acl_md5_dataset

9.12.16.1 svp_acl_md5_create_dataset

函数功能：创建svp_acl_md5_dataset类型的数据，该数据类型用于描述模型推理时的输入数据、输出数据，模型可能存在多个输入、多个输出，每个输入/输出的内存地址、内存大小用[svp_acl_data_buffer](#)类型的数据来描述。同步接口。

函数原型：

```
svp_acl_md5_dataset_t *svp_acl_md5_create_dataset()
```

参数说明：无

返回值说明：返回svp_acl_md5_dataset类型的指针。



9.12.16.2 svp_acl_mdl_destroy_dataset

函数功能：销毁svp_acl_mdl_dataset类型的数据。同步接口。

函数原型：

```
svp_acl_error svp_acl_mdl_destroy_dataset(const svp_acl_mdl_dataset *dataset)
```

参数说明：

参数名	输入/输出	说明
dataset	输入	待销毁的svp_acl_mdl_dataset类型的指针。

返回值说明：返回0表示成功，返回非0表示失败。

9.12.16.3 svp_acl_mdl_add_dataset_buffer

函数功能：向svp_acl_mdl_dataset中增加svp_acl_data_buffer。同步接口。

约束说明：最多添加256个data_buffer。

函数原型：

```
svp_acl_error svp_acl_mdl_add_dataset_buffer(svp_acl_mdl_dataset *dataset,  
svp_acl_data_buffer *data_buffer)
```

参数说明：

参数名	输入/输出	说明
dataset	输出	待增加svp_acl_data_buffer的svp_acl_mdl_dataset地址。
data_buffer	输入	待增加的svp_acl_data_buffer地址

返回值说明：返回0表示成功，返回非0表示失败。

9.12.16.4 svp_acl_mdl_get_dataset_num_buffers

函数功能：获取svp_acl_mdl_dataset中svp_acl_data_buffer的个数，同步接口。

函数原型：

```
size_t svp_acl_mdl_get_dataset_num_buffers(const svp_acl_mdl_dataset *dataset)
```

参数说明：

参数名	输入/输出	说明
dataset	输入	svp_acl_mdl_dataset地址。



返回值说明：svp_acl_data_buffer的个数。

9.12.16.5 svp_acl_mdl_get_dataset_buffer

函数功能：获取svp_acl_mdl_dataset中的第n个svp_acl_data_buffer。同步接口。

函数原型：

```
svp_acl_data_buffer *svp_acl_mdl_get_dataset_buffer(const svp_acl_mdl_dataset *dataset, size_t index)
```

参数说明：

参数名	输入/输出	说明
dataset	输入	svp_acl_mdl_dataset地址。
index	输入	表明获取的是第几个 svp_acl_data_buffer。

返回值说明：

- 获取成功，返回svp_acl_data_buffer的地址。
- 获取失败返回空地址。

9.12.17 svp_acl_mdl_desc

9.12.17.1 svp_acl_mdl_create_desc

函数功能：创建svp_acl_mdl_desc类型的数据，表示模型描述信息。同步接口。

函数原型：

```
svp_acl_mdl_desc *svp_acl_mdl_create_desc()
```

参数说明：无

返回值说明：返回svp_acl_mdl_desc类型的指针。

9.12.17.2 svp_acl_mdl_destroy_desc

函数功能：销毁svp_acl_mdl_desc类型的数据。同步接口。

函数原型：

```
svp_acl_error svp_acl_mdl_destroy_desc(svp_acl_mdl_desc *model_desc)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	待销毁的svp_acl_mdl_desc类型的指针。



返回值说明：返回0表示成功，返回非0表示失败。

9.12.17.3 `svp_acl_md5_get_desc`

函数功能：根据模型ID获取该模型的模型描述信息。同步接口。

函数原型：

```
svp_acl_error svp_acl_md5_get_desc(svp_acl_md5_desc *model_desc, uint32_t model_id)
```

参数说明：

参数名	输入/输出	说明
model_desc	输出	<code>svp_acl_md5_desc</code> 类型数据的指针。 模型ID。 <code>svp_acl_md5_load_from_mem</code> 接口加载模型成功后，会返回模型ID。
model_id	输入	

返回值说明：返回0表示成功，返回非0表示失败。

9.12.17.4 `svp_acl_md5_get_reuse_buffer_type`

函数功能：根据模型描述信息获取模型的WorkBuf与输入输出内存的复用类型。同步接口。

函数原型：

```
svp_acl_buffer_reuse_type svp_acl_md5_get_reuse_buffer_type(const svp_acl_md5_desc *model_desc)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	<code>svp_acl_md5_desc</code> 类型数据的指针。

返回值说明：WorkBuf与输入输出内存的复用类型，返回SVP_ACL_BUFFER_REUSE_UNDEFINED表示接口调用异常。

9.12.17.5 `svp_acl_md5_get_num_inputs`

函数功能：根据模型描述信息获取模型的输入个数。同步接口。

函数原型：

```
size_t svp_acl_md5_get_num_inputs(const svp_acl_md5_desc *model_desc)
```

参数说明：



参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。

返回值说明：模型的输入个数。

9.12.17.6 svp_acl_mdl_get_num_outputs

函数功能：根据模型描述信息获取模型的输出个数。同步接口。

函数原型：

```
size_t svp_acl_mdl_get_num_outputs(const svp\_acl\_mdl\_desc *modelDesc)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。

返回值说明：模型的输出个数。

9.12.17.7 svp_acl_mdl_get_input_size_by_index

函数功能：根据模型描述信息获取指定输入的大小，单位为Byte。同步接口。

函数原型：

```
size_t svp_acl_mdl_get_input_size_by_index(const svp\_acl\_mdl\_desc *model_desc, size_t index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。
index	输入	指定获取第几个输入的大小，index值从0开始。

返回值说明：

- 返回指定输入单batch大小。单位是Byte。
- 非YUV输入的input_size = d_0 * d_1 *... * d_n-2 * default_stride;
- YUV420SP_U8输入的input_size = d_0 * d_i * ... * d_n-2 * default_stride / 2;
- YUV422SP_U8输入的input_size = d_0 * d_i * ... * d_n-2 * default_stride / 3 * 2;
- 其中d_i表示第i维长度，n表示指定输入的维度个数，default_stride为默认stride(详细请参见[svp_acl_mdl_get_input_default_stride](#))。
- 实际使用中的stride可能不等于default_stride，实际输入大小actual_input_size可以通过input_size / default_stride * actual_stride换算得到，也可将上述的计算公



式中的default_stride替换为actual_stride计算获得，其中actual_input_size表示实际所需输入内存大小，actual_stride表示实际stride大小。

9.12.17.8 svp_acl_mdl_get_output_size_by_index

函数功能：根据模型描述信息获取指定输出的大小，单位为Byte。同步接口。

函数原型：

```
size_t svp_acl_mdl_get_output_size_by_index(const svp_acl_mdl_desc *model_desc, size_t index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc类型数据的指针。
index	输入	指定获取第几个输出的大小，index值从0开始。

返回值说明：

- 返回指定输出的单batch大小。单位是Byte。
- 返回 $output_size = d_0 * d_1 * \dots * d_{n-2} * default_stride$ ，其中d_i表示第i维长度，n表示指定输出的维度个数，default_stride为默认stride(详细请参见 svp_acl_mdl_get_output_default_stride)。
- 实际使用中的stride可能不等于default_stride，实际输出大小actual_output_size可以通过 $output_size / default_stride * actual_stride$ 换算得到，也可将上述的计算公式中的default_stride替换为actual_stride计算获得。其中actual_output_size表示实际所需输出内存大小，actual_stride表示实际stride大小。

9.12.17.9 svp_acl_mdl_get_input_dims

函数功能：根据模型描述信息获取模型的输入tensor的维度信息。同步接口。

函数原型：

```
svp_acl_error svp_acl_mdl_get_input_dims(const svp_acl_mdl_desc *model_desc, size_t index, svp_acl_mdl_io_dims *dims)
```

参数说明：

表 9-3 图像输入维度定义规则

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc类型数据的指针。
index	输入	指定获取第几个输入的Dims，index值从0开始。
dims	输出	获取输入维度信息。



图像格式	Format参考格式	维度定义规则
YUV420SP_U8	NCHW	n,c,h,w, 其中c=3
YUV422SP_U8	NCHW	n,c,h,w, 其中c=3
XRGB_PACKAGE_U8	NCHW	n,c,h,w, 其中c=1
XRGB_PLANAR_U8	NCHW	n,c,h,w, 其中c=4
RGB_PACKAGE_U8	NCHW	n,h,w,c, 其中c=1
RGB_PLANAR_U8	NCHW	n,c,h,w, 其中c=3
YUV400_U8	NCHW	n,h,w,c, 其中c=1
UV_PACKAGE_U8	NCHW	n,h,w,c, 其中c=1

返回值说明：返回0表示成功，返回非0表示失败。

说明

PACKAGE类型的图像格式的维度按照图像在DDR排布设定，c为1，图像的数据位宽会根据图像类型扩展。

9.12.17.10 `svp_acl_md5_get_output_dims`

函数功能：根据模型描述信息获取指定的模型输出tensor的维度信息。同步接口。

函数原型：

```
svp_acl_error svp_acl_md5_get_output_dims(const svp_acl_md5_desc *model_desc, size_t index, svp_acl_md5_io_dims *dims)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	<code>svp_acl_md5_desc</code> 类型数据的指针。
index	输入	指定获取第几个输出的Dims，index值从0开始。
dims	输出	获取输出维度信息。

返回值说明：返回0表示成功，返回非0表示失败。

9.12.17.11 `svp_acl_md5_get_input_name_by_index`

函数功能：根据模型描述信息获取模型中指定输入的bottom名称。同步接口。

函数原型：

```
const char *svp_acl_md5_get_input_name_by_index(const svp_acl_md5_desc *model_desc, size_t index)
```



参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。
index	输入	指定获取第几个输入的名称，index值从0开始。

返回值说明：返回指定输入的bottom名称，task_buf和work_buf返回的名称分别为“task_buf”和“work_buf”。

9.12.17.12 svp_acl_mdl_get_output_name_by_index

函数功能：根据模型描述信息获取模型中指定输出的top名称。同步接口。

函数原型：

```
const char *svp_acl_mdl_get_output_name_by_index(const svp\_acl\_mdl\_desc *model_desc, size_t index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。
index	输入	指定获取第几个输出的top名称，index值从0开始。

返回值说明：返回指定输出的top名称。

9.12.17.13 svp_acl_mdl_get_input_format

函数功能：根据模型描述信息获取模型中指定输入的Format。同步接口。

函数原型：

```
svp\_acl\_format svp_acl_mdl_get_input_format(const svp\_acl\_mdl\_desc *model_desc, size_t index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。
index	输入	指定获取第几个输入的Format，index值从0开始。

返回值说明：返回指定输入的Format。



9.12.17.14 svp_acl_md5_get_output_format

函数功能：根据模型描述信息获取模型中指定输出的Format。同步接口。

函数原型：

```
svp_acl_format svp_acl_md5_get_output_format(const svp_acl_md5_desc *model_desc, size_t index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_md5_desc类型数据的指针。
index	输入	指定获取第几个输出的Format，index值从0开始。

返回值说明：返回指定输出的Format。

9.12.17.15 svp_acl_md5_get_input_data_type

函数功能：根据模型描述信息获取模型中指定输入的数据类型。同步接口。

函数原型：

```
svp_acl_data_type svp_acl_md5_get_input_data_type(const svp_acl_md5_desc *model_desc, size_t index)
```

参数说明：

表 9-4 图像输入数据类型定义规则

参数名	输入/输出	说明
model_desc	输入	svp_acl_md5_desc类型数据的指针。
index	输入	指定获取第几个输入的数据类型，index值从0开始。

图像格式	数据类型定义规则
YUV420SP_U8	SVP_ACL_UINT8
YUV422SP_U8	SVP_ACL_UINT8
XRGB_PACKAGE_U8	SVP_ACL_UINT32
XRGB_PLANAR_U8	SVP_ACL_UINT8
RGB_PACKAGE_U8	SVP_ACL_UINT24
RGB_PLANAR_U8	SVP_ACL_UINT8
UV_PACKAGE_U8	SVP_ACL_UINT16



图像格式	数据类型定义规则
YUV400_U8	SVP_ACL_UINT8

返回值说明：返回指定输入的数据类型。

9.12.17.16 `svp_acl_md5_get_output_data_type`

函数功能：根据模型描述信息获取模型中指定输出的数据类型。同步接口。

函数原型：

```
svp_acl_data_type svp_acl_md5_get_output_data_type(const svp_acl_md5_desc *model_desc, size_t index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	<code>svp_acl_md5_desc</code> 类型数据的指针。
index	输入	指定获取第几个输出的数据类型，index值从0开始。

返回值说明：返回指定输出的数据类型。

9.12.17.17 `svp_acl_md5_get_input_index_by_name`

函数功能：根据模型中的输入名称获取对应输入的索引编号。同步接口。

函数原型：

```
svp_acl_error svp_acl_md5_get_input_index_by_name(const svp_acl_md5_desc *model_desc, const char *name, size_t *index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	<code>svp_acl_md5_desc</code> 类型数据的指针。
name	输入	指定输入名称。 获取task_buf， work_buf索引时，输入的名称分别为“task_buf”，“work_buf”。
index	输出	获取对应输入的索引编号。

返回值说明：返回0表示成功，返回非0表示失败。



9.12.17.18 `svp_acl_md5_get_output_index_by_name`

函数功能：根据模型中的输出名称获取对应输出的索引编号。同步接口。

函数原型：

```
svp_acl_error_t svp_acl_md5_get_output_index_by_name(const svp_acl_md5_desc_t *model_desc,
const char *name, size_t *index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	<code>svp_acl_md5_desc_t</code> 类型数据的指针。
name	输入	指定输出名称。
index	输出	获取对应输出的索引编号。

返回值说明：返回0表示成功，返回非0表示失败。

9.12.17.19 `svp_acl_md5_get_input_default_stride`

函数功能：根据模型描述信息获取模型中指定输入的默认stride。同步接口。

函数原型：

```
size_t svp_acl_md5_get_input_default_stride(const svp_acl_md5_desc_t *model_desc, size_t index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	<code>svp_acl_md5_desc_t</code> 类型数据的指针。
index	输入	获取对应输入的索引编号。

返回值说明：

- 返回指定输入的默认stride大小。单位是Byte。
- 默认stride = ceil (ceil(last_dim * `svp_acl_data_type_size`(data_type) / byte_bit_num) / default_align_byte) * default_align_byte。
- 其中ceil()函数表示向上取整；last_dim表示指定输入的最后一维长度；data_type表示指定输入的数据类型；byte_bit_num 表示一个字节所占bit数，数值为8；default_align_byte表示生成.om时[internal_stride]项所配置的对齐字节数，数值大于0且为16的整数倍并且不超过1024。
- 如果实际使用中用户需要根据实际需要计算输入stride，可将上述计算中default_align_byte替换成实际所需的对齐字节数actual_align_byte，actual_align_byte不能为0并且为16的整数倍。



说明

计算输入默认stride所使用的实际对齐字节数是ATC模型离线生成时根据"--internal_stride"关键字指定的，详情请参考《ATC工具使用指南》。

9.12.17.20 `svp_acl_md5_get_output_default_stride`

函数功能：根据模型描述信息获取模型中指定输出的默认stride。同步接口。

函数原型：

```
size_t svp_acl_md5_get_output_default_stride(const svp_acl_md5_desc *model_desc, size_t index)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	<code>svp_acl_md5_desc</code> 类型数据的指针。
index	输入	获取对应输出的索引编号。

返回值说明：

- 返回指定输出的默认stride大小。单位是Byte。
- 默认stride = ceil (ceil(last_dim * `svp_acl_data_type_size`(data_type) / byte_bit_num) / default_align_byte) * default_align_byte。
- 其中ceil()函数表示向上取整；last_dim表示指定输出的最后一维长度；data_type表示指定输出的数据类型；byte_bit_num 表示一个字节所占bit数，数值为8；default_align_byte表示生成.om时[internal_stride]项所配置的对齐字节数，数值为16的整数倍且不超过1024。
- 如果实际使用中用户需要根据实际需要计算输出stride，可将上述计算中default_align_byte替换成实际所需的对齐字节数actual_align_byte，actual_align_byte不能为0并且为16的整数倍。

说明

计算输出默认stride所使用的实际对齐字节数是ATC模型离线生成时根据"--internal_stride"关键字指定的，详情请参考《ATC工具使用指南》。

9.12.17.21 `svp_acl_md5_get_dynamic_hw`

函数功能：

根据模型描述信息获取模型支持的动态宽高信息。同步接口。

函数原型：

```
svp_acl_error svp_acl_md5_get_dynamic_hw(const svp_acl_md5_desc *model_desc, size_t index, svp_acl_md5_hw *hw)
```

参数说明：



参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。
index	输入	标识动态HW输入的输入index，当前固定为SVP_ACL_DYNAMIC_TENSOR_NAME的index，值为0。
hw	输出	<pre>#define SVP_ACL_MAX_HW_NUM 100 typedef struct { size_t hw_count; /* 模型中支持的宽高分档数 */ uint64_t hw[SVP_ACL_MAX_HW_NUM][2]; /* <div>模型中支持的具体分档，组分档中，数组下标为0代表的是高，下标为1代表的是宽 */ } svp_acl_mdl_hw</pre> hwCount等于0时，表示不支持设置档位信息，以模型中的档位为准。

返回值说明：
返回0表示成功，返回非0表示失败。

9.12.17.22 svp_acl_mdl_get_input_class_id_by_index

函数功能：根据模型描述信息获取模型中指定输入的class_id。同步接口。
函数原型：

```
svp_acl_error svp_acl_mdl_get_input_class_id_by_index(const svp\_acl\_mdl\_desc *model_desc,
size_t index, size_t *class_id)
```

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。
index	输入	获取对应输出的索引编号。
class_id	输出	获取对应的class_id。

返回值说明：返回0表示成功，返回非0表示失败。
该接口仅用于开启数据安全后获取模型对输入要求（是否使用安全内存及具体允许使用的id）的场景。
class_id说明详见《安全开发指南》。
注：仿真不支持该接口

9.12.17.23 svp_acl_mdl_get_output_class_id_by_index

函数功能：根据模型描述信息获取模型中指定输出的class_id。同步接口。



函数原型：

```
svp_acl_error svp_acl_mdl_get_output_class_id_by_index(const svp_acl_mdl_desc *model_desc,  
size_t index, size_t *class_id
```

参数说明：

参数名	输入/ 输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。
index	输入	获取对应输出的索引编号。
class_id	输出	获取对应的class_id。

返回值说明：返回0表示成功，返回非0表示失败。

该接口仅用于开启数据安全后获取模型对输出要求（是否使用安全内存及具体允许使用的id）的场景。

class_id说明详见《安全开发指南》。

注：仿真不支持该接口。

9.12.17.24 svp_acl_mdl_get_input_reuse_type

函数功能：根据模型描述信息获取模型中指定输入的输入内存与WorkBuf内存的复用类型。同步接口。

函数原型：

```
svp_acl_inout_reuse_type svp_acl_mdl_get_input_reuse_type(const svp_acl_mdl_desc  
*model_desc, size_t index)
```

参数说明：

参数名	输入/ 输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。
index	输入	获取对应输入的索引编号。

返回值说明：输入内存与WorkBuf的复用类型，返回SVP_ACL_INOUT_REUSE_UNDEFINED表示接口调用异常。

9.12.17.25 svp_acl_mdl_get_output_reuse_type

函数功能：根据模型描述信息获取模型中指定输出的输出内存与WorkBuf内存的复用类型。同步接口。

函数原型：



svp_acl_inout_reuse_type svp_acl_mdl_get_output_reuse_type(const **svp_acl_mdl_desc** *model_desc, size_t index)

参数说明：

参数名	输入/输出	说明
model_desc	输入	svp_acl_mdl_desc 类型数据的指针。
index	输入	获取对应输出的索引编号。

返回值说明：输出内存与WorkBuf的复用类型，返回SVP_ACL_INOUT_REUSE_UNDEFINED表示接口调用异常。

9.12.18 svp_acl_prof_config

9.12.18.1 svp_acl_prof_create_config

函数功能：

- 创建svp_acl_prof_config类型的数据，表示创建Profiling配置数据。同步接口。
- svp_acl_prof_config类型数据可以只创建一次、多处使用，用户需要保证数据的一致性和准确性。
- 如需销毁svp_acl_prof_config类型的数据，请参见 **svp_acl_prof_destroy_config**。

函数原型：

svp_acl_prof_config *svp_acl_prof_create_config(const uint32_t *device_id_list, uint32_t device_num, **svp_acl_prof_aicore_metrics** aicore_metrics, const svp_acl_prof_aicore_events *aicore_events, uint64_t data_type_config)

参数说明：

参数名	输入/输出	说明
device_id_list	输入	Device ID列表。
device_num	输入	Device的个数。需由用户保证device_id_list中的Device个数与device_num参数值一致，否则可能会导致后续业务异常。
aicore_metrics	输入	表示模式识别Core性能指标采集项。只支持配置为0。
aicore_events	输入	表示模式识别Core事件，目前配置为NULL。



参数名	输入/输出	说明
data_type_config	输入	用户选择如下多个宏进行逻辑或（例如：SVP_ACL_PROF_ACL_API SVP_ACL_PROF_AICORE_METRICS），作为data_type_config参数值。每个宏表示某一类性能数据，详细说明如下： • SVP_ACL_PROF_ACL_API：表示采集ACL接口的性能数据。 • SVP_ACL_PROF_TASK_TIME：表示采集模式识别Core算子的执行时间。 • SVP_ACL_PROF_AICORE_METRICS：表示采集模式识别Core性能指标数据，逻辑或时必须包括该宏，aicore_metrics入参处配置的性能指标采集项才有效。 • SVP_ACL_PROF_AICPU：表示采集模式识别CPU任务的开始、结束轨迹数据。

返回值说明：

- 返回svp_acl_prof_config类型的指针，表示成功。
- 返回nullptr，表示失败。

9.12.18.2 svp_acl_prof_destroy_config

函数功能：销毁通过[svp_acl_prof_create_config](#)接口创建的svp_acl_prof_config类型的数据。同步接口。

函数原型：

svp_acl_error svp_acl_prof_destroy_config(const **svp_acl_prof_config** *profiler_config)

参数说明：

参数名	输入/输出	说明
profiler_config	输入	待销毁的svp_acl_prof_config类型的指针。

返回值说明：返回0表示成功，返回非0表示失败。

9.12.19 svp_acl_prof_subscribe_config



9.12.19.1 svp_acl_prof_create_subscribe_config

函数功能：

- 创建svp_acl_prof_subscribe_config类型的数据，表示创建订阅配置信息。同步接口。
- 如需销毁svp_acl_prof_subscribe_config类型的数据，请参见[svp_acl_prof_destroy_subscribe_config](#)。

函数原型：

```
svp_acl_prof_subscribe_config *svp_acl_prof_create_subscribe_config(int8_t time_info_switch,  
svp_acl_prof_aicore_metrics aicore_metrics, const void *fd)
```

参数说明：

参数名	输入/输出	说明
time_info_switch	输入	是否采集网络模型中算子的性能数据： • 1：采集 • 0：不采集
aicore_metrics	输入	表示模式识别Core性能指标采集项。只支持配置为0。
fd	输入	用户创建的管道写指针。 指针fd指向的值范围不能小于0。

返回值说明：

- 返回svp_acl_prof_subscribe_config类型的指针，表示成功。
- 返回nullptr，表示失败。

9.12.19.2 svp_acl_prof_destroy_subscribe_config

函数功能：销毁通过[svp_acl_prof_create_subscribe_config](#)接口创建的svp_acl_prof_subscribe_config类型的数据。同步接口。

函数原型：

```
svp_acl_error svp_acl_prof_destroy_subscribe_config(const svp_acl_prof_subscribe_config  
*prof_subscribe_config)
```

参数说明：

参数名	输入/输出	说明
prof_subscribe_config	输入	待销毁的svp_acl_prof_subscribe_config类型的指针。

返回值说明：返回0表示成功，返回非0表示失败。



9.12.20 svp_acl_mdl_config_attr

```
typedef enum {
    SVP_ACL_MDL_PRIORITY_INT32 = 0, //模型执行的优先级，高优先级任务会优先调度，数字越小优先级越高，取值[0,7]，可选项。默认值为3。
    SVP_ACL_MDL_LOAD_TYPE_SIZET, //模型加载方式，见下文详细的取值说明，必选项。
    SVP_ACL_MDL_PATH_PTR, //离线模型文件路径的指针，如果选择从文件加载模型，则该选项必选，目前不支持。
    SVP_ACL_MDL_MEM_ADDR_PTR, //模型在内存中的起始地址，如果选择从内存加载模型，则该选项必选。
    SVP_ACL_MDL_MEM_SIZET, //模型在内存中的大小，如果选择从内存加载模型，则该选项必选，与SVP_ACL_MDL_MEM_ADDR_PTR选项配合使用。
    SVP_ACL_MDL_WEIGHT_ADDR_PTR, //Device上模型权值内存（存放权值数据）的指针，如果需要由用户管理权值内存，则该选项必选。若不配置该选项，则表示由系统管理内存，目前不支持。
    SVP_ACL_MDL_WEIGHT_SIZET, //权值内存大小，单位为Byte，如果需要由用户管理权值内存，则该选项必选，与SVP_ACL_MDL_WEIGHT_ADDR_PTR选项配合使用，目前不支持。
    SVP_ACL_MDL_WORKSPACE_ADDR_PTR, //模型所需工作内存（存放模型输入/输出等数据）的指针，如果需要由用户管理工作内存，则该选项必选。若不配置该选项，则表示由系统管理内存，目前不支持。
    SVP_ACL_MDL_WORKSPACE_SIZET, //模型所需工作内存的大小，单位为Byte，如果需要由用户管理权值内存，则该选项必选，与SVP_ACL_MDL_WORKSPACE_ADDR_PTR选项配合使用，目前不支持。
    SVP_ACL_MDL_INPUTQ_NUM_SIZET, //模型输入队列大小，带队列加载模型时，该选项必选，与ACL_MDL_INPUTQ_ADDR_PTR选项配合使用。当前版本不支持该接口，目前不支持。
    SVP_ACL_MDL_INPUTQ_ADDR_PTR, //模型输入队列ID的指针，带队列加载模型时，该选项必选，一个模型输入对应一个队列ID。当前版本不支持该接口，目前不支持。
    SVP_ACL_MDL_OUTPUTQ_NUM_SIZET, //模型输出队列大小，带队列加载模型时，该选项必选，与ACL_MDL_OUTPUTQ_ADDR_PTR选项配置使用。当前版本不支持该接口，目前不支持。
    SVP_ACL_MDL_OUTPUTQ_ADDR_PTR, //模型输出队列ID的指针，带队列加载模型时，该选项必选，一个模型输出对应一个队列ID。当前版本不支持该接口，目前不支持。
    SVP_ACL_MDL_PRIORITY_PREEMP_EN_BOOL, //是否使能可抢占低优先级任务，默认开启抢占。
    SVP_ACL_MDL_PRIORITY_UP_STEP_TIMEOUT_UINT32, //优先级逐步提升超时时间（单位毫秒），取值0或者[15,600000]，为0代表不开启优先级逐步提升。
    SVP_ACL_MDL_PRIORITY_UP_TOP_TIMEOUT_UINT32, //优先级最高提升超时时间（单位毫秒），取值0或者[15,600000]，为0代表不开启优先级最高提升。
} svp_acl_mdl_config_attr;
```

SVP_ACL_MDL_LOAD_TYPE_SIZET（表示模型加载方式）的取值使用如下宏：

- SVP_ACL_MDL_LOAD_FROM_FILE（目前不支持）
#define SVP_ACL_MDL_LOAD_FROM_FILE 1
- SVP_ACL_MDL_LOAD_FROM_FILE_WITH_MEM（目前不支持）
#define SVP_ACL_MDL_LOAD_FROM_FILE_WITH_MEM 2
- SVP_ACL_MDL_LOAD_FROM_MEM
#define SVP_ACL_MDL_LOAD_FROM_MEM 3
- SVP_ACL_MDL_LOAD_FROM_MEM_WITH_MEM（目前不支持）
#define SVP_ACL_MDL_LOAD_FROM_MEM_WITH_MEM 4
- SVP_ACL_MDL_LOAD_FROM_FILE_WITH_Q（目前不支持）
#define SVP_ACL_MDL_LOAD_FROM_FILE_WITH_Q 5

9.12.21 svp_acl_mdl_config_handle

9.12.21.1 svp_acl_mdl_create_config_handle

函数功能：创建svp_acl_mdl_config_handle类型的数据，表示一个模型加载的配置对象。同步接口。如需销毁svp_acl_mdl_config_handle类型的数据，请参见[svp_acl_mdl_destroy_config_handle](#)。

函数原型：

```
svp_acl_mdl_config_handle *svp_acl_mdl_create_config_handle()
```

参数说明：无。



返回值说明：

- 返回svp_acl_mdل_config_handle类型的指针，表示成功。
- 返回nullptr，表示失败。

9.12.21.2 svp_acl_mdل_destroл_config_handle

函数功能：销毁通过svp_acl_mdل_create_config_handle接口创建的svp_acl_mdل_config_handle类型的数据。同步接口。

函数原型：

svp_acl_error svp_acl_mdل_destroл_config_handle (svp_acl_mdل_config_handle *handle)

参数说明：

参数名	输入/输出	说明
handle	输入	待销毁的svp_acl_mdل_config_handle类型的指针。

返回值说明：返回0表示成功，返回非0表示失败。

9.12.22 svp_acl_aipp_format

支持的图像格式详情如下：

```
typedef enum {
    SVP_ACL_YUV420SP_U8 = 0x1,
    SVP_ACL_XRGB8888_U8 = 0x2,
    SVP_ACL_RGB888_U8 = 0x3,
    SVP_ACL_YUV400_U8 = 0x4,
    SVP_ACL_NC1HWC0DI_FP16 = 0x5, /* not support */
    SVP_ACL_NC1HWC0DI_S8 = 0x6, /* not support */
    SVP_ACL_ARGB8888_U8 = 0x7, /* not support */
    SVP_ACL_YUYV_U8 = 0x8, /* not support */
    SVP_ACL_YUV422SP_U8 = 0x9,
    SVP_ACL_AYUV444_U8 = 0xA, /* not support */
    SVP_ACL_RAW10 = 0xB, /* not support */
    SVP_ACL_RAW12 = 0xC, /* not support */
    SVP_ACL_RAW16 = 0xD, /* not support */
    SVP_ACL_RAW24 = 0xE, /* not support */
    SVP_ACL_FEATURE_MAP_U8 = 0x100,
    SVP_ACL_FEATURE_MAP_S8 = 0x101,
    SVP_ACL_FEATURE_MAP_U16 = 0x102,
    SVP_ACL_FEATURE_MAP_S16 = 0x103,
    SVP_ACL_FEATURE_MAP_FP16 = 0x104,
    SVP_ACL_FEATURE_MAP_FP32 = 0x105,
    SVP_ACL_YVU420SP_U8 = 0x106,
    SVP_ACL_YVU422SP_U8 = 0x107,
    SVP_ACL_BGR_PLANAR_U8 = 0x108,
    SVP_ACL_RGB_PLANAR_U8 = 0x109,
    SVP_ACL_BGR_PACKAGE_U8 = 0x10A,
    SVP_ACL_XRGB_PLANAR_U8 = 0x10B,
    SVP_ACL_XBGR_PLANAR_U8 = 0x10C,
    SVP_ACL_RGBX_PLANAR_U8 = 0x10D,
    SVP_ACL_BGRX_PLANAR_U8 = 0x10E,
```



```

SVP_ACL_XBGR_PACKAGE_U8 = 0x10F,
SVP_ACL_RGBX_PACKAGE_U8 = 0x110,
SVP_ACL_BGRX_PACKAGE_U8 = 0x111,
SVP_ACL_RGG8_RAW8 = 0x112,
SVP_ACL_GRBG_RAW8 = 0x113,
SVP_ACL_GBRG_RAW8 = 0x114,
SVP_ACL_BGGR_RAW8 = 0x115,
SVP_ACL_RGG8_RAW10 = 0x116,
SVP_ACL_GRBG_RAW10 = 0x117,
SVP_ACL_GBRG_RAW10 = 0x118,
SVP_ACL_BGGR_RAW10 = 0x119,
SVP_ACL_RGG8_RAW12 = 0x11a,
SVP_ACL_GRBG_RAW12 = 0x11b,
SVP_ACL_GBRG_RAW12 = 0x11c,
SVP_ACL_BGGR_RAW12 = 0x11d,
SVP_ACL_RGG8_RAW16 = 0x11e,
SVP_ACL_GRBG_RAW16 = 0x11f,
SVP_ACL_GBRG_RAW16 = 0x120,
SVP_ACL_BGGR_RAW16 = 0x121,
SVP_ACL_UV_PLANNAR_U8 = 0x122, /* not support */
SVP_ACL_VU_PLANNAR_U8 = 0x123, /* not support */
SVP_ACL_UV_PACKAGE_U8 = 0x124,
SVP_ACL_VU_PACKAGE_U8 = 0x125,

SVP_ACL_AIPP_RESERVED = 0xFFFF
} svp_acl_aipp_format;

```

9.12.23 svp_acl_aipp_input_format

```
typedef svp_acl_aipp_format svp_acl_aipp_input_format;
```

9.12.24 svp_acl_mdl_io_dims

```

#define SVP_ACL_MAX_DIM_CNT      128
#define SVP_ACL_MAX_TENSOR_NAME_LEN 128

typedef struct svp_acl_mdl_io_dims {
    char name[SVP_ACL_MAX_TENSOR_NAME_LEN]; /* tensor name */
    size_t dim_count; /* dim array count */
    int64_t dims[SVP_ACL_MAX_DIM_CNT]; /* dim data array */
} svp_acl_mdl_io_dims;

```

9.12.25 svp_acl_aipp_dims

```

typedef struct {
    svp_acl_mdl_io_dims src_dims; /* input dims before model transform */
    size_t src_size; /* input size before model transform */
    svp_acl_mdl_io_dims aipp_out_dims; /* aipp output dims */
    size_t aipp_out_size; /* aipp output size */
} svp_acl_aipp_dims;

```

9.12.26 svp_acl_aipp_padding_switch

```

typedef enum {
    AIPP_PADDING_SWITCH_OFF = 0,
    AIPP_PADDING_SWITCH_CONSTANT, /* 固定值补边模式 */
    AIPP_PADDING_SWITCH_REPLICATION, /* 复制补边模式 */
    AIPP_PADDING_SWITCH_REFLECTION, /* 镜像补边模式 */
    AIPP_PADDING_SWITCH_SYMMETRIC, /* 对称补边模式 */
    AIPP_PADDING_SWITCH_BUTT
} svp_acl_aipp_padding_switch;

```



9.12.27 svp_acl_aipp_resize_switch

```
typedef enum {  
    AIPP_RESIZE_SWITCH_OFF = 0,  
    AIPP_RESIZE_SWITCH_NEAREST, /* 最近邻插值模式缩放 */  
    AIPP_RESIZE_SWITCH_BILINEAR, /* 双线性插值模式缩放 */  
    AIPP_RESIZE_SWITCH_BUTT  
} svp_acl_aipp_resize_switch;
```

9.12.28 svp_acl_mdl_aipp_type

```
typedef enum {  
    SVP_ACL_DATA_WITHOUT_AIPP = 0,  
    SVP_ACL_DATA_WITH_STATIC_AIPP,  
    SVP_ACL_DATA_WITH_DYNAMIC_AIPP,  
    SVP_ACL_DYNAMIC_AIPP_NODE  
} svp_acl_mdl_aipp_type;
```

9.12.29 svp_acl_aipp_info

```
#define SVP_ACL_MAX_SHAPE_COUNT    1  
typedef struct svp_acl_aipp_extend_info svp_acl_aipp_extend_info; /* 扩展信息，不支持 */  
typedef struct {  
    svp_acl_aipp_format aipp_format;  
    int32_t src_image_width;  
    int32_t src_image_height;  
    int8_t crop_switch;  
    int32_t crop_start_width;  
    int32_t crop_start_height;  
    int32_t crop_width;  
    int32_t crop_height;  
    int8_t resize_switch;  
    int32_t resize_output_width;  
    int32_t resize_output_height;  
    int8_t padding_switch;  
    int32_t left_padding_size;  
    int32_t right_padding_size;  
    int32_t top_padding_size;  
    int32_t bottom_padding_size;  
    int8_t csc_switch;  
    int8_t rbuv_swap_switch;  
    int8_t ax_swap_switch;  
    int8_t cmp_switch; /* 不支持 */  
    int32_t matrix_r0c0; /* CSC矩阵系数 */  
    int32_t matrix_r0c1;  
    int32_t matrix_r0c2;  
    int32_t matrix_r1c0;  
    int32_t matrix_r1c1;  
    int32_t matrix_r1c2;  
    int32_t matrix_r2c0;  
    int32_t matrix_r2c1;  
    int32_t matrix_r2c2;  
    int32_t output_bias0;  
    int32_t output_bias1;  
    int32_t output_bias2;  
    int32_t input_bias0;  
    int32_t input_bias1;  
    int32_t input_bias2;  
    float mean_chn0;  
    float mean_chn1;  
    float mean_chn2;
```



```
float mean_chn3;
float min_chn0;
float min_chn1;
float min_chn2;
float min_chn3;
float var_reci_chn0;
float var_reci_chn1;
float var_reci_chn2;
float var_reci_chn3;
svp_acl_format src_format;
svp_acl_data_type src_data_type;
size_t src_dim_num; /* 默认为1 */
size_t shape_count;
svp_acl_aipp_dims out_dims[SVP_ACL_MAX_SHAPE_COUNT];
svp_acl_aipp_extend_info *aipp_extend;
} svp_acl_aipp_info;
```

9.12.30 svp_acl_mdl_aipp

9.12.30.1 svp_acl_mdl_create_aipp

函数功能：

动态AIPP场景下，根据模型支持的Batch数创建的**svp_acl_mdl_aipp** 类型的数据，用于存放动态AIPP的参数。同步接口。

函数原型：

```
svp_acl_mdl_aipp *svp_acl_mdl_create_aipp(uint64_t batch_size)
```

参数说明：

参数名	输入/输出	说明
batch_size	输入	原始模型的Batch数。只支持取值1。

返回值说明：返回**svp_acl_mdl_aipp** 类型的指针。

9.12.30.2 svp_acl_mdl_set_aipp_input_format

函数功能：

动态AIPP场景下，必须设置原始输入图像的格式，同步接口。

函数原型：

```
svp_acl_error svp_acl_mdl_set_aipp_input_format(svp_acl_mdl_aipp *aipp_set,
svp_acl_aipp_input_format input_format);
```

约束说明：

- 不支持**svp_acl_aipp_input_format**枚举中明确标注的“not support”的数据格式。
- 输入数据格式需要与模型数据格式匹配：
 - 除了CSC支持的YUV和RGB之间的格式转换外，其它只支持通道之间的数据格式转换。



- b. RAW格式不支持位宽变化的格式转换。
- c. PACKAGE和PLANAR的格式不能转换。

参数说明：

参数名	输入/输出	说明
aipp_set	输出	动态AIPP参数对象。 提前调用 svp_acl_mdl_create_aipp 接口创建 svp_acl_mdl_aipp 类型的数据。
input_format	输入	表示原始输入图像的格式。

返回值说明：返回0表示成功，返回其它值表示失败。

9.12.30.3 svp_acl_mdl_set_aipp_csc_params

函数功能：

动态AIPP场景下，设置CSC色域转换相关的参数，若色域转换开关关闭，则调用该接口设置以下参数无效。同步接口。

```
YUV转BGR:  
| B | | matrix_r0c0 matrix_r0c1 matrix_r0c2 | | Y - input_bias_r0 |  
| G | = | matrix_r1c0 matrix_r1c1 matrix_r1c2 | | U - input_bias_r1 | >> 10  
| R | | matrix_r2c0 matrix_r2c1 matrix_r2c2 | | V - input_bias_r2 |
```

色域转换参数值与转换前图片的格式、转换后图片的格式强相关，用户可以根据实际需求配置色域转换参数。

约束说明：

图像的源格式通过[svp_acl_mdl_set_aipp_input_format](#)接口设置，只支持YUV420SP/YVU420SP/YUV422SP/YVU422SP输入格式。

函数原型：

```
svp\_acl\_error svp_acl_mdl_set_aipp_csc_params(svp\_acl\_mdl\_aipp *aipp_set, int8_t csc_switch,  
int16_t matrix_r0c0, int16_t matrix_r0c1, int16_t matrix_r0c2,  
int16_t matrix_r1c0, int16_t matrix_r1c1, int16_t matrix_r1c2,  
int16_t matrix_r2c0, int16_t matrix_r2c1, int16_t matrix_r2c2,  
uint8_t output_bias_r0, uint8_t output_bias_r1, uint8_t output_bias_r2,  
uint8_t input_bias_r0, uint8_t input_bias_r1, uint8_t input_bias_r2)
```

参数说明：

参数名	输入/输出	说明
aipp_set	输出	动态AIPP参数对象。 提前调用 svp_acl_mdl_create_aipp 接口创建 svp_acl_mdl_aipp 类型的数据。



参数名	输入/输出	说明
csc_switch	输入	色域转换开关，取值范围： 0：关闭色域转换开关，设置为0时，则设置以下参数无效 1：打开色域转换开关
matrix_r0c0	输入	色域转换矩阵参数。 取值范围：[-4096, 4095]
matrix_r0c1	输入	色域转换矩阵参数。 取值范围：[-4096, 4095]
matrix_r0c2	输入	色域转换矩阵参数。 取值范围：[-4096, 4095]
matrix_r1c0	输入	色域转换矩阵参数。 取值范围：[-4096, 4095]
matrix_r1c1	输入	色域转换矩阵参数。 取值范围：[-4096, 4095]
matrix_r1c2	输入	色域转换矩阵参数。 取值范围：[-4096, 4095]
matrix_r2c0	输入	色域转换矩阵参数。 取值范围：[-4096, 4095]
matrix_r2c1	输入	色域转换矩阵参数。 取值范围：[-4096, 4095]
matrix_r2c2	输入	色域转换矩阵参数。 取值范围：[-4096, 4095]
output_bias_r0	输入	RGB转YUV时的输出偏移。 取值范围：[0, 255]
output_bias_r1	输入	RGB转YUV时的输出偏移。 取值范围：[0, 255]
output_bias_r2	输入	RGB转YUV时的输出偏移。 取值范围：[0, 255]
input_bias_r0	输入	YUV转RGB时的输入偏移。 取值范围：[0, 255]
input_bias_r1	输入	YUV转RGB时的输入偏移。 取值范围：[0, 255]



参数名	输入/输出	说明
input_bias_r2	输入	YUV转RGB时的输入偏移。 取值范围：[0, 255]

返回值说明：返回0表示成功，返回其它值表示失败。

须知

CSC处理时会通过接口的转换参数计算得到一个偏置参数，偏置参数取值范围[-2097152, 2097151]，如果计算结果超出范围，后续关联AIPP通道时会报错。常用的CSC参数请参考文档《ATC工具使用指南》。

9.12.30.4 `svp_acl_mdl_set_aipp_rbuvs_swap_switch`

函数功能：

动态AIPP场景下，设置是否交换R通道与B通道、或者是否交换U通道与V通道，同步接口。

函数原型：

```
svp_acl_error svp_acl_mdl_set_aipp_rbuvs_swap_switch(svp_acl_mdl_aipp *aipp_set, int8_t rbuvs_swap_switch);
```

参数说明：

参数名	输入/输出	说明
aipp_set	输出	动态AIPP参数对象。 提前调用 svp_acl_mdl_create_aipp 接口创建 svp_acl_mdl_aipp 类型的数据。
rbuvs_swap_switch	输入	表示是否交换R通道与B通道、或者是否交换U通道与V通道的开关，取值范围： 0：不交换 1：交换

返回值说明：返回0表示成功，返回其它值表示失败。

9.12.30.5 `svp_acl_mdl_set_aipp_ax_swap_switch`

函数功能：

动态AIPP场景下，设置RGBX->XRGB的交换开关，同步接口。



函数原型：

```
svp_acl_error svp_acl_md1_set_aipp_ax_swap_switch(svp_acl_md1_aipp *aipp_set, int8_t ax_swap_switch);
```

参数说明：

参数名	输入/输出	说明
aipp_set	输出	动态AIPP参数对象。 提前调用 svp_acl_md1_create_aipp 接口创建 svp_acl_md1_aipp 类型的数据。
ax_swap_switch	输入	表示RGBX->XRGB的交换开关，取值范围： • 0：不交换 • 1：交换

返回值说明：返回0表示成功，返回其它值表示失败。

9.12.30.6 svp_acl_md1_set_aipp_src_image_size

函数功能：

动态AIPP场景下，必须设置原始图片的宽和高，同步接口。

函数原型：

```
svp_acl_error svp_acl_md1_set_aipp_src_image_size(svp_acl_md1_aipp *aipp_set, int32_t image_width, int32_t image_height);
```

参数说明：

参数名	输入/输出	说明
aipp_set	输出	动态AIPP参数对象。 提前调用 svp_acl_md1_create_aipp 接口创建 svp_acl_md1_aipp 类型的数据。
image_width	输入	原始图片的宽。 取值范围： [1, 16384]
image_height	输入	原始图片的高。 取值范围： [1, 16384]

返回值说明：返回0表示成功，返回其它值表示失败。

9.12.30.7 svp_acl_md1_set_aipp_scf_params

函数功能：

动态AIPP场景下，设置缩放相关的参数，同步接口。



函数原型：

```
svp_acl_error svp_acl_mdl_set_aipp_scf_params(svp_acl_mdl_aipp *aipp_set, int8_t scf_switch,  
int32_t input_width,  
int32_t input_height, int32_t output_width, int32_t output_height, uint64_t batch_index);
```

约束说明：

- 当缩放模式为最近邻插值模式时，缩放比例（ $\text{output_width}/\text{input_width}$ 和 $\text{output_height}/\text{input_height}$ ）仅支持{1/4, 1/2, 1, 2}。
- 当缩放模式为双线性插值模式时，缩放比例 $\text{output_width}/\text{input_width} \in [1/16, 16]$ 、 $\text{output_height}/\text{input_height} \in [1/16, 16]$ ；输入输出图像的宽高不超过4096。
- 动态AIPP涉及图像尺寸改变的处理顺序是：源图像->裁剪->补边->缩放->目标图像，所以缩放输入输出的宽高需要与动态AIPP处理的前后宽高一致。

参数说明：

参数名	输入/输出	说明
aipp_set	输出	动态AIPP参数对象。 提前调用 svp_acl_mdl_create_aipp 接口创建 svp_acl_mdl_aipp 类型的数据。
scf_switch	输入	是否对图片执行缩放操作，参考 svp_acl_aipp_resize_switch ，取值范围： • 0：不执行缩放操作，设置为0时，则设置input_width、input_height、output_width、output_height参数无效 • 1：执行NEAREST模式缩放操作 • 2：执行BILINEAR模式缩放操作
input_width	输入	缩放前图片的宽。 NEAREST模式取值范围：[1, 16384] BILINEAR模式取值范围：[1, 4096]
input_height	输入	缩放前图片的高。 NEAREST模式取值范围：[1, 16384] BILINEAR模式取值范围：[1, 4096]
output_width	输入	缩放后图片的宽。 NEAREST模式取值范围：[1, 16384] BILINEAR模式取值范围：[1, 4096]
output_height	输入	缩放后图片的高。 NEAREST模式取值范围：[1, 16384] BILINEAR模式取值范围：[1, 4096]



参数名	输入/输出	说明
batch_index	输入	指定对第几个Batch上的图片执行缩放操作。 取值范围：[0, batch_size) batch_size是在调用 svp_acl_md1_create_aipp 接口创建 svp_acl_md1_aipp 类型的数据时设置。

返回值说明：返回0表示成功，返回其它值表示失败。

9.12.30.8 svp_acl_md1_set_aipp_crop_params

函数功能：

动态AIPP场景下，设置抠图相关的参数，同步接口。

函数原型：

```
svp_acl_error svp_acl_md1_set_aipp_crop_params(svp_acl_md1_aipp *aipp_set, int8_t  
crop_switch,  
int32_t width_start_pos, int32_t height_start_pos, int32_t width, int32_t height, uint64_t  
batch_index);
```

约束说明：

- 若开启抠图功能，则通过[svp_acl_md1_set_aipp_src_image_size](#)接口设置的参数与通过[svp_acl_md1_set_aipp_crop_params](#)接口设置的参数之间必须满足以下公式：
 - $width + width_start_pos \leq image_width$
 - $height + height_start_pos \leq image_height$
- 抠图起始点 $width_start_pos * 图像像素位宽$ 必须是8的倍数。

参数说明：

参数名	输入/输出	说明
aipp_set	输出	动态AIPP参数对象。 提前调用 svp_acl_md1_create_aipp 接口创建 svp_acl_md1_aipp 类型的数据。
crop_switch	输入	是否对图片执行抠图操作，取值范围： 0：不执行抠图操作，设置为0时，则设置 width_start_pos、height_start_pos、 width、height参数无效 1：执行抠图操作



参数名	输入/输出	说明
width_start_pos	输入	抠图时，坐标点起始位置在图中横向的坐标。 对于YUV420SP/YUV422SP/YVU420SP/YVU422SP类型的图像，参数取值要求是偶数。 取值范围：[0, 16383]
height_start_pos	输入	抠图时，坐标点起始位置在图中纵向的坐标。 对于YUV420SP/YVU420SP类型的图像，参数取值要求是偶数。 取值范围：[0, 16383]
width	输入	抠图区域的宽度。 对于YUV420SP/YUV422SP/YVU420SP/YVU422SP/RAW类型的图像，参数取值要求是偶数。 取值范围：[1, 16384]
height	输入	抠图区域的高度。 对于YUV420SP/YVU420SP/RAW类型的图像，参数取值要求是偶数。 取值范围：[1, 16384]
batch_index	输入	指定对第几个Batch上的图片执行抠图操作。 取值范围：[0, batch_size) batch_size是在调用 svp_acl_md_create_aipp 接口创建 svp_acl_md_aipp 类型的数据时设置。

返回值说明：返回0表示成功，返回其它值表示失败。

9.12.30.9 svp_acl_md_set_aipp_padding_params

函数功能：

动态AIPP场景下，设置补边相关的参数，同步接口。

函数原型：

```
svp_acl_error svp_acl_md_set_aipp_padding_params(svp_acl_md_aipp *aipp_set, int8_t padding_switch, int32_t top, int32_t bottom, int32_t left, int32_t right, uint64_t batch_index);
```

约束说明：

对于RAW格式的图像，要求（top + bottom）的结果为偶数，要求（left + right）的结果为偶数。

参数说明：



参数名	输入/输出	说明
aipp_set	输出	动态AIPP参数对象。 提前调用 svp_acl_md1_create_aipp 接口创建 svp_acl_md1_aipp 类型的数据。
padding_switch	输入	对图片执行补边操作选项，参考 svp_acl_aipp_padding_switch 取值范围： 0：不执行补边操作，设置为0时，则设置top、bottom、left、right参数无效 1：执行固定值模式补边操作 2：执行复制模式补边操作 3：执行镜像模式补边操作 4：执行对称模式补边操作 Hi3516CV610只支持配置0，1，2三种模式。
top	输入	在图片上方填充的值。 取值范围：[0, 15]
bottom	输入	在图片下方填充的值。 取值范围：[0, 15]
left	输入	在图片左方填充的值。 取值范围：[0, 15]
right	输入	在图片右方填充的值。 取值范围：[0, 15]
batch_index	输入	指定对第几个Batch上的图片执行补边操作。 取值范围：[0, batch_size) batch_size是在调用 svp_acl_md1_create_aipp 接口创建 svp_acl_md1_aipp 类型的数据时设置。

返回值说明：返回0表示成功，返回其它值表示失败。

9.12.30.10 svp_acl_md1_set_aipp_dtc_pixel_mean

函数功能：

动态AIPP场景下，设置通道的均值，同步接口。

函数原型：

```
svp\_acl\_error svp_acl_md1_set_aipp_dtc_pixel_mean(svp\_acl\_md1\_aipp *aipp_set, float mean_chn0, float mean_chn1, float mean_chn2, float mean_chn3, uint64_t batch_index);
```

参数说明：



参数名	输入/输出	说明
aipp_set	输出	动态AIPP参数对象。 提前调用 svp_acl_mdsl_create_aipp 接口创建 svp_acl_mdsl_aipp 类型的数据。
mean_chn0	输入	通道0的均值。 取值范围：[-65504f, 65504f]
mean_chn1	输入	通道1的均值。 取值范围：[-65504f, 65504f]
mean_chn2	输入	通道2的均值。 取值范围：[-65504f, 65504f]
mean_chn3	输入	通道3的均值。 如果只有3个通道，将该参数设置为0。 取值范围：[-65504f, 65504f]
batch_index	输入	指定对第几个Batch上的图片设置通道均值。 取值范围：[0, batch_size) batch_size是在调用 svp_acl_mdsl_create_aipp 接口创建 svp_acl_mdsl_aipp 类型的数据时设置。

返回值说明：返回0表示成功，返回其它值表示失败。

9.12.30.11 svp_acl_mdsl_set_aipp_dtc_pixel_min

函数功能：

动态AIPP场景下，设置通道的最小值，同步接口。

函数原型：

[svp_acl_error](#) svp_acl_mdsl_set_aipp_dtc_pixel_min([svp_acl_mdsl_aipp](#) *aipp_set, float min_chn0, float min_chn1, float min_chn2, float min_chn3, uint64_t batch_index);

参数说明：

参数名	输入/输出	说明
aipp_set	输出	动态AIPP参数对象。 提前调用 svp_acl_mdsl_create_aipp 接口创建 svp_acl_mdsl_aipp 类型的数据。
min_chn0	输入	通道0的最小值。 取值范围：[-65504f, 65504f]



参数名	输入/输出	说明
min_chn1	输入	通道1的最小值。 取值范围：[-65504f, 65504f]
min_chn2	输入	通道2的最小值。 取值范围：[-65504f, 65504f]
min_chn3	输入	通道3的最小值。如果只有3个通道，将该参数设置为0。 取值范围：[-65504f, 65504f]
batch_index	输入	指定对第几个Batch上的图片设置通道最小值。 取值范围：[0, batch_size) batch_size是在调用 svp_acl_md_create_aipp 接口创建 svp_acl_md_aipp 类型的数据时设置。

返回值说明：返回0表示成功，返回其它值表示失败。

9.12.30.12 svp_acl_md_set_aipp_pixel_var_reci

函数功能：

动态AIPP场景下，设置通道的方差倒数，同步接口。

函数原型：

```
svp_acl_error svp_acl_md_set_aipp_pixel_var_reci(svp_acl_md_aipp *aipp_set, float reci_chn0, float reci_chn1, float reci_chn2, float reci_chn3, uint64_t batch_index);
```

参数说明：

参数名	输入/输出	说明
aipp_set	输出	动态AIPP参数对象。 提前调用 svp_acl_md_create_aipp 接口创建 svp_acl_md_aipp 类型的数据。
reci_chn0	输入	通道0的方差倒数，默认值为1.0。 取值范围：[0f, 65504f]
reci_chn1	输入	通道1的方差倒数，默认值为1.0。 取值范围：[0f, 65504f]
reci_chn2	输入	通道2的方差倒数，默认值为1.0。 取值范围：[0f, 65504f]



参数名	输入/输出	说明
reci_chn3	输入	通道3的方差倒数，默认值为1.0。如果只有3个通道，则将该参数设置为1.0。 取值范围：[0f, 65504f]
batch_index	输入	指定对第几个Batch上的图片设置通道的方差。 取值范围：[0, batch_size) batch_size是在调用 svp_acl_md_create_aipp 接口创建 svp_acl_md_aipp 类型的数据时设置。

返回值说明：返回0表示成功，返回其它值表示失败。

9.12.30.13 svp_acl_md_destroy_aipp

函数功能：

销毁[svp_acl_md_aipp](#)类型的数据。同步接口。

函数原型：

```
svp_acl_error svp_acl_md_destroy_aipp(svp_acl_md_aipp *aipp_set);
```

参数说明：

参数名	输入/输出	说明
aipp_set	输入	待销毁的 svp_acl_md_aipp 类型的指针。

返回值说明：返回0表示成功，返回其它值表示失败。

9.12.31 svp_acl_status

9.12.31.1 svp_acl_query_status

函数功能：获取当前设备上NPU核的信息。

函数原型：

```
svp_acl_error svp_acl_query_status(svp_acl_status *status)
```

参数说明：



参数名	输入/输出	说明
status	输出	<pre>#define SVP_ACL_MAX_IP_NUM 8 typedef struct { uint32_t ip_num; uint32_t hw_utilization[SVP_ACL_MAX_IP_NUM]; uint32_t tflops_100x[SVP_ACL_MAX_IP_NUM]; /* NPU的算力 * 100后的值 */ } svp_acl_status;</pre>

返回值说明：返回0表示成功，返回非0表示失败。



10 SVP ACL 样例使用指导

10.1 总体说明

当前版本支持的样例以及样例在各形态的编译运行指导请参见“[如何获取Sample](#)”。

说明

- 本章主要介绍基于PC包的Sample用例，另外SDK包中也包含一些板端的Sample，Sample路径为smp/a55_linux/source/mpp/sample/svp/svp_npu。
- 本章主要介绍典型的同步推理和异步推理的Sample，更多丰富的Sample样例可查阅样例说明和代码。

10.2 基于 Caffe ResNet-50 网络实现图片分类（同步推理）

10.2.1 样例介绍

10.2.1.1 获取样例

参考“[如何获取Sample](#)”，获取样例代码。

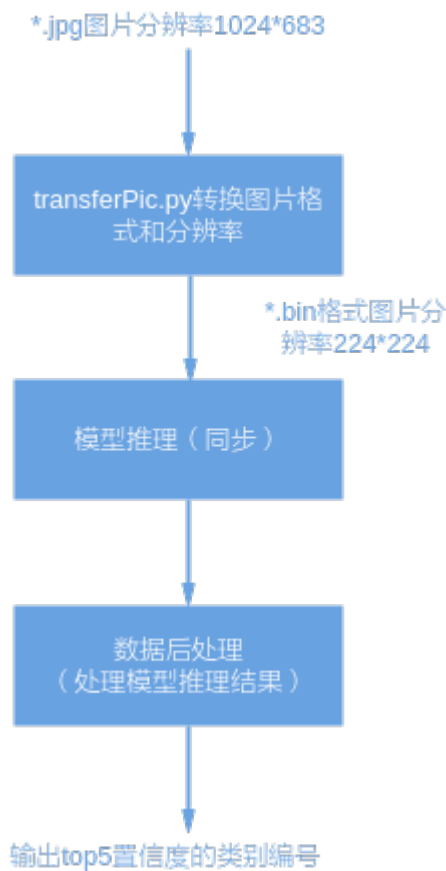
10.2.1.2 功能描述

该样例主要是基于Caffe ResNet-50网络（单输入、单Batch）实现图片分类的功能。

将Caffe ResNet-50网络的模型文件转换为适配SoC的离线模型（*.om文件），在样例中，加载该om文件，对2张*.jpg图片进行同步推理，分别得到推理结果后，再对推理结果进行处理，输出top5置信度的类别标识。



图 10-1 Sample 示例



10.2.1.3 原理介绍

在该Sample中，涉及的关键功能点，如表10-1所示。API接口的详细介绍请参见“SVP ACL API参考”。

说明

Device表示板端环境上的图像分析引擎，SOC场景下，当前只有一个Device。

表 10-1 Sample 中的关键功能点

初始化	- 调用svp_acl_init接口初始化SVP ACL配置。 - 调用svp_acl_finalize接口实现SVP ACL去初始化。
Device管理	- 调用svp_acl_rt_set_device接口指定用于运算的Device。 - 调用svp_acl_rt_get_run_mode接口获取svp_acl软件栈的运行模式，根据运行模式的不同，内部处理流程不同。 - 调用svp_acl_rt_reset_device接口复位当前运算的Device，回收Device上的资源。
Context管理	- 调用svp_acl_rt_create_context接口创建Context。 - 调用svp_acl_rt_destroy_context接口销毁Context。



Stream管理	- 调用svp_acl_rt_create_stream接口创建Stream。 - 调用svp_acl_rt_destroy_stream接口销毁Stream。
内存管理	- 调用svp_acl_rt_malloc接口申请Device上的内存。 - 调用svp_acl_rt_free接口释放Device上的内存。
模型推理	- 调用svp_acl_mdl_load_from_mem接口从存放*.om文件内存中加载模型。 - 调用svp_acl_mdl_execute接口执行模型推理，同步接口。 - 调用svp_acl_mdl_unload接口卸载模型。
数据后处理	提供样例代码，在接口OutputModelResult中，处理模型推理的结果，直接在终端上显示top5置信度的类别编号。 另外，在接口DumpModelOutputResult中，将模型推理的结果写入文件（运行可执行文件后，推理结果文件在应用可执行文件的同级目录下）。 <pre>processModel.OutputModelResult(); processModel.DumpModelOutputResult();</pre>

10.2.1.4 目录结构

样例代码结构如下所示。

— caffe_model	
— resnet50.caffemodel	//测试权重数据,需要按指导获取测试权重数据,放到caffe_model目录下
— resnet50.prototxt	//测试模型网络,需要按指导获取测试模型网络,放到caffe_model目录下
— pixel_mean.txt	//均值文件,数据归一化使用
— data	
— dog1_1024_683.jpg	//测试数据,需要按指导获取测试图片,放到data目录下
— dog2_1024_683.jpg	//测试数据,需要按指导获取测试图片,放到data目录下
— image_ref_list.txt	//保存测试数据对应文件名的文件
— inc	
— model_process.h	//声明模型处理相关函数的头文件
— sample_process.h	//声明资源初始化/销毁相关函数的头文件
— utils.h	//声明公共函数（例如：文件读取函数）的头文件
— model	//存放转换后模型目录
— script	
— transferPic.py	//将*.jpg转换为*.bin,同时将图片从1024*683的分辨率缩放为224*224
— src	
— acl.json	//系统初始化的配置文件
— CMakeLists.txt	//编译脚本
— main.cpp	//主函数,图片分类功能的实现文件
— model_process.cpp	//模型处理相关函数的实现文件
— sample_process.cpp	//资源初始化/销毁相关函数的实现文件
— utils.cpp	//公共函数（例如：文件读取函数）的实现文件
— .project	//工程信息文件,包含工程类型、工程描述、运行目标设备类型等
— CMakeLists.txt	//编译脚本,调用src目录下的CMakeLists文件
— resnet50_config.json	//配置模型生成信息文件
— README.md	



10.2.2 编译及运行应用

步骤1 模型转换

1. 以运行用户登录开发环境。
2. 参考《ATC工具使用指南》中的“使用入门”，执行“准备动作”，包括获取工具、设置环境变量。
3. 准备数据。

从https://gitee.com/ascend/ModelZoo-TensorFlow/tree/master/TensorFlow/contrib/cv/resnet50/ATC_resnet50_caffe_AE 链接中获取 ResNet-50网络的模型文件 (*.prototxt)、预训练模型文件 (*.caffemodel)，并以运行用户将获取的文件上传至开发环境的“样例目录/caffe_model”目录下，也可查看README.md。如果目录不存在，需要自行创建。

请从以下链接获取该样例的输入图片，并以运行用户将获取的文件上传至开发环境的“样例目录/data”目录下。如果目录不存在，需自行创建。

- https://c7xcode.obs.cn-north-4.myhuaweicloud.com/models/aclsample/dog1_1024_683.jpg
- https://c7xcode.obs.cn-north-4.myhuaweicloud.com/models/aclsample/dog2_1024_683.jpg

在“样例目录/data”目录下，创建参考图片列表文件 (image_ref_list.txt) 并在其中填写图片路径data/dog1_1024_683.jpg及data/dog2_1024_683.jpg，各路径换行填写。

4. 将ResNet-50网络转换为适配SoC的离线模型 (*.om文件)。

切换到“样例目录”目录，执行如下命令：

```
atc --save_original_model="true" --soc_version=${soc_version} --log_level="0" --
input_format="NCHW" --weight="./caffe_model/resnet50.caffemodel" --batch_num="256" --
compile_mode="0" --image_list="data:/data/image_ref_list.txt" --input_shape="data:1,3,224,224" --
online_model_type="0" --framework="0" --insert_op_conf="resnet50_insert_op.cfg" --dump_data="0"
--output="./model/resnet50" --model="./caffe_model/resnet50.prototxt"
```

- --soc_version: SoC的版本，用户根据具体板端环境形态，选择对应的取值。
- --insert_op_conf: 插入图像预处理的配置。配置了图像预处理的数据层表示输入数据格式为图像，没有配置的为Feature Map，可参考《ATC工具使用指南》配置。
- --output: 生成的resnet50.om文件存放在“样例目录/model”目录下，建议使用命令中的默认设置，否则在编译代码前，您还需要修改sample_process.cpp中的omModelPath参数值。

```
const char* omModelPath = "../model/resnet50.om";
```

关于各参数的详细解释，请参见《ATC工具使用指南》。

步骤2 编译代码

1. 以运行用户登录开发环境。
2. 设置环境变量，编译脚本CMakeLists.txt通过环境变量所设置的头文件、库文件的路径来编译代码。

如下为设置环境变量的示例，请将“\$HOME/acl”替换为ACLlib组件的实际安装路径。

注意：stub路径中的库仅作为编译使用，在板端环境运行时请使用SDK包中的库。

```
export DDK_PATH=$HOME/acl/ascend-toolkit/svp_latest/x86_64-linux
export NPU_HOST_LIB=$HOME/acl/ascend-toolkit/svp_latest/x86_64-linux/acllib/libxx/stub
```



3. 切换到样例目录，创建目录用于存放编译文件，例如，本文中，创建的目录为“build/intermediates/soc”。

```
mkdir -p build/intermediates/soc
```

4. 切换到“build/intermediates/soc”目录，执行cmake生成编译文件。

“.././../src”表示CMakeLists.txt文件所在的目录，请根据实际目录层级修改。

```
cd build/intermediates/soc
```

您需要根据所安装的交叉编译器，执行对应的cmake命令：

若在开发环境上安装的ACCLib软件包为Linux操作系统SoC-cs形态，则执行如下编译命令：

```
cmake .././../src -Dtarget=board -DCMAKE_BUILD_TYPE=Release -DCMAKE_CXX_COMPILER=aarch64-v01c01-linux-musl-gcc
```

5. 执行make命令，生成的可执行文件main在“样例目录/out”目录下。

```
make
```

步骤3 准备测试数据。

1. 请参考步骤1，在“样例目录/data”目录下，存放测试图片dog1_1024_683.jpg和dog2_1024_683.jpg。
2. 以运行用户登录开发环境。
3. 切换到“样例目录/data”目录下，执行transferPic.py脚本，将*.jpg转换为*.bin，同时将图片从1024*683的分辨率缩放为224*224。在“样例目录/data”目录下生成2个*.bin文件。

```
python3.7.5 ../script/transferPic.py
```

说明

如果执行脚本报错“ModuleNotFoundError: No module named 'PIL'”，则表示缺少Pillow库，请使用**pip3.7.5 install Pillow --user**命令安装Pillow库。

步骤4 运行可执行文件。

1. 将开发环境的样例目录及目录下的文件上传到板端环境。

板端环境的操作系统为Linux时，以root用户将开发环境上的目录及其文件上传到板端环境的指定目录下，例如“/root/acl_resnet50”。

2. 登录板端环境。

板端环境的操作系统为Linux时，以root用户登录板端环境。

3. 设置环境变量。

板端环境的操作系统为Linux时，必须设置如下环境变量，并根据libsvp_acl.so和libsvp_aicpu.so文件所在的路径设置LD_LIBRARY_PATH环境变量。

```
export LD_LIBRARY_PATH=/home/demo/out/lib
```

4. 切换到可执行文件main所在的目录，例如“\$HOME/acl_resnet50/out”，给该目录下的main文件加执行权限。

```
chmod +x main
```

5. 切换到可执行文件main所在的目录，例如“\$HOME/acl_resnet50/out”，运行可执行文件。

```
./main
```

执行成功后，在屏幕上的关键提示信息示例如下，index及其value会根据板端环境的实际情况有所不同：

```
[INFO] acl init success
[INFO] open device 0 success
[INFO] create context success
[INFO] create stream success
[INFO] load model ../model/resnet50.om success
[INFO] create model description success
```



```
[INFO] create model output success
[INFO] start to process file:../data/dog1_1024_683.bin
[INFO] model execute success
[INFO] top 1: index[161] value[0.900391]
[INFO] top 2: index[164] value[0.038666]
[INFO] top 3: index[163] value[0.019287]
[INFO] top 4: index[166] value[0.016357]
[INFO] top 5: index[167] value[0.012161]
[INFO] output data success
[INFO] start to process file:../data/dog2_1024_683.bin
[INFO] model execute success
[INFO] top 1: index[267] value[0.974609]
[INFO] top 2: index[266] value[0.013062]
[INFO] top 3: index[265] value[0.010017]
[INFO] top 4: index[129] value[0.000335]
[INFO] top 5: index[372] value[0.000179]
[INFO] output data success
[INFO] Unload model success, modelId is 1
[INFO] execute sample success
[INFO] end to destroy stream
[INFO] end to destroy context
[INFO] end to reset device is 0
```

----结束

10.3 基于 Caffe ResNet-50 网络实现图片分类（异步推理）

10.3.1 样例介绍

10.3.1.1 获取样例

参考“[如何获取Sample](#)”，获取样例代码。

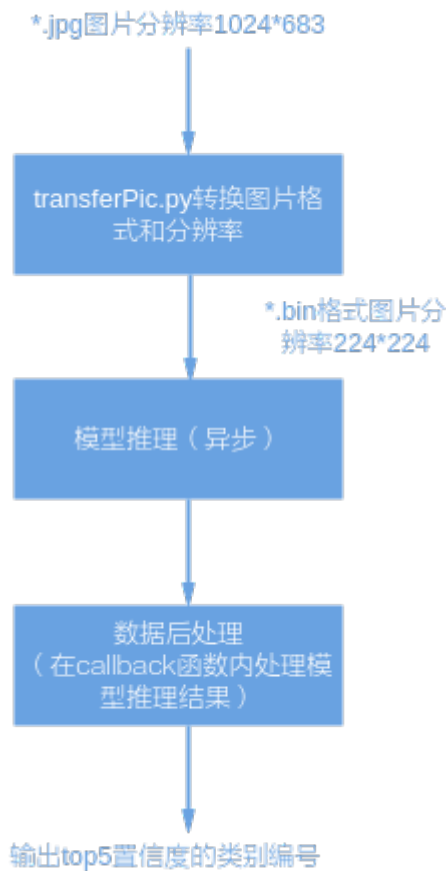
10.3.1.2 功能描述

该样例主要是基于Caffe ResNet-50网络（单输入、单Batch）实现图片分类的功能。

将Caffe ResNet-50网络的模型文件转换为适配SoC的离线模型（*.om文件），在样例中，加载该om文件，对2张*.jpg图片进行n次异步推理（n作为运行应用的参数，由用户配置），分别得到n次推理结果后，再对推理结果进行处理，输出top1置信度的类别标识。



图 10-2 Sample 示例



10.3.1.3 原理介绍

在该Sample中，涉及的关键功能点，如表10-2所示。API接口的详细介绍请参见“SVP ACL API参考”。

说明

Device表示板端环境上的图像分析引擎，SOC场景下，当前只有一个Device。

表 10-2 Sample 中的关键功能点

初始化	- 调用svp_acl_init接口初始化SVP ACL配置。 - 调用svp_acl_finalize接口实现SVP ACL去初始化。
Device管理	- 调用svp_acl_rt_set_device接口指定用于运算的Device。 - 调用svp_acl_rt_get_run_mode接口获取软件栈的运行模式，根据运行模式的不同，内部处理流程不同。 - 调用svp_acl_rt_reset_device接口复位当前运算的Device，回收Device上的资源。



Context管理	- 调用svp_acl_rt_create_context接口创建Context。 - 调用svp_acl_rt_set_current_context接口设置线程的Context。 - 调用svp_acl_rt_destroy_context接口销毁Context。
Stream管理	- 调用svp_acl_rt_create_stream接口创建Stream。 - 调用svp_acl_rt_destroy_stream接口销毁Stream。
内存管理	- 调用svp_acl_rt_malloc_host接口申请Host上内存。 - 调用svp_acl_rt_free_host释放Host上的内存。 - 调用svp_acl_rt_malloc接口申请Device上的内存。 - 调用svp_acl_rt_free接口释放Device上的内存。
模型推理	- 调用svp_acl_mdl_load_from_mem接口从*.om文件加载模型。 - 创建新线程（例如t1），在线程函数内调用svp_acl_rt_process_report接口，等待指定时间后，触发回调函数（例如CallBackFunc，用于处理模型推理结果）。 - 调用svp_acl_rt_subscribe_report接口，指定处理Stream上回调函数（CallBackFunc）的线程（t1）。 - 调用svp_acl_mdl_execute_async接口执行模型推理，异步接口。 - 调用svp_acl_rt_launch_callback接口，在Stream的任务队列中增加一个需要在Host/Device上执行的回调函数（CallBackFunc）。 - 调用svp_acl_rt_synchronize_stream接口，阻塞应用程序运行，直到指定Stream中的所有任务都完成。 - 调用svp_acl_rt_unsubscribe_report接口，取消线程注册，Stream上的回调函数（CallBackFunc）不再由指定线程（t1）处理。 - 模型推理结束后，调用svp_acl_mdl_unload接口卸载模型。
数据后处理	提供样例代码，在接口OutputModelResult中，处理模型推理的结果，直接在终端上显示top5置信度的类别编号。 另外，在接口DumpModelOutputResult中，将模型推理的结果写入文件（运行可执行文件后，推理结果文件在应用可执行文件的同级目录下）。 <pre>processModel.OutputModelResult(); processModel.DumpModelOutputResult();</pre>

10.3.1.4 目录结构

样例代码结构如下所示。

```
├── caffe_model
│   ├── resnet50.caffemodel    //测试权重数据,需要按指导获取测试权重数据，放到caffe_model目录下
│   ├── resnet50.prototxt     //测试模型网络,需要按指导获取测试模型网络，放到caffe_model目录下
│   └── pixel_mean.txt        //均值文件，数据归一化使用
├── data
│   └── dog1_1024_683.jpg     //测试数据,需要按指导获取测试图片，放到data目录下
```



```

├── dog2_1024_683.jpg      //测试数据,需要按指导获取测试图片, 放到data目录下
├── image_ref_list.txt     //保存测试数据对应文件名的文件
├── inc
│   ├── memory_pool.h     //声明内存池处理相关函数的头文件
│   ├── model_process.h   //声明模型处理相关函数的头文件
│   ├── sample_process.h  //声明资源初始化/销毁相关函数的头文件
│   └── utils.h           //声明公共函数（例如：文件读取函数）的头文件
├── model                 //存放转换后模型目录
├── script
│   └── transferPic.py     //将*.jpg转换为*.bin, 同时将图片从1024*683的分辨率缩放为224*224
├── src
│   ├── acl.json          //系统初始化的配置文件
│   ├── CMakeLists.txt    //编译脚本
│   ├── main.cpp          //主函数, 图片分类功能的实现文件
│   ├── memory_pool.cpp   //内存池处理相关函数的实现文件
│   ├── model_process.cpp //模型处理相关函数的实现文件
│   ├── sample_process.cpp //资源初始化/销毁相关函数的实现文件
│   └── utils.cpp         //公共函数（例如：文件读取函数）的实现文件
├── .project              //工程信息文件, 包含工程类型、工程描述、运行目标设备类型等
├── CMakeLists.txt        //编译脚本, 调用src目录下的CMakeLists文件
├── resnet50_config.json  //配置模型生成信息文件
└── README.md

```

10.3.2 编译及运行应用

步骤1 模型转换。

1. 以运行用户登录开发环境。
2. 参考《ATC工具使用指南》中的“使用入门”，执行“准备动作”，包括获取工具、设置环境变量。
3. 准备数据。

从https://gitee.com/ascend/ModelZoo-TensorFlow/tree/master/TensorFlow/contrib/cv/resnet50/ATC_resnet50_caffe_AE 链接中获取ResNet-50网络的模型文件 (*.prototxt)、预训练模型文件 (*.caffemodel)，并以运行用户将获取的文件上传至开发环境的“样例目录/caffe_model”目录下，也可查看README.md。如果目录不存在，需要自行创建。

请从以下链接获取该样例的输入图片，并以运行用户将获取的文件上传至开发环境的“样例目录/data”目录下。如果目录不存在，需自行创建。

- https://c7xcode.obs.cn-north-4.myhuaweicloud.com/models/aclsample/dog1_1024_683.jpg
- https://c7xcode.obs.cn-north-4.myhuaweicloud.com/models/aclsample/dog2_1024_683.jpg

在“样例目录/data”目录下，创建参考图片列表文件（image_ref_list.txt）并在其中填写图片路径data/ dog1_1024_683.jpg及data/ dog2_1024_683.jpg，各路径换行填写。

4. 将ResNet-50网络转换为适配SoC的离线模型 (*.om文件)。

切换到“样例目录”目录，执行如下命令：

```

atc --save_original_model="true" --soc_version=${soc_version} --log_level="0" --
input_format="NCHW" --weight="./caffe_model/resnet50.caffemodel" --batch_num="256" --
compile_mode="0" --image_list="data:/data/image_ref_list.txt" --input_shape="data:1,3,224,224" --
online_model_type="0" --framework="0" --insert_op_conf="resnet50_insert_op.cfg" --dump_data="0"
--output="./model/resnet50" --model="./caffe_model/resnet50.prototxt"

```



- --soc_version: SoC的版本, 用户根据具体板端环境形态, 选择对应的取值。
- --insert_op_conf: 插入图像预处理的配置。配置了图像预处理的Data层表示输入数据格式为图像, 没有配置的为Feature Map, 可参考《ATC工具使用指南》配置。
- --output: 生成的resnet50.om文件存放在“样例目录/model”目录下, 建议使用命令中的默认设置, 否则在编译代码前, 您还需要修改sample_process.cpp中的omModelPath参数值。

```
const char* omModelPath = "../model/resnet50.om";
```

 关于各参数的详细解释, 请参见《ATC工具使用指南》。

步骤2 编译代码。

1. 以运行用户登录开发环境。
2. 设置环境变量, 编译脚本CMakeLists.txt通过环境变量所设置的头文件、库文件的路径来编译代码。

如下为设置环境变量的示例, 请将“\$HOME/acl”替换为ACLib组件的实际安装路径。

注意: stub路径中的库仅作为编译使用, 在板端环境运行时请使用SDK包中的库。

```
export DDK_PATH=$HOME/acl/ascend-toolkit/svp_latest/x86_64-linux
export NPU_HOST_LIB=$HOME/acl/ascend-toolkit/svp_latest/x86_64-linux/acllib/libxx/stub
```

3. 切换到样例目录, 创建目录用于存放编译文件, 例如, 本文中, 创建的目录为“build/intermediates/soc”。

```
mkdir -p build/intermediates/soc
```

4. 切换到“build/intermediates/soc”目录, 执行cmake生成编译文件。

“../././src”表示CMakeLists.txt文件所在的目录, 请根据实际目录层级修改。

```
cd build/intermediates/soc
```

您需要根据所安装的交叉编译器, 执行对应的cmake命令:

若在开发环境上安装的ACLib软件包为Linux操作系统SoC-cs形态, 则执行如下编译命令:

```
cmake ../././src -DCMAKE_CXX_COMPILER=aarch64-v01c01-linux-musl-g++ -
DCMAKE_SKIP_RPATH=TRUE
```

5. 执行make命令, 生成的可执行文件main在“样例目录/out”目录下。

```
make
```

步骤3 准备测试数据。

1. 请参考步骤1, 在“样例目录/data”目录下, 存放测试图片dog1_1024_683.jpg和dog2_1024_683.jpg
2. 以运行用户登录开发环境。
3. 切换到“样例目录/data”目录下, 执行transferPic.py脚本, 将*.jpg转换为*.bin, 同时将图片从1024*683的分辨率缩放为224*224。在“样例目录/data”目录下生成2个*.bin文件。

```
python3.7.5 ../script/transferPic.py
```

说明

如果执行脚本报错“ModuleNotFoundError: No module named 'PIL'”, 则表示缺少Pillow库, 请使用pip3.7.5 install Pillow --user命令安装Pillow库。

步骤4 运行可执行文件。

1. 将开发环境的样例目录及目录下的文件上传到板端环境。



板端环境的操作系统为Linux时，以root用户将开发环境上的目录及其文件上传到板端环境的指定目录下，例如“/root/acl_resnet50_async”。

2. 登录板端环境。

板端环境的操作系统为Linux时，以root用户登录板端环境。

3. 设置环境变量。

板端环境的操作系统为Linux时，必须设置如下环境变量，并根据libsvp_acl.so和libsvp_aicpu.so文件所在的路径设置LD_LIBRARY_PATH环境变量。

```
export LD_LIBRARY_PATH=/home/demo/out/lib
```

4. 切换到可执行文件main所在的目录，例如“\$HOME/acl_resnet50_async/out”，给该目录下的main文件加执行权限。

```
chmod +x main
```

5. 切换到可执行文件main所在的目录，例如“\$HOME/acl_resnet50_async/out”，运行可执行文件。

- 运行可执行文件，不带参数时：

- 执行模型异步推理的次数默认为10次；
- callback间隔默认为，表示1次异步推理后，下发一次callback任务；
- 内存池中的内存块的个数默认为10个。

- 运行可执行文件，带参数时：

- 第一个参数表示执行模型异步推理的次数；
- 第二个参数表示下发callback间隔，参数值为0时表示不下发callback任务，参数值为非0值（例如m）时表示m次异步推理后下发一次callback任务；
- 第三个参数表示内存池中内存块的个数，内存块个数需大于等于模型异步推理的次数。

```
./main
```

执行成功后，在屏幕上的关键提示信息示例如下，index及其value会根据板端环境的实际情况有所不同：

备注：日志中出现“…… get callback task failed”，是因为执行过程中推理还没有开始执行就起线程进行callback处理，此为正常现象。

```
[INFO] ./main param1 param2 param3, param1 is execute model times(default 10), param2 is
callback interval(default 1), param3 is memory pool size(default 10)
[INFO] execute times = 10
[INFO] callback interval = 1
[INFO] memory pool size = 10
[INFO] acl init success
[INFO] open device 0 success
[INFO] create context success
[INFO] create stream success
[INFO] get run mode success
[INFO] load model ../model/resnet50.om success
[INFO] create model description success
[INFO] init memory pool success
[INFO] subscribe report success
[INFO] top 1: index[267] value[0.889648]
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[267] value[0.889648]
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[267] value[0.889648]
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[267] value[0.889648]
```



```
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[267] value[0.889648]
.....
[INFO] top 1: index[161] value[0.836914]
[INFO] top 1: index[267] value[0.889648]
[INFO] model execute success
[INFO] unsubscribe report success
[INFO] unload model success, modelId is 1
[INFO] execute sample success
[INFO] end to destroy stream
[INFO] end to destroy context
[INFO] end to reset device is 0
[INFO] end to finalize acl
```

----结束



11 FAQ

11.1 SVP_NPU 离线模型保护

当前svp_npu中未提供OM的完整性与机密性保护机制，由于模型OM文件中包含了模型的指令及参数等信息，被篡改可能导致推理失败，越界访问等异常，所以需客户对OM的完整性与机密性进行保护。

11.2 第一次推理耗时长的问题

用户在进行第一次推理时，可能会碰到推理时间比后续推理耗时长的问题。这种现象可能是因为linux内存管理的缺页机制引发的。建议用户在申请内存时，对workbuf、output等内存进行初始化为0的操作，这样提前把缺页机制完成，使第一次推理耗时与后续的推理基本一致。

11.3 系统出现 OOM 异常处理

用户业务可能出现占用系统内存较多，从而出现oom killer异常的情况，此时需要扩大系统内存，修改系统内存方法可参考《XXXX SDK安装及升级使用说明》文档中的”地址空间分配与使用”章节。

11.4 任务优先级调度使用约束

相同任务优先级的等待调度原则是等待时间长的任务先执行，但是硬件在等待时长的使用上有一些约束：

- 硬件记录等待时长精度为10.6ms，所以会出现两个不超过10.6ms的时间差任务被认定为等待相同时长的情况，此时硬件会优先调度任务所在流ID编号较小的任务。此场景下如果需要保证任务的执行顺序，建议设置不同的任务优先级。
- 硬件记录等待最大时长为339.2ms，所以会出现两个超过339.2ms的等待时间任务被认定为等待相同时长的情况，此时硬件会优先调度任务所在流ID编号较小的任务。此场景下如果需要保证任务的执行顺序，建议设置合适的优先级提升超时时间。



11.5 任务优先级抢占使用约束

任务优先级抢占是为了保障高优先级任务的优先级执行权，但当前高优先级任务并不能实时打断低优先级任务的执行，而是需要等待低优先级任务执行完当前所在的融合层后才开始执行，所以对高优先级任务的实时性造成一定的影响，特别在带宽和性能压力较大的场景下，会增加丢帧的风险。此场景建议手动设置合理的低优先级任务的帧率。